

UNIVERSIDADE DE SÃO PAULO – USP
ESCOLA POLITÉCNICA
Department of Computer Engineering and Digital Systems (PCS)

Lucas von Ancken Garcia

Attention-Based Deep Learning for Predicting Recurrent Hospital Readmissions

São Paulo

2025

Lucas von Ancken Garcia

Attention-Based Deep Learning for Predicting Recurrent Hospital Readmissions

Original Version

Undergraduate completion monograph submitted to the Department of Computer Engineering and Digital Systems of the Escola Politécnica da Universidade de São Paulo, as part of the requirements for the degree of Computer Engineer.

Area of Concentration: Computer Engineering

Supervisor: Prof. Dr. Antonio Vieira da Silva Neto.

São Paulo

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

von Ancken Garcia, Lucas

Attention-Based Deep Learning for Predicting Recurrent Hospital Readmissions / L. von Ancken Garcia -- São Paulo, 2025.

111 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Aprendizado de máquina 2.Aprendizagem profunda 3.Ciência de dados 4.Modelos analíticos 5.Readmissão hospitalar I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t.

Acknowledgements

I would like to thank my father, my mother, and my brother for supporting me in every moment. Their love, support, and encouragement made this work possible.

I also thank my supervisor for all the guidance, help, and advice throughout this and previous projects. I learned a lot from his dedication and example as a researcher.

I am grateful to all the professors in my department, who contributed to my learning and helped shape my academic path.

Finally, I thank my friends and colleagues for being by my side during my undergraduate years. Their support and friendship made this journey much easier and more meaningful.

“What we know is a drop; what we do not know is an ocean.”
– Isaac Newton

Resumo

As readmissões hospitalares representam um desafio significativo para sistemas de saúde em todo o mundo. Elas podem comprometer a recuperação do paciente, aumentar os custos e demandar recursos clínicos consideráveis. Prever se um paciente será readmitido em até 30 dias pode, portanto, trazer benefícios importantes. Neste trabalho, o foco consiste na previsão desses eventos para pacientes com hospitalizações recorrentes, para os quais há um histórico longitudinal disponível, e investiga-se se mecanismos de atenção conseguem obter uma representação mais rica dessa trajetória passada, de tal modo que o desempenho preditivo seja superior a modelos clássicos. Devido ao número limitado de amostras no conjunto de dados, são exploradas arquiteturas simplificadas baseadas em atenção para reduzir o risco de *overfitting*. O desempenho dessas novas arquiteturas é comparado com modelos tradicionais de aprendizado de máquina: *Random Forest* e *Logistic Regression*. Tais abordagens tradicionais utilizam representações mais simples do passado, baseadas em resumos das internações anteriores, como a média e o desvio padrão dos intervalos de readmissão. Todos os modelos são avaliados em condições desafiadoras, incluindo poucas amostras, históricos curtos e variáveis com baixa correlação com o alvo. Além disso, uma aplicação *web* é implementada utilizando-se as bibliotecas *Streamlit* e *FastAPI* para demonstrar como esses modelos preditivos podem ser integrados a sistemas de informação hospitalar, oferecendo visualização e explicabilidade das previsões.

O projeto segue um método estruturado: planejamento, especificação de requisitos, *design* de arquiteturas, treinamento, ajuste de hiperparâmetros, seleção de variáveis, testes e análise dos resultados. Todos os modelos analisados neste estudo apresentam desempenho baixo a moderado, com valores da métrica *Area Under the Curve* entre 0,61 e 0,63. Apesar dos esforços para desenvolver arquiteturas baseadas em atenção capazes de capturar relações mais complexas sem a ocorrência de *overfitting*, os novos modelos não superam os modelos clássicos. Isso indica que, dadas as limitações do conjunto de dados, mecanismos de atenção não necessariamente resultam em melhor desempenho preditivo, embora seu potencial possa ser mais bem explorado em bases maiores ou mais informativas.

Palavras-chave: Aprendizado de Máquina, Aprendizagem Profunda, Ciência de Dados, Modelos Analíticos, Readmissão Hospitalar.

Abstract

Hospital readmissions pose significant challenges to healthcare systems worldwide. They can significantly compromise patient recovery, increase financial burden, and demand considerable clinical resources. Predicting whether a patient will be readmitted within 30 days can therefore bring important benefits. In this work, the focus is on the prediction of such events for patients with recurrent hospitalizations, for whom a longitudinal history is available, and on the investigation of whether attention mechanisms can obtain a richer representation of this past trajectory, leading to superior prediction performance. Due to the limited number of samples in the dataset, simplified attention-based architectures are explored to reduce the risk of overfitting. Their performances are compared with classical machine learning baselines, namely Random Forest and Logistic Regression. These baselines rely on simpler representations of the past: summary features of past hospitalizations, such as the mean and standard deviation of previous readmission intervals. All models are evaluated under challenging dataset conditions, including limited samples, short patient histories, and features with low correlation to the target. In addition, a web application is also implemented using the libraries *Streamlit* and *FastAPI* to demonstrate how such predictive models could be integrated into hospital information systems, providing visualization and explainability of the predictions.

The project follows a structured method: project planning, requirements specification, model design, training, hyperparameter tuning, feature selection, testing, and results analysis. All models analyzed in this study show low to moderate performance, with Area Under the Curve scores in the range of 0.61 to 0.63. Despite efforts to design attention-based architectures that capture more complex relationships while avoiding overfitting, the newly investigated models do not outperform the classical baselines. This indicates that, under the given data constraints, attention mechanisms do not necessarily translate into improved predictive performance, even though their potential may be better realized in more informative or larger datasets.

Keywords: Machine Learning, Deep Learning, Data Science, Analytical Model, Hospital Readmission.

List of Figures

Figure 1 – Comparison of RNN, LSTM, and GRU architectures. Source: Sit et al. (2020)	25
Figure 2 – Gantt chart for project planning	37
Figure 3 – High-level system architecture	42
Figure 4 – Data preprocessing pipeline	45
Figure 5 – Distribution of the number of prior hospital visits before last event . .	47
Figure 6 – Distribution of diseases in the selected cohort	47
Figure 7 – Distribution of the number of diseases per patient	48
Figure 8 – Pearson correlation of binary features to target variable	49
Figure 9 – Rank biserial correlation of continuous features to target variable . . .	49
Figure 10 – Shared MLP classification head architecture	51
Figure 11 – Overview of the GRU-based model architecture.	52
Figure 12 – Overview of the SimpleAttnPool model architecture.	53
Figure 13 – Overview of the SimpleAttnPoolCurrQuery model architecture.	54
Figure 14 – Overview of the MultiHead-Attention-Pooling model architecture. . . .	55
Figure 15 – Use case diagram of the web application	63
Figure 16 – Home screen - Frontend	64
Figure 17 – Data upload interface - Frontend	64
Figure 18 – Cohort predictions table - Frontend	64
Figure 19 – Cohort overview - Frontend	65
Figure 20 – Specific patient overview - Frontend	65
Figure 21 – Patient explainability - Frontend	66
Figure 22 – Model performance metrics - Frontend	66
Figure 23 – Execution of unit tests	68
Figure 24 – SHAP feature importances – Random Forest Baseline	73
Figure 25 – Feature set selection – SimpleAttnPool example	75
Figure 26 – IG attributions for current features – SimpleAttnPool example	76
Figure 27 – IG attributions for longitudinal features – SimpleAttnPool example . .	76
Figure 28 – Comparison of AUC scores	78
Figure 29 – IG attributions plots for high-risk patient	82
Figure 30 – IG attributions tables for high-risk patient	82
Figure 31 – IG attributions plots for low-risk patient	83
Figure 32 – IG attributions tables for low-risk patient	84
Figure 33 – Evolution of selected clinical features across the last four admissions . .	85
Figure 34 – Attention weights assigned to the last four admissions of the patient . .	85
Figure 35 – API response time for single-patient explanation	86

Figure 36 – API response time for cohort prediction	87
Figure 37 – Description of project tasks present in the Gantt chart	98
Figure 38 – Sequence diagram for the " <i>Explore Cohort Predictions</i> " and " <i>View Cohort Model Performance</i> " use cases	102
Figure 39 – Sequence diagram for the " <i>Explain Single Patient</i> " use case	103
Figure 40 – Confusion matrix for the model used by the API on test data	104
Figure 41 – Calibration curve for the model used by the API on test data	105
Figure 42 – Predicted probability distribution for the model used by the API on test data	105

List of Tables

Table 1 – Functional Requirements	39
Table 2 – Non-functional requirements	40
Table 3 – System components overview	43
Table 4 – Input data frames, required columns (with types) and descriptions . . .	44
Table 5 – Hyperparameters for model training	58
Table 6 – Hyperparameters of model architectures	59
Table 7 – Hyperparameters of classical ML baseline models	59
Table 8 – End-to-End Test Cases (Part 1)	69
Table 9 – End-to-End Test Cases (Part 2)	70
Table 10 – Baseline classifiers: training and cross-validation performance metrics . .	71
Table 11 – Baseline classifiers: test performance metrics	71
Table 12 – Deep learning: feature selection results	74
Table 13 – Deep learning: model tuning results	77
Table 14 – Deep learning: training performance metrics	78
Table 15 – Deep learning: test performance metrics	78
Table 16 – Deep learning: two-stage approach test performance metrics	80
Table 17 – Main libraries used in this project	99
Table 18 – Numerical features with definitions and types	100
Table 19 – Categorical and binary features with definitions and types	101
Table 20 – API specification – <code>PredictionPayload</code> fields	108
Table 21 – API specification – <code>AdmissionsRow</code> fields	108
Table 22 – API specification – <code>DiagnosesRow</code> fields	108
Table 23 – API specification – <code>ICUStayRow</code> fields	108
Table 24 – API specification – <code>PrescriptionRow</code> fields	109
Table 25 – API specification – <code>PatientRow</code> fields	109
Table 26 – API specification – <code>ProcedureRow</code> fields	109
Table 27 – API specification – <code>TargetRow</code> fields	109
Table 28 – API specification – <code>PredictionBatchEnvelope</code> fields	109
Table 29 – API specification – <code>PredictionBody</code> fields	110
Table 30 – API specification – <code>PredictionMetadata</code> fields	110
Table 31 – API specification – <code>PredictionMetrics</code> (optional)	110
Table 32 – API specification – <code>ExplainSinglePatientEnvelope</code> fields	111
Table 33 – API specification – <code>InputFeatures</code> fields	111
Table 34 – API specification – <code>ExplanationBody</code> fields	111
Table 35 – API specification – <code>FeatureAttributionSplit</code> fields	111
Table 36 – API specification – <code>FeatureAttributionRow</code> fields	111

List of Abbreviations and Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve (Receiver Operating Characteristic)
AUC@30	Area Under the Curve at 30 Days
Avg	Average
CCI	Charlson Comorbidity Index
COPD	Chronic Obstructive Pulmonary Disease
CSV	Comma-Separated Values
Curr.	Current
CV	Cross-Validation
DT-HMM	Discrete-Time Hidden Markov Model
E2E	End-to-End
Feat.	Feature
FN	False Negative
FP	False Positive
FR	Functional Requirement
GRU	Gated Recurrent Unit
HF	Congestive Heart Failure
HMM	Hidden Markov Model
Hosp.	Hospitalization
ICD-9	International Classification of Diseases 9th Revision
ICU	Intensive Care Unit
i.i.d.	Independent and Identically Distributed

IG	Integrated Gradients
Long.	Longitudinal
LSTM	Long Short-Term Memory
MIMIC-III	Medical Information Mart for Intensive Care III
ML	Machine Learning
MLP	Multi-Layer Perceptron
NFR	Non-Functional Requirement
RETAIN	Reverse Time Attention
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
TP	True Positive
TN	True Negative

List of Symbols

$\mathbb{N}$	Set of natural numbers
$\mathbb{N}^+$	Set of natural numbers excluding zero
$\mathbb{R}$	Set of real numbers
$\mathbb{R}^+$	Set of positive real numbers
N	Number of past (historical) visits in a patient's sequence, $N \in \mathbb{N}^+$
$\mathbf{s}$	Static covariate vector, $\mathbf{s} \in \mathbb{R}^m$, describing patient-specific information
T	Clinically-defined fixed prediction window
v_k	Hospital visit $k \in \mathbb{N}^+$ in a patient's sequence of visits
$\mathbf{x}_k$	Covariate vector of dimension d , $\mathbf{x}_k \in \mathbb{R}^d$, describing the context of visit v_k
y_k	Time between visit v_k and the next visit v_{k+1}
y_{N+1}	Time between visit v_k and the next visit v_{N+1} to be predicted
y'_k	Binary variant of the time to next visit, equal to 1 if y_k exceeds the clinical threshold T and 0 otherwise
$\mathcal{H}_N$	Patient's visit history up to visit N : $\{(\mathbf{x}_k, y_k)\}_{k=1}^N$
$\mathcal{P}$	Set of patients in the dataset
$\mathcal{V}^{(i)}$	Ordered sequence of visits for patient i
$L(\cdot)$	Likelihood function
$\ell(\cdot)$	Log-likelihood function, $\log L(\cdot)$
$P(\cdot)$	Probability

Contents

1	INTRODUCTION	16
1.1	Motivation	16
1.2	Objectives	17
1.3	Rationale	18
1.4	Mathematical Formulation of the Problem	19
1.5	Outline	20
2	THEORETICAL BACKGROUND	22
2.1	Basic Machine Learning Background	22
2.1.1	Logistic Regression	22
2.1.2	Random Forests	22
2.1.3	Binary Classification Metrics	23
2.2	Deep Learning and Attention Mechanisms	23
2.2.1	Multilayer Perceptron (MLP)	24
2.2.2	Sequential Deep Learning Models	24
2.2.3	Attention Mechanisms	26
2.3	Explainability in Machine Learning	26
2.3.1	SHAP Values	27
2.3.2	Integrated Gradients Attributions	28
2.4	Literature Review	28
2.4.1	Longitudinal Healthcare Data Modeling	29
2.4.2	Hospital Readmission Prediction	30
2.4.3	Attention Mechanisms in Longitudinal Healthcare Data	30
2.4.3.1	Attention Combined with RNNs	31
2.4.3.2	Attention in Transformer-Style Architectures (No Recurrence)	31
2.4.3.3	Hierarchical Attention: Visit and Feature Levels	32
3	WORK METHOD	34
3.1	Project Phases	34
3.1.1	Preparation	34
3.1.2	Analytics Backend	35
3.1.3	Frontend	35
3.1.4	Tests	35
3.1.5	Results Analysis	36
3.1.6	Documentation	36
3.2	Project Planning	36

4	REQUIREMENTS SPECIFICATION	38
4.1	Functional Requirements	38
4.2	Non-Functional Requirements	39
5	PROJECT DEVELOPMENT	41
5.1	Used Technologies	41
5.2	Design and Implementation	41
5.2.1	System Architecture	42
5.2.1.1	Input Data	43
5.2.2	About the Datasets	44
5.2.3	Data Preprocessing Pipeline	45
5.2.3.1	Feature Engineering	45
5.2.3.2	Filtering and Selection	46
5.2.3.3	Stratified Data Splitting	46
5.2.3.4	Normalization and Missing-Value Input Entry	46
5.2.4	Exploratory Data Analysis (EDA) - Challenges of the Dataset	46
5.2.5	Classical ML Baselines	50
5.2.6	Deep Learning Architectures	50
5.2.6.1	GRUNet	52
5.2.6.2	Simple Attention Pooling: SimpleAttnPool	52
5.2.6.3	Simple Attention Pooling with Current Query: SimpleAttnPoolCurrQuery	53
5.2.6.4	MultiHead-Attention-Pooling: MultiHeadAttnPool	54
5.2.7	Model Training	56
5.2.8	Model Evaluation	57
5.2.9	Hyperparameter Tuning	58
5.2.10	Feature Selection	60
5.2.11	Additional Experiment: Two-Stage Training Approach	61
5.2.12	API for Inference	61
5.2.12.1	POST "/predict"	62
5.2.12.2	POST "/explain_single_patient"	62
5.2.13	Web Application (Frontend)	62
5.3	Testing	67
5.3.1	Unit Tests	67
5.3.2	End-To-End Tests	68
6	RESULTS ANALYSIS	71
6.1	Classical ML Baselines Results	71
6.2	Deep Learning Feature Selection	74
6.3	Deep Learning Model Tuning	77
6.4	Deep Learning Training and Evaluation Results	78

6.5	Two-Stage Approach Results	80
6.6	Assessment of Model Performance Functional Requirements	81
6.7	Examples of Explainability Results	81
6.7.1	IG Attributions for Individual Predictions	81
6.7.2	Example of Attention Weights Over Time	84
6.8	API Response Times	85
6.9	Ease-of-Use of the Web Application	87
7	FINAL REMARKS	89
7.1	Conclusion	89
7.2	Summary of Contributions and Lessons Learned	90
7.3	Future Work	91
	REFERENCES	93
	APPENDICES	97
	APPENDIX A – PROJECT PLANNING DETAILS	98
	APPENDIX B – PYTHON PACKAGES USED IN THE PROJECT	99
	APPENDIX C – FEATURES DESCRIPTION	100
	APPENDIX D – WEB APPLICATION SEQUENCE DIAGRAMS	102
	APPENDIX E – FURTHER DETAILS OF API MODEL PERFORMANCE	104
	APPENDIX F – HOSPITAL READMISSION PREDICTION API SPECIFICATION	107

1 Introduction

In this chapter, the motivation of the work is presented: why predicting hospital readmissions is important and what potential benefits such predictions can offer (section 1.1). Then, the specific objectives of this project are described, including how it builds upon previous work and what new approaches will be explored (section 1.2). The rationale behind the use of Machine Learning (ML) techniques and the reason behind the choice of the modeling approach to be explored in this work are discussed in section 1.3. In section 1.4, the problem is mathematically defined. Finally, an overview of the structure of the report is provided (section 1.5).

1.1 Motivation

In healthcare, predicting whether a patient will likely experience a recurrent adverse outcome is a central challenge. Early identification of risk can support timely interventions and improve patient outcomes as well as the efficiency of healthcare systems. Among such problems, hospital readmissions stand out as a major concern.

Hospital readmission is typically defined as the unplanned return of a patient to the hospital within a fixed period after discharge—most commonly 30, 60 or 90 days (Dhaliwal; Dang, 2024). Such events are frequent and costly. In the United States, the 30-day all-cause readmission rate has remained around 14% in recent years (Jiang; Hensche, 2023), with 3.8 million readmissions in 2018 alone and average costs of 16,300 USD, which are 12.4% higher than the initial admission (Weiss; Jiang, 2021). Chronic diseases such as Chronic Obstructive Pulmonary Disease (COPD) and congestive Heart Failure (HF) show even higher recurrence rates, with more than half of patients readmitted within six months to one year (Shah et al., 2016).

Reducing readmissions is critical, as they cause several problems (Dhaliwal; Dang, 2024):

- **Patient impact:** Emotional and physical strain, diminished confidence in care, and heightened risk of complications.
- **Financial burden:** Significant costs for patients, hospitals, and health systems.
- **Resource pressure:** Fewer available hospital beds and shifting clinical resources away from other patients.

A significant proportion of these readmissions are considered avoidable. A systematic review estimated that around 27% of readmissions could be prevented, with some studies reporting much higher values depending on the context (Walraven et al., 2011). Therefore, solutions for reducing such adverse events are feasible and urgent.

1.2 Objectives

This project is built upon the foundation established in *Predictive Modeling of Recurrent Health Events: Applications to Hospital Readmission and Drug Relapse* (Garcia, 2025b). While the previous study emphasized interpretable, statistically grounded models, the present work corresponds to an extension of this line of research, focusing on attention-based deep learning approaches to extract relevant patterns in the trajectory of a patient’s past hospitalizations.

In the earlier work, a Duration-Time Hidden Markov Model (DT-HMM) was proposed as a feature extractor for patient trajectories. The extracted representations were then used in two predictive tasks: binary classification of hospital readmission and survival modeling (Kleinbaum; Klein, 2012) for estimating the time until readmission. The emphasis was placed on computational efficiency and interpretability, favoring lightweight models based on statistical principles. However, such a DT-HMM-based feature extractors did not provide significant performance gains compared to simple historical summary features, i.e., the baseline approach adopted in that work (Garcia, 2025b).

Since the HMM-based features did not yield clear improvements over simple baselines, the **focus of the current project is shifted to deep learning, exploring attention mechanisms as feature extractors**. The rationale behind this choice is described in section 1.3. The **final goal remains the same: to predict hospital readmission within 30 days of discharge using longitudinal patient data**.

Although the importance of explainability for clinical decision-support applications remains, this requirement is relaxed slightly. This relaxation is an attempt to capture new patterns that might not have been captured by the simpler models exercised in previous work (Garcia, 2025b).

Additionally, the development of a web application is also within the project’s scope in order to illustrate the usage of the model, including the visualization and explainability of the model’s predictions. The purpose of this interface is to illustrate how the proposed attention-based models could be integrated into the information system of hospitals to support clinicians. The rationale of such system is also discussed in section 1.3.

Overall, the main objectives and additions of this work compared to the previous work (Garcia, 2025b) are summarized below.

Reused from previous work:

1. Reuse the overall motivation and problem definition;
2. Apply the same dataset, preprocessing, and feature extraction steps;
3. Incorporate results of the Exploratory Data Analysis (EDA);
4. Reuse baseline models as benchmarks for comparison.

Objectives in this work (additions):

1. Design and evaluate new simplified attention-based architectures with fewer parameters than common approaches in the literature in order to avoid overfitting in scenarios with limited data;
2. Compare the performance of the new architectures against the baseline models from previous work to assess if the complexity added by the attention mechanism yields significant improvements;
3. Develop a prototype frontend interface to demonstrate how the model could be used and integrated into a hospital information system.

1.3 Rationale

Machine learning (ML) has emerged as a powerful tool for supporting clinical decision-making. By analyzing large datasets of patient information, ML models can learn patterns in patients' longitudinal data that are difficult for humans to assess. In the context of hospital discharge and post-care treatment, clinicians must evaluate large volumes of clinical and non-clinical information – laboratory results, comorbidities, medical examinations, patient complaints – often under time and resource constraints. This process is highly complex and partly subjective, increasing the likelihood of premature or unsafe decisions that can lead to early readmissions (Võ et al., 2025). Therefore, developing a predictive ML model, as stated in section 1.2, is a promising approach to assist clinicians in identifying patients at high risk of readmission, enabling more informed discharge planning and follow-up care.

Beyond the prediction itself, model explainability is crucial: transparent and explainable Artificial Intelligence (AI) can improve patient outcomes, reduce medical errors, and enhance trust in AI-driven care (Kumar et al., 2024). A system that allows the visualization of the model's predictions for readmissions and the factors driving it could be a valuable tool for multiple stakeholders:

1. **Physicians and Care Teams:** make timely follow-ups, prioritize high-risk patients, and adjust treatments;
2. **Patients:** understand better their own risk and adhere more closely to treatment and lifestyle changes;
3. **Hospital Administrators:** anticipate readmission peaks, optimize staff allocation, and utilize scarce resources more effectively.

Hence, incorporating explainability into the system, as emphasized in section 1.2, is an essential part of the ML-based solution.

The investigation of attention mechanisms is motivated by prior work that has demonstrated their effectiveness across domains such as natural language processing, time-series forecasting, and healthcare applications (Vaswani et al., 2023; Choi et al., 2017; Song et al., 2017). Their ability to focus on the most relevant parts of the input makes them particularly suitable for modeling complex patient trajectories (Choi et al., 2017; Song et al., 2017), in which certain past events or time periods may be more indicative of future outcomes than others. Moreover, attention weights enhance explainability by highlighting which aspects of the input most influenced a prediction. By leveraging attention mechanisms, the model can capture patterns in patient histories that traditional models may not, while also providing explanations that are critical for clinical decision support.

1.4 Mathematical Formulation of the Problem

In this section, the hospital readmission prediction task based on the longitudinal patient data is formally defined. The notation and base definitions were adapted from the previous work (Garcia, 2025b).

Problem Statement: Each patient is represented by a sequence of N historical hospital visits as defined in Equation 1.1

$$\mathcal{H}_N = \{v_k\}_{k=1}^N = \{(\mathbf{x}_k, y_k)\}_{k=1}^N \quad (1.1)$$

On Equation 1.1:

- v_k denotes the k -th visit, described by:
 - $\mathbf{x}_k \in \mathbb{R}^d$: a d -dimensional covariate vector¹ containing observed clinical and treatment-related variables observed at visit k ;

¹ Covariate vector refers to explanatory variables used by the model for the prediction.

- $y_k \in \mathbb{R}^+$: the time interval until the next visit v_{k+1} , or equivalently, the readmission time after visit v_k (e.g., time between the discharge and the next hospital encounter).
- N is the number of past visits available for the patient.

The prediction task is defined as estimating the probability that the next readmission time y_{N+1} occurs within a clinically defined time window T (e.g., 30 days) given the patient's history $\mathcal{H}_N$, covariates describing its current visit $\mathbf{x}_{N+1}$, and the patient's static covariates $\mathbf{s}$ (demographic information, for example), as stated in Equation 1.2:

$$P(y_{N+1} < T \mid \mathcal{H}_N, \mathbf{x}_{N+1}, \mathbf{s}). \quad (1.2)$$

Thus, the target variable is binary:

$$y'_{N+1} = \begin{cases} 1 & \text{if the patient is readmitted within } T \text{ days of discharge,} \\ 0 & \text{otherwise.} \end{cases}$$

Therefore, the modeling task is to learn from sequences of patient visits how to predict short-term hospital readmission risk based on both historical and most recent visit information.

For such learning, the models receive a dataset containing a set of patients $\mathcal{P}$ during training. Each patient i is described by an ordered sequence of visits $\mathcal{V}^{(i)}$ and static covariates $\mathbf{s}^{(i)}$. The length of the sequence is not fixed, varying depending on the patient. The last visit available in $\mathcal{V}^{(i)}$ is used as target during training, such that it contains the prediction target $y_{N+1}^{(i)}$.

1.5 Outline

This monograph is organized into seven main chapters. Its overall structure is as follows:

- **Chapter 1 — Introduction:** Covers the motivation for the work, the research objectives, the rationale of the chosen approach, the mathematical formulation of the problem, and an overview of the document structure;
- **Chapter 2 — Theoretical Background:** Comprises the review of the fundamental concepts required for the development of the project, including machine learning and deep learning methods, the attention mechanism, and the state of the art on hospital readmission prediction;

- **Chapter 3 — Work Method:** Outlines the project execution method, including its phases and its planning, the latter illustrated by a Gantt chart;
- **Chapter 4 — Requirements Specification:** Provides the system's functional and non-functional requirements, serving as basis for implementation and evaluation;
- **Chapter 5 — Project Development:** Dwells in detail on the adopted technologies, as well as on the design and on the implementation of the target system;
- **Chapter 6 — Analysis of Results:** Provides the results from the conducted experiments and their analyses. Themes such as model tuning, training, and evaluation, as well as requirements fulfillment, are within the scope of this chapter;
- **Chapter 7 — Conclusion:** Summarizes the final remarks by highlighting its main contributions and providing directions for future work.

2 Theoretical Background

In this chapter, the main concepts, models, and prior research that support the development of this work are presented. It is divided into four main parts: basic machine learning background, deep learning, explainability on AI models and attention mechanisms, and a literature review of related studies in healthcare focusing hospital readmission prediction.

2.1 Basic Machine Learning Background

A binary classification problem consists of predicting the label $y \in \{0, 1\}$ of an observation $\mathbf{x} \in \mathbb{R}^p$ from a given dataset. The objective is to learn a mapping $f : \mathbb{R}^p \rightarrow \{0, 1\}$ that generalizes well to unseen data. Given the input features $\mathbf{x}$, the model outputs a probability score $\hat{p} = P(y = 1 \mid \mathbf{x})$. A threshold (commonly 0.5) is then applied to convert this probability into a binary prediction $\hat{y}$.

The goal is to minimize a loss function that quantifies the discrepancy between the predicted labels $\hat{y}$ and the true labels y . A common choice for binary classification is the binary cross-entropy loss defined in Equation 2.1:

$$\mathcal{L}(y, \hat{p}) = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})] \quad (2.1)$$

2.1.1 Logistic Regression

Logistic regression models the log-odds of the probability of the positive class as a linear function of the predictors:

$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \beta_0 + \mathbf{x}^\top \boldsymbol{\beta}. \quad (2.2)$$

It can be interpreted as linear regression applied to the logits. To improve generalization and interpretability, regularization methods such as L1 (lasso) or L2 (ridge) penalties can be applied, promoting sparsity in the estimated coefficients and reducing the impact of outliers.

2.1.2 Random Forests

Random Forests are ensemble learners composed of multiple decision trees. Each tree is trained on a bootstrap sample of the data, and predictions are obtained by majority voting across the ensemble. At each split in a tree, only a random subset of features is considered, which increases diversity among trees. Splits are chosen to maximize class

purity in the resulting nodes, typically by minimizing the Gini impurity or entropy (Hastie; Tibshirani; Friedman, 2009).

2.1.3 Binary Classification Metrics

In the context of binary classification, several key metrics are commonly used to evaluate model performance. These metrics are derived from the confusion matrix, which summarizes the counts of true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions made by the model. The definitions of these metrics are as follows ((Hastie; Tibshirani; Friedman, 2009)):

- **Accuracy** (Equation 2.3): The proportion of correctly classified instances among all instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.3)$$

- **Precision** (Equation 2.4): The proportion of true positive predictions among all positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.4)$$

- **Recall (Sensitivity)** (Equation 2.5): The proportion of true positive predictions among all actual positive instances.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.5)$$

- **F1-score** (Equation 2.6): The harmonic mean of precision and recall, providing a balance between the two metrics.

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.6)$$

- **Area Under the Curve (AUC) - Receiver Operating Characteristic (ROC)**: A measure of a model's ability to distinguish between classes across different threshold settings. It plots the true positive rate (recall) against the false positive rate (1 - specificity). The minimally acceptable threshold of AUC is 0.5, which represents the behavior of fully random classifier.

2.2 Deep Learning and Attention Mechanisms

In this section, a brief overview of the main deep learning architectures and attention mechanisms concepts relevant to the project's development are provided. For further details, comprehensive overview works such as the one provided by Perumal et al. (2024) are suggested.

2.2.1 Multilayer Perceptron (MLP)

Multilayer Perceptron (MLP) is a type of feedforward artificial neural network. Its primary use is in supervised learning tasks, including both classification and regression. In such settings, the samples are assumed to be independent and identically distributed (i.i.d.) (Perumal et al., 2024).

A typical MLP consists of input, hidden, and output layers. Each layer is made up of nodes (or neurons) that are fully connected to the nodes in the subsequent layer. The connections between nodes have associated weights that are learned during training (Perumal et al., 2024). The output of a neuron is computed by applying a weighted sum of its inputs followed by a non-linear activation function as stated in Equation 2.7:

$$o_j = f\left(\sum_i w_{ij}x_i + b_j\right) \quad (2.7)$$

On Equation 2.7 o_j is the output of neuron j , x_i are the inputs, w_{ij} are the weights, b_j is the bias term, and f is the activation function.

Activation functions f introduce non-linearity into the model, enabling it to learn data patterns other than linear relations. Common activation functions include, for example:

- Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$;
- ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$;
- Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$.

2.2.2 Sequential Deep Learning Models

Unlike MLPs, which assume i.i.d. data, sequential models are designed to handle ordered data with temporal dependencies (Perumal et al., 2024). Common architectures include:

- Recurrent Neural Networks (RNNs);
- Long Short-Term Memory (LSTM) networks;
- Gated Recurrent Units (GRUs).

Such models work by processing input sequences one element at a time. At each step, they update a hidden state vector h_t . Such representation is computed by combining information from the current input x_t and the previous hidden state h_{t-1} through linear transformations and nonlinear activations (Perumal et al., 2024). This allows the model

to transform variable-length input sequences into a fixed-size representation vector h_T obtained at the end of the sequence, so that it summarizes all relevant past information for downstream tasks.

RNNs correspond to the simplest form of sequential deep learning models. They have a single hidden state vector h_t and an output vector o_t , which is computed from h_t by a MLP. However, they struggle to retain information over long sequences due to issues such as vanishing gradients (Perumal et al., 2024). LSTMs and GRUs were introduced to address these limitations by incorporating gating mechanisms. These gates help the model decide what information to keep, forget, or output at each time step, enabling better modeling of long-term dependencies.

LSTM networks have the most complex architecture among the herein listed sequential models. They include two hidden state representations: one related to short-term memory (h_t) and another related to long-term memory (c_t). These representations are updated through three gates: input gate, forget gate, and output gate (Perumal et al., 2024).

GRUs are in the middle in terms of complexity. They combine the forget and input gates into a single update gate (Perumal et al., 2024). This results in fewer parameters than LSTMs while still effectively capturing long-term dependencies. Unlike LSTM, the GRU does not include a separate cell state c_t .

Figure 1 (Sit et al., 2020) illustrates the architectures of RNN, LSTM, and GRU cells. One can observe how LSTM and GRU introduce additional gates and state representations compared to the simpler RNN structure. All RNNs, LSTMs, and GRUs follow the structure of Figure 1 (Sit et al., 2020).

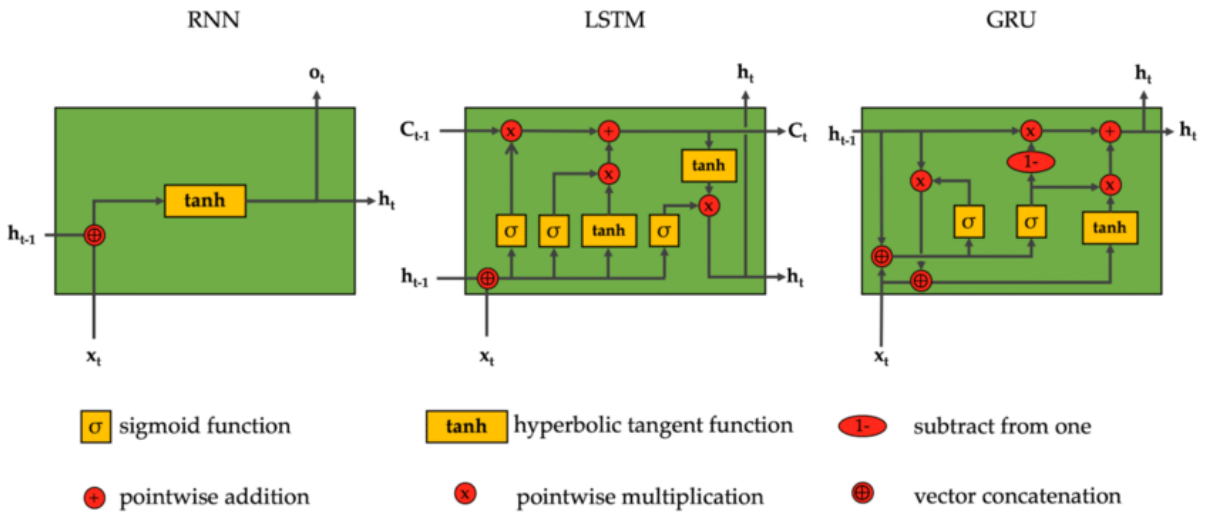


Figure 1 – Comparison of RNN, LSTM, and GRU architectures. Source: Sit et al. (2020)

2.2.3 Attention Mechanisms

Sequential models such as RNNs and GRUs transform a sequence into a single fixed-size vector. This can make it difficult for the model to keep detailed information about earlier elements in the sequence, especially when some parts are more relevant than others for the prediction. Attention mechanisms address this limitation by allowing the model to focus on the most important parts of the input sequence when creating its final representation (Vaswani et al., 2023).

The main idea of attention is to compute a weighted average of input vectors, where each weight shows how important that element is. Given a *query* vector $\mathbf{q}$, a set of *key* vectors $\{\mathbf{k}_i\}$, and corresponding *value* vectors $\{\mathbf{v}_i\}$, the attention mechanism calculates a score for each element that measures how well its key matches the query. These scores are then normalized with a softmax function and computed as stated in Equation 2.8:

$$\alpha_i = \frac{\exp(\text{score}(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(\text{score}(\mathbf{q}, \mathbf{k}_j))}, \quad (2.8)$$

On Equation 2.8, the *score* function is a measure of similarity between the query and key vectors, such as dot product or cosine similarity. When using dot product, the score is computed as in Equation 2.9 ($\top$ refers to the transpose matrix):

$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i. \quad (2.9)$$

The final context vector is obtained by combining the value vectors using these weights as stated in Equation 2.10:

$$\mathbf{z} = \sum_i \alpha_i \mathbf{v}_i. \quad (2.10)$$

The resulting vector $\mathbf{z}$ summarizes the sequence, giving more influence to the elements that are most relevant to the query.

In this work, simpler attention mechanisms are used instead of more complex ones. For instance, it is possible to combine sequential models with attention, in which the query is derived from the current hidden state of the RNN, and the keys and values are the hidden states from all previous time steps. Examples of such use in healthcare are briefly presented in section 2.4.3. Transformer architectures, which rely entirely on attention mechanisms without recurrence, are also discussed in section 2.4.3.

2.3 Explainability in Machine Learning

In the context of ML, explainability refers to the ability of model to make decisions easy for humans to understand by showing why it made a particular prediction (Kumar

et al., 2024). Explainable models are specially important because they bring the level of transparency and insight required to support sound clinical decisions (Kumar et al., 2024).

In this section, a brief overview of two explainability methods used in this work is provided: SHapley Additive exPlanations (SHAP) and Integrated Gradients (IG) attributions. The first, already explored in the previous work (Garcia, 2025b), is used to measure the impact of the features on the classical ML baseline models analyzed in section 6.1. The second is used in this work to explain the deep learning models and perform feature selection on them (see section 6.2).

It is important to mention that attention mechanisms, explained in section 2.2.3, also help bring explainability to deep learning models by highlighting which part of input data were most relevant or influential in a model's decision-making process (Kumar et al., 2024).

2.3.1 SHAP Values

SHAP values are a method to explain the output of machine learning models (Lundberg; Lee, 2017) and have been used in biomedical applications to allow better explainability of model predictions for clinical decision-making (Lundberg et al., 2018).

The central SHAP is to measure how much each feature contributes to a prediction by comparing the model's output with and without that feature across many possible subsets of features (Lundberg; Lee, 2017). Formally, the *Shapley* value $\phi_i(f, \mathbf{x})$ for feature i and model f at input vector $\mathbf{x}$ is computed as stated in Equation 2.11.

$$\phi_i(f, \mathbf{x}) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} (f(S \cup \{i\}, \mathbf{x}) - f(S, \mathbf{x})), \quad (2.11)$$

On Equation 2.11, N is the set of all features and $f(S)$ is the model prediction using only the subset S . The notation $|\cdot|$ denotes the size of the subset. This definition, inherited from cooperative game theory, averages the marginal contribution of i across all possible coalitions of features.

Intuitively, a SHAP value indicates how much the prediction for a given sample moves away from the average prediction when a particular feature is taken into account, as stated in Equation 2.12.

$$f(\mathbf{x}) = \phi_0 + \sum_{i=1}^M \phi_i, \quad (2.12)$$

On Equation 2.12, ϕ_0 is the baseline (expected) model output and the ϕ_i values distribute the difference across the M features. Positive values mean the feature increases the predicted risk (or decreases survival time), while negative values indicate the opposite.

When aggregating SHAP values across a dataset composed of N samples, the overall importance of a feature i (i.e., I_i) is often computed as the mean absolute value, which reflects how much that feature contributes on average, regardless of direction, as described in Equation 2.13.

$$I_i = \frac{1}{N} \sum_{j=1}^N |\phi_i(\mathbf{x}^{(j)})|, \quad (2.13)$$

2.3.2 Integrated Gradients Attributions

The IG method is a gradient-based attribution approach that explains a model's prediction by quantifying how each input feature influences the output (Kokhlikyan et al., 2020). The central idea is to compute the gradient of the model's output with respect to each input feature while moving along a straight path from a neutral *baseline* input (such as zeros or the mean values of features computed over the training set) to the actual input. By integrating these gradients along the path, the method produces an attribution score indicating the contribution of each feature.

The attribution for feature i is defined on Equation 2.14:

$$\text{IG}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha, \quad (2.14)$$

On Equation 2.14, x is the input, x' is the baseline, and F is the model. The integral is typically approximated by summing gradients at discrete points between x' and x . The other features remain fixed while computing the partial derivative with respect to feature i .

The result of $\text{IG}_i(x)$ on Equation 2.14 indicates how much feature i contributed to the difference in model output between the baseline and the actual feature x_i . Positive values mean that increasing the feature from the baseline to its actual value increased the prediction score, while negative values indicate a decrease on the prediction score (Kokhlikyan et al., 2020).

2.4 Literature Review

In this section, the state of the art on both longitudinal healthcare data modeling and hospital readmission prediction is summarized in sections 2.4.1 and 2.4.2, respectively.

Finally, section 2.4.3 covers the review of representative strategies for incorporating attention mechanisms into longitudinal clinical data modeling, focusing on key approaches from selected relevant papers.

2.4.1 Longitudinal Healthcare Data Modeling

Hospital readmission prediction can be naturally framed as a longitudinal dataset problem. In such settings, multiple individuals are followed over time, and an outcome variable of interest – along with relevant explanatory variables – is repeatedly measured (Twisk, 2023). Longitudinal datasets differ from the traditional cross-sectional (or transverse) studies, which analyze a population at a single point in time (Cascarano et al., 2023), i.e. there is only a single measurement per subject. This longitudinal nature introduces specific methodological challenges:

1. **Correlation between repeated measures:** Observations from the same individual are not independent, violating the i.i.d. assumption and potentially biasing results;
2. **Missing values and irregular intervals:** Dropouts, skipped measurements, and uneven time gaps make the use of standard time-series algorithms challenging;
3. **Non-linear trajectories:** Individual health patterns may follow non-linear progressions;
4. **Different number of measurements per individual:** Some patients may have more frequent assessments than others, complicating the analysis;
5. **Uncertainties:** Measurement accuracy and reliability can change over time due to instrument limitations or individual-specific factors.

Several strategies have been proposed for supervised prediction tasks in longitudinal healthcare data. According to Cascarano et al. (2023), the most common approaches can be grouped into four categories.

The first family of methods relies on **summary features**, in which past measurements are aggregated into statistics such as averages or last values and then modeled with standard ML techniques (Cascarano et al., 2023). This strategy is computationally simple and robust to missing data, but it discards temporal order.

The second family includes **sequential models**, including the deep-learning-based ones introduced in section 2.2, which explicitly capture temporal dependencies and can handle variable-length sequences (Cascarano et al., 2023). While powerful, these approaches are data-hungry, sensitive to irregular sampling, and less explainable.

Other approaches include **non-sequential encodings** and **mixed-effects models**, but these methods overlook temporal dependencies in the data (Cascarano et al., 2023).

2.4.2 Hospital Readmission Prediction

Specifically to hospital readmission, several systematic reviews highlight common practices and findings. [Chen et al. \(2022\)](#) show that most models focus on single diseases such as HF or COPD, though some attempt generalization across conditions. Common predictive features include demographic variables (age, gender), comorbidity indices as the Charlson Comorbidity Index (CCI) ([Charlson et al., 1987](#)), prior admissions, length of hospital stay, medication use, and socioeconomic indicators ([Chen et al., 2022](#)). More granular features such as laboratory tests, vital signs, and unstructured clinical notes are increasingly used to improve performance. Data preparation typically excludes pediatric patients, very short hospitalizations, elective admissions, and patients who died during hospitalization ([Chen et al., 2022](#)).

From a methodological standpoint, readmission prediction is commonly posed as a binary classification task. Logistic regression remains a baseline due to its interpretability, while other most popular ML binary classification approaches include random forests, gradient boosting, and support vector machines. The use of such models relies mainly on the summary feature approach mentioned in section 2.4.1. Performance is usually evaluated using AUC, which is robust to imbalanced datasets, alongside metrics such as sensitivity, precision, and F1 score ([Chen et al., 2022](#)).

Another systematic review was carried out by [Huang et al. \(2021\)](#). The authors show that the overall discriminative performance of hospital 30-day readmission models can be modest, with a median AUC around 0.68 (inter quartile range: 0.64–0.76). About two-thirds of the reviewed models achieve AUC above 0.70. However, the performance varies widely depending on the dataset and used features. Among the reviewed models, deep learning methods have gained prominence. Overall, many of the studies consider only a single index hospitalization per patient, ignoring the longitudinal nature of readmissions.

[Huang et al. \(2021\)](#) report that neural networks were the second most common algorithms ($\approx 33\%$ of studies), including sequential architectures and convolutional neural networks (CNNs). [Chen et al. \(2022\)](#) further confirm that they achieved higher AUCs than traditional models.

2.4.3 Attention Mechanisms in Longitudinal Healthcare Data

In this section, strategies for incorporating attention mechanisms into longitudinal clinical data modeling are covered. The focus lies only on key approaches described in selected relevant papers.

2.4.3.1 Attention Combined with RNNs

In this class of models, attention is applied over hidden states produced by recurrent units such as RNNs or LSTMs. The attention mechanism assigns weights to these hidden states to compute a context vector that selectively summarizes past information for prediction. In the most common formulation, the final context vector at certain time t is computed as a sum of the past hidden states weighted by attention scores: $c_t = \sum_{i=1}^t \alpha_i h_i$, where α_i are attention weights derived from a compatibility function (e.g., dot product or feedforward network) between the current state and past hidden states.

Some variations of this approach exist. A representative example is the architecture **RETAIN** (Reverse Time Attention) (Choi et al., 2017), which introduces a dual-level attention architecture and attentions that multiply the embeddings of the input feature vectors x_t , as follows (Choi et al., 2017):

- The input consists of a sequence of visit vectors x_t , each encoding multivariate clinical codes from a patient visit.
- Two RNNs are used:
 - RNN_α computes visit-level attention scores α_t from hidden states g_t .
 - RNN_β computes variable-level attention scores β_t from hidden states h_t .
- The attention score α_t is not used to weight a hidden state directly; instead, it weights the visit embedding v_t to compute the context vector as described in Equation 2.15:

$$c = \sum_t \alpha_t \cdot (\beta_t \odot v_t) \quad (2.15)$$

This architecture enhances explainability: α_t reflects the relevance of each visit, while β_t highlights the most important features within a visit. Unlike standard RNN-attention architectures, RETAIN separates attention score computation from the input representations, making it clinically meaningful and easier to analyze (Choi et al., 2017).

2.4.3.2 Attention in Transformer-Style Architectures (No Recurrence)

Transformer-based models implement attention without any recurrence. Such type of architecture was first introduced by Vaswani et al. (2023)¹. Rather than processing the sequence in temporal order, self-attention enables the model to attend to all sequence elements simultaneously, allowing full parallelization during both training and inference.

First, each input visit x_t is embedded into a vector space $\mathbb{R}^d$, in which d is the embedding dimension. Such sequence of embeddings, $(z_1, z_2, \dots, z_T)$, where T is the number

¹ Even though the latest version of the paper dates back to 2023, its first version was published in April 2017

of observations, is commonly stacked in a matrix $Z \in \mathbb{R}^{T \times d}$. The transformation of raw input features into embeddings can be learned jointly with the rest of the model.

Then, self-attention is applied to compute relationships between all visits. Intuitively, self-attention allows the model to weigh the importance of each visit relative to others when forming a representation for prediction. These models compute three projections of the embedded input matrix Z as described in Equation 2.16: the query matrix Q , the key matrix K , and the value matrix V .

$$Q = ZW_Q \in \mathbb{R}^{T \times d_k}, \quad K = ZW_K \in \mathbb{R}^{T \times d_k}, \quad V = ZW_V \in \mathbb{R}^{T \times d_v} \quad (2.16)$$

The attention mechanism is then defined as stated in Equation 2.17:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (2.17)$$

The projection matrices W_Q, W_K, W_V are learned parameters. The matrix M is a mask that is normally used to make the attention causal, i.e. the final context vector for visit t can only attend to visits up to t and not future visits.

Each row t of the resulting attention matrix $\text{Attention}(Q, K, V) \in \mathbb{R}^{T \times d_v}$ can be seen as a context vector of dimension d_v that contains an updated representation of the visit t with information of the previous ones. It is a weighted combination of the value vectors $v_k \in \mathbb{R}^{d_v}$ with $k \in 1, \dots, t$, whose weights are determined by the similarity between the corresponding rows of query Q and keys K . The scaling factor $\sqrt{d_k}$ helps stabilize gradients during training.

The **Attend and Diagnose** (Song et al., 2017) model adopts this transformer-style attention to model sequences of patient visits. Each visit is embedded into a vector space and then passed through a transformer encoder, eliminating the need for recurrence.

One major advantage highlighted in this work is that, by removing recurrence, the model is no longer constrained to sequential processing. This enables parallel computation across the sequence, which improves training efficiency and scalability for large electronic health record datasets.

2.4.3.3 Hierarchical Attention: Visit and Feature Levels

Beyond the way attention is computed, models also vary in how attention is applied:

- **RETAIN** (Choi et al., 2017) applies dual-level attention:
 - Visit-level attention (α_t) emphasizes which past visits are important.

- Feature-level attention (β_t) identifies which variables within a visit are most informative.
- **Attend and Diagnose** ([Song et al., 2017](#)) applies attention only at the visit level, consistent with its transformer-based design.

3 Work Method

In this chapter, the work method adopted for this project is described. It includes the project phases, their respective activities, and the planning strategy used to organize the tasks over time.

3.1 Project Phases

The development of the project is structured into six main phases: preparation, analytics backend, frontend, tests, results analysis, and documentation. Each phase and its corresponding activities as follows.

3.1.1 Preparation

In this initial phase, the foundations for the project are established. The goal is to ensure that all subsequent work is systematically organized. The following activities are included:

1. **Project definition:** formalize the problem scope, main objectives, and expected contributions of the work;
2. **Planning:** organize the timeline of activities, dependencies, and milestones, setting priorities for execution;
3. **Requirements specification:** define functional and non-functional requirements for the system, providing a clear development target;
4. **Literature review:** synthesize prior research on longitudinal healthcare prediction and attention-based deep learning models, in order to establish the state of the art for the project's problem. A continuous follow-up review is also included to capture additional sources identified during development;
5. **Study of software tools:** investigate frameworks, libraries, and development platforms suitable for implementing the analytics backend and frontend. A continuous and complementary study addresses unforeseen needs or new options that arise during the project;
6. **System architecture:** design the high-level architecture, defining how the backend and frontend interact.

3.1.2 Analytics Backend

In the analytics backend phase, the focus is on implementing and training the predictive models. The analytics backend comprises choosing datasets, developing data extraction routines, carrying out preprocessing and feature engineering, and collecting baseline models (Garcia, 2025b). The main activities in this phase include:

1. **Model implementation:** implement new proposed attention-based deep learning architectures;
2. **Model training:** train the models on the selected healthcare dataset, including both baselines and main models, as well as tracking experiments using `neptune.ai` (Neptune Labs, 2025);
3. **API model inference:** implement an Application Programming Interface (API) to handle input data from the interface, generate predictions, and compute attribution values for explainability;
4. **Model tuning:** optimize hyperparameters through informed and random search in order to improve generalization performance.

3.1.3 Frontend

In the frontend phase, the focus is on providing an interactive interface to display predictions and support explainability. The main tasks performed in this step are:

1. **Base interface:** develop a core user interface for visualizing prediction results across multiple patients;
2. **Features' explainability:** extend the interface to show explainability plots for the model predictions and the impact of each feature;
3. **Patient data visualization:** add functionality to visualize the trajectory of a specific patient in the dataset.

3.1.4 Tests

In this phase, the goal is to ensure the correctness of the system. For that, the following activities are carried out:

1. **Unit tests:** verify preprocessing, feature engineering functions, and backend components used during model implementation and training;
2. **End-to-End (E2E) tests:** validate the communication between backend and frontend layers, ensuring stable deployment.

The purpose of the tests includes providing necessary evidence to verify whether the functional and non-functional requirements specified in Chapter 4 are met. Further needed evidence to conclude whether they are indeed met also depends on the 'Results Analysis' activities.

3.1.5 Results Analysis

In this step, the focus is on evaluating model performance, comparing approaches, and collecting additional evidence to verify whether functional and non-functional requirements are met. The activities include:

1. **Model selection:** identify the best-performing models of each architecture after hyperparameter tuning, based on metrics such as AUC for the 30-days readmission binary classification;
2. **Performance comparison:** compare the selected models with baseline approaches, analyzing strengths and weaknesses.

3.1.6 Documentation

The outcomes of the project and its results are consolidated in the documentation phase, simultaneously with other activities. The tasks in this phase are:

1. **Project monograph:** write the final academic monograph of the project, containing all phases, methods, and results;
2. **Poster and press release:** prepare materials to communicate the project in a concise and accessible format;
3. **Presentation video:** record a video to demonstrate the system, highlighting its usability and results.

3.2 Project Planning

To ensure a structured and organized workflow, a Gantt chart was developed to guide the execution of project phases and activities described in the previous section 3.1. In the chart illustrated in Figure 2, the main tasks, their dependencies, and their expected start and end dates are shown. The time measured for each activity is represented in business days, excluding weekends.

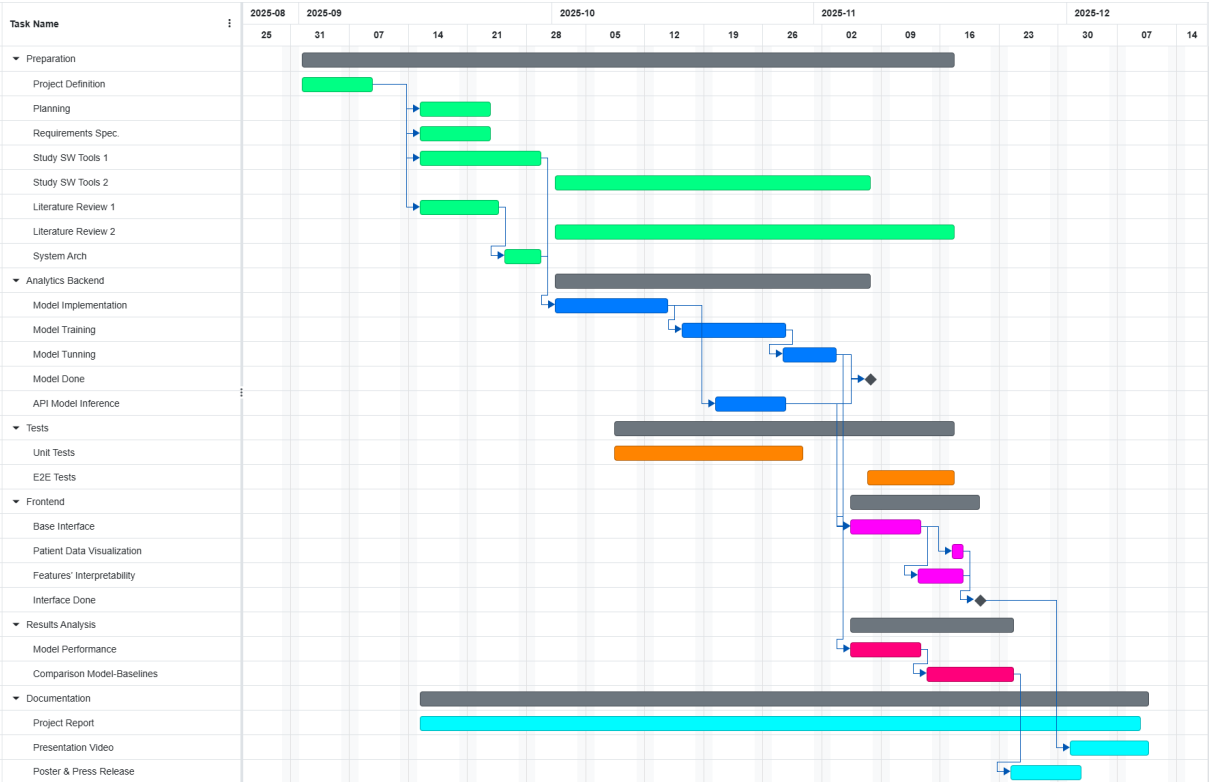


Figure 2 – Gantt chart outlining the main tasks and their expected duration in business days

More details about the activities presented in the Gantt chart of Figure 2 are provided in Appendix A.

4 Requirements Specification

Building upon the objectives and rationale discussed in Chapter 1, details about the specific requirements for the system, including both the model and the user interface, are reported in this chapter. The requirements are organized into functional and non-functional categories.

4.1 Functional Requirements

Functional requirements (FRs) correspond to services that the system must provide (Sommerville, 2016). These requirements were defined based on the project objectives described in section 1.2. Table 1 lists and provides descriptions of the key system FRs, each of which identified by a unique code for reference. The FRs are also mapped to specific components of the system architecture, which is presented in section 5.2.1.

Table 1 – Functional Requirements

Code	Requirement	System Component
FR-01	Upload a cohort Comma-Separated Values (CSV) file with multiple patients	Streamlit App (Frontend)
FR-02	Validate CSV schema: check required columns, data types, missing values in file uploaded by FR-01; send specific error messages to interface	Data Validation & Pre-processing (Backend)
FR-03	Display in the user interface eventual error messages generated by FR-02	Streamlit App (Frontend)
FR-04	Make predictions for all patients present in the uploaded CSV file of FR-01; returns per-patient probability	Deep Learning Model Inference (Backend)
FR-05	Display a cohort predictions table (patient_id, probability, optional risk band) generated by FR-04 with sorting and filtering	Streamlit App (Frontend)
FR-06	Display plots and statistics about predicted readmissions in FR-05: distribution of predicted probabilities, total number of predicted readmissions and total number of true readmissions (if given)	Streamlit App (Frontend)
FR-07	Compute metrics on the predictions of cohort generated by FR-04 (use ground-truth label column when present): Accuracy, Precision, Recall, F1, AUC, confusion matrix	Analytics & Explainability (Backend)
FR-08	Display the metrics related to FR-07 in the user interface	Streamlit App (Frontend)
FR-09	Compute explainability values for a specific patient of the cohort based on the predictions generated by FR-04	Analytics & Explainability (Backend)
FR-10	Allow selecting a patient and display, in single-patient view, the patient's predicted probability and a top-features explanation based on the values computed by FR-09	Streamlit App (Frontend)
FR-11	Make predictions with performance of $AUC \geq 0.70$ on unseen data	Deep Learning Model Inference (Backend)
FR-12	Make predictions with a performance higher than a Logistic Regression baseline (e.g., +0.05 AUC) on unseen data	Deep Learning Model Inference (Backend)

4.2 Non-Functional Requirements

The non-functional requirements (NFRs) define constraints on the services or functions offered by the system (Sommerville, 2016). In the current project, these requirements focus on performance and timing that the system must meet. Table 2 outlines the key NFRs for the system, each of which identified by a unique code for reference.

Table 2 – Non-functional requirements

Code	Requirement	Target
NFR-01	Explanation time (Single Patient)	Response time $\leq 5s$
NFR-02	Prediction time (Cohort)	$\leq 10s$ for 1,000 patients;
NFR-03	User Interface (Ease of Use)	Intuitive interface with clear instructions, consistent visual elements (buttons, tables, icons), and minimal steps to complete core tasks.

For NFR-01 and NFR-02, the response times shall be measured using the hardware environment available for the project development, namely a devcontainer running on Docker Desktop with a Windows Subsystem for Linux 2 backend. The host machine is equipped with a 12th Gen Intel® Core™ i7-1255U processor (6 physical cores and 12 threads), and the container had access to approximately 7.6 GB of RAM and 2 GB of swap memory. The execution environment inside the container is based on Debian GNU/Linux 13 (trixie). No GPU acceleration is available or used.

5 Project Development

In this chapter, the project development process is detailed, covering the used technologies, system architecture, design and implementation decisions and tradeoffs, as well as testing strategies employed to ensure the correctness of the developed system.

5.1 Used Technologies

The software environment and many of the tools used in this project build directly on the setup established in the author’s previous work (Garcia, 2025b). To ensure full reproducibility across environments, both development and experimentation are carried out inside a *devcontainer* – a Docker-based container – providing an isolated and consistent development environment (Microsoft, 2025). The programming language used in the project is Python 3.10.14, and dependency management is handled with *Poetry* (Eustace; Poetry contributors, 2025). The code repository is publicly available on GitHub¹ (Garcia, 2025a).

All experiments are tracked and logged using *Neptune.ai* (Neptune Labs, 2025), a platform for experiment management and model registry. This tool facilitated the organization of all results obtained in this work and was specially important for performance comparisons between models and baselines.

All Python packages used to implement and evaluate the models are listed in Table 17 in the Appendix B. Among these packages, *PyTorch* (Paszke et al., 2019) is the primary deep learning framework for implementing and training the models. The hyperparameters of such models are optimized with *Optuna* (Akiba et al., 2019). For training the ML baselines and computing all evaluation metrics in this work, *scikit-learn* (Pedregosa et al., 2011) is used. Explainability analyses are performed using *SHAP* (Lundberg; SHAP contributors, 2025) and *Captum* (Kokhlikyan et al., 2020).

The user interface was built with *Streamlit* (Streamlit Inc., 2025), enabling rapid development of interactive web applications. The backend API was implemented using *FastAPI* (Ramírez, 2018).

5.2 Design and Implementation

This section covers the design choices and implementation details of the developed system.

¹ <<https://github.com/LucasVon0645/msc-thesis-recurrent-health-modeling>>

5.2.1 System Architecture

A high-level overview of the system architecture is presented in Figure 3. The system consists of two main components: the analytics backend and the frontend. The backend's API layer, implemented using FastAPI (Ramírez, 2018), connects them. Each component in the architecture can be directly linked to one or more functional requirements described in Table 1. In the Table 3, an overview of all system components and their high-level roles is provided.

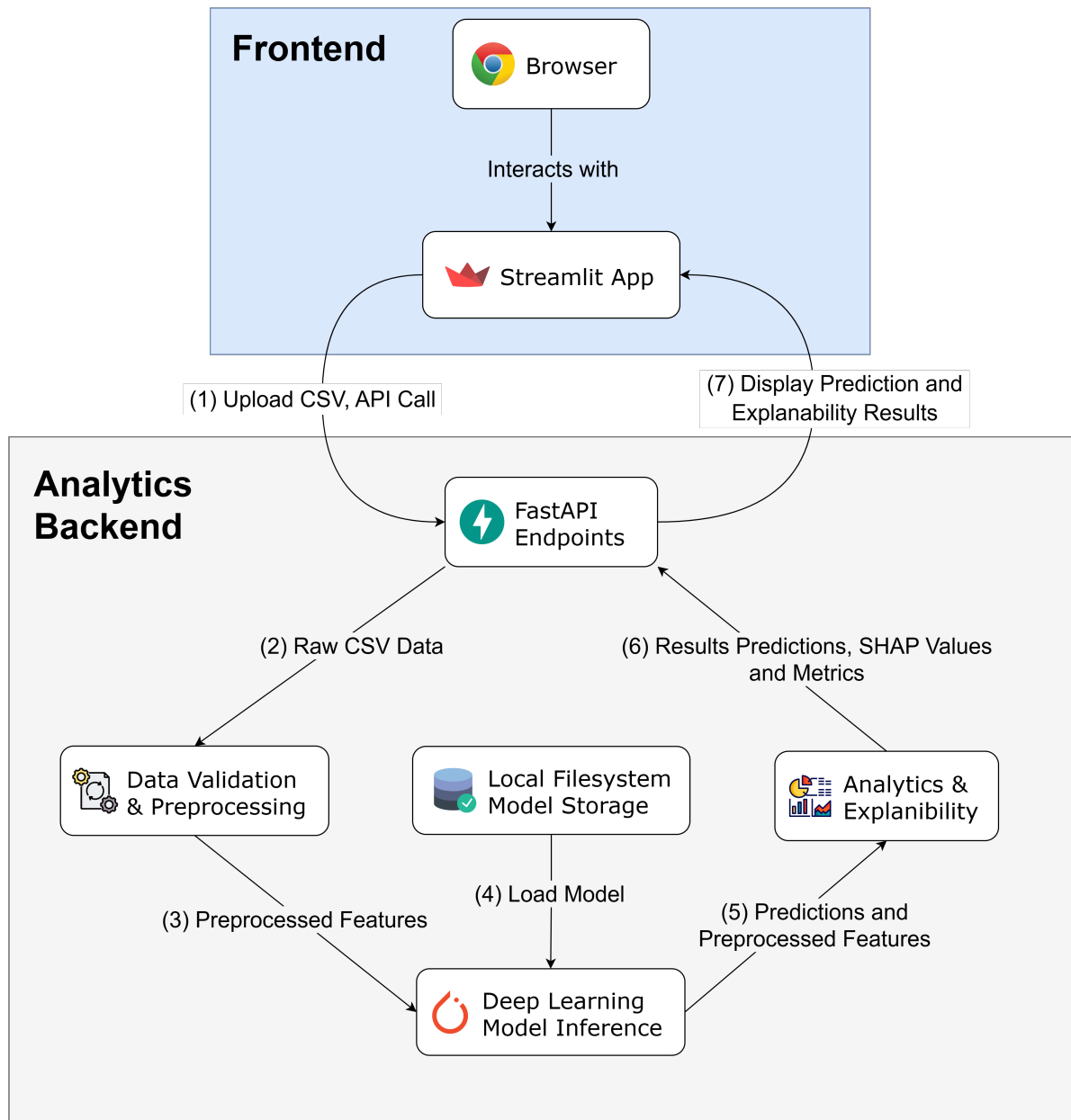


Figure 3 – High-level system architecture

Table 3 – System components overview

Component Name	Description
Browser (Frontend)	Entry point through which the user interacts with the application, typically to upload CSVs, view results, and navigate through prediction outputs.
Streamlit App (Frontend)	Provides the user interface, displaying predictions, error messages, analytics results, and explainability plots in an intuitive and interactive way.
FastAPI Endpoints (Backend)	Acts as the orchestrator between frontend and backend services, receiving raw data, managing requests, and returning predictions, metrics, and explainability outputs.
Data Validation & Pre-processing (Backend)	Validates input CSVs for schema correctness and prepares features through preprocessing to ensure clean, structured data for model inference.
Local Filesystem Model Storage (Backend)	Stores trained models locally and makes them available for inference when required. Ensures consistent and fast access to model files.
Deep Learning Model Inference (Backend)	Runs the trained deep learning model on preprocessed features to generate predictions, supporting performance benchmarks and accuracy requirements.
Analytics & Explainability (Backend)	Produces cohort-level statistics, performance metrics, and model explainability outputs (e.g., SHAP values), enabling both global and patient-level explainability.

5.2.1.1 Input Data

The input data for the system, which is then forwarded to the backend for processing, consists of six CSV files representing different aspects of patient admissions. Each CSV corresponds to a specific dataframe with required columns and types, as detailed in Table 4.

SUBJECT_ID identifies an unique patient, and HADM_ID identifies a specific hospital admission for that patient; together, they are used to link records across data frames. The ICD9_CODE column (Table 4) contains codes from the International Classification of Diseases, Ninth Revision (ICD-9). It appears in both the *Diagnoses* and *Procedures* data frames but denotes different concepts: in the former, diagnosis codes assigned during the admission; in the latter, procedure codes recorded during the hospital stay.

Dataframe	Required Columns (with types)		Description
Admissions	HADM_ID	integer	Information associated with the admission event. Admission type can only be one of the following: <i>elective</i> , <i>urgent</i> , or <i>emergency</i> .
	SUBJECT_ID	integer	
	ADMITTIME	timestamp/string	
	DISCHTIME	timestamp/string	
	ADMISSION_TYPE	string	
	INSURANCE	string	
	ETHNICITY	string	
	DISCHARGE_LOCATION	string	
Diagnoses	HADM_ID	integer	Diagnosis codes (ICD-9) assigned during the admission. Multiple ICD9_CODE values may be present for a single admission.
	SUBJECT_ID	integer	
	ICD9_CODE	string	
Patients	SUBJECT_ID	integer	Patient demographics and birth date.
	GENDER	string (<i>F M</i>)	
	DOB	timestamp/string	
ICU Stays	SUBJECT_ID	integer	ICU-level encounters linked to admissions.
	HADM_ID	integer	
	INTIME	timestamp/string	
	OUTTIME	timestamp/string	
Procedures	SUBJECT_ID	integer	Procedure codes (ICD-9) recorded per admission and patient.
	HADM_ID	integer	
	ICD9_CODE	string	
Prescriptions	HADM_ID	integer	Medications prescribed during the hospital stay.
	SUBJECT_ID	integer	
	DRUG	string	

Table 4 – Input data frames, required columns (with types) and descriptions

Typing note: Use consistent timestamp parsing (e.g., ISO 8601 YYYY-MM-DD HH:MM:SS). Columns listed as *timestamp/string* may be provided as strings parsable into timestamps. Columns listed as *string* contain categorical values.

5.2.2 About the Datasets

This study employs the same dataset and extraction procedures as described by [Garcia \(2025b\)](#). The data are derived from the MIMIC-III, a publicly available and de-identified (anonymized) critical care database containing comprehensive clinical information from hospital admissions at the Beth Israel Deaconess Medical Center between 2001 and 2012 ([Johnson et al., 2016](#); [Johnson; Pollard; Mark, 2016](#)). Data access is provided via the *PhysioNet* platform, following completion of credentialing and human-subjects research training ([Goldberger et al., 2000](#)).

As in the previous work, patient cohorts were defined according to diagnostic categories based on ICD-9 codes mapped to the CCI ([Charlson et al., 1987](#)). In the present work, however, the disease cohort was expanded to include patients with at least one of the following conditions: **renal disease**, **congestive heart failure**, **chronic pulmonary disease**, or **diabetes with complications**. This expansion was motivated by the need

to increase the dataset size and heterogeneity, since deep learning models explored in the current study generally require larger training samples for robust generalization.

Further details on the original inclusion criteria, extraction logic, and cohort construction can be found in the previous work of [Garcia \(2025b\)](#).

5.2.3 Data Preprocessing Pipeline

The data preprocessing steps follow the same general structure described by [Garcia \(2025b\)](#), with few changes on how data is split between train, validation and test sets. The complete pipeline is summarized in Figure 4.

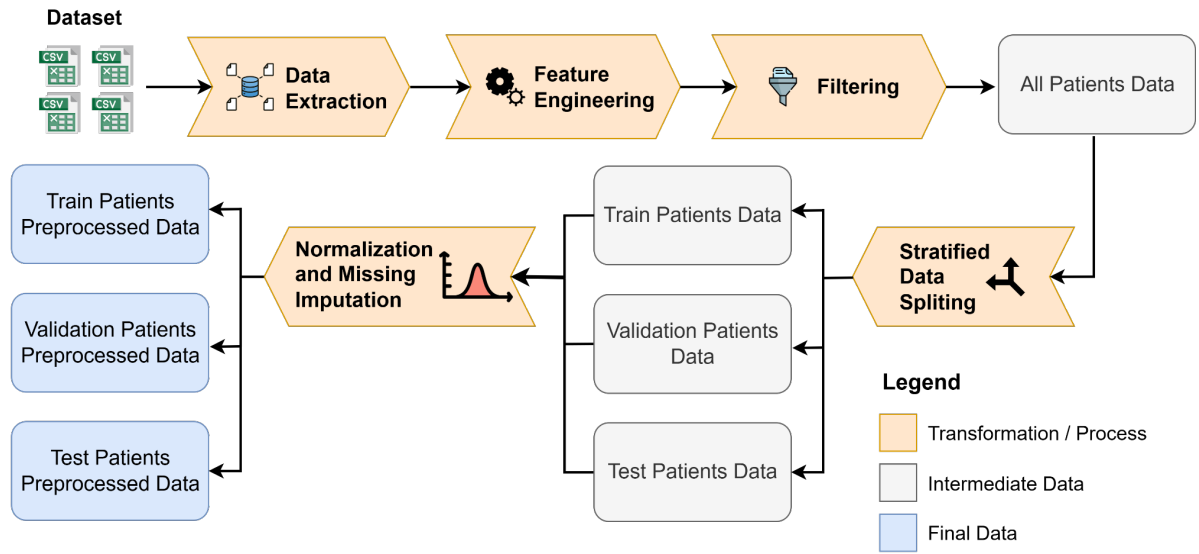


Figure 4 – Data preprocessing pipeline

The process begins with the extraction of hospitalization records from the MIMIC-III database locally stored as CSV files. Diagnoses are mapped to the CCI categories, and patients meeting the expanded disease cohort criteria were selected (see section 5.2.2). Records of all hospitalizations of the selected patients are collected from the raw dataset.

After extraction, multiple preprocessing stages were applied to prepare the data for modeling, as outlined below.

5.2.3.1 Feature Engineering

Following the same logic of [Garcia \(2025b\)](#), several groups of features are computed, including hospitalization-level, time-related, historical, demographic, administrative, and comorbidity indicator variables. Low-frequency categories in some categorical variables (e.g., ethnicity, insurance, and discharge location) are grouped to simplify representation and mitigate overfitting. Such categorical features are then one-hot encoded to create binary indicator variables. The names and descriptions of all features used in this work

are provided in Appendix C.

5.2.3.2 Filtering and Selection

Standard filtering criteria are applied to remove implausible or inconsistent records, following practices recommended in the literature and previously adopted by Garcia (2025b). These include excluding implausible ages, unrealistically short or long hospitalizations, and scheduled elective readmissions.

5.2.3.3 Stratified Data Splitting

After preprocessing, the dataset was divided into *train*, *validation*, and *test* subsets using a stratified sampling approach. The stratification ensured that the following patient-level metrics remained similar across all subsets:

- average number of visits per patient;
- average readmission rate per patient;
- average CCI (mean across all visits for each patient).

5.2.3.4 Normalization and Missing-Value Input Entry

Missing values are inputted using the median of each variable due to its robustness against outliers, that were detected while exploring the dataset (Garcia, 2025b). Continuous features are then normalized using scaling parameters (mean and standard deviation) estimated exclusively from the training set in order to improve stability and convergence of the training. The same parameters are subsequently applied to the validation and test sets to prevent data leakage and ensure a fair evaluation.

5.2.4 Exploratory Data Analysis (EDA) - Challenges of the Dataset

As noted by Garcia (2025b), the subset obtained after applying the inclusion criteria contains several challenges. The most notable one is the strong dominance of patients with a single hospital admission and no past information available for modeling. Such patients represent almost 86.1% of the selected cohort.

Since such cases do not offer a temporal sequence of hospital visits from which sequential or attention-based deep learning models (the main focus of this work) could learn from, such cases were removed from the final cohort. However, even after this additional filtering step, the dataset still contains a significant proportion of patients with only one or two prior hospitalizations before the final index admission (so called "last events" in this dataset). This behavior is depicted in Figure 5, which shows the distribution of the

number of prior hospital visits per patient, with respect to the last readmission event. The great majority of patients in this subset have only up to two prior hospitalizations (75.5%), which limits the amount of longitudinal information available for modeling.

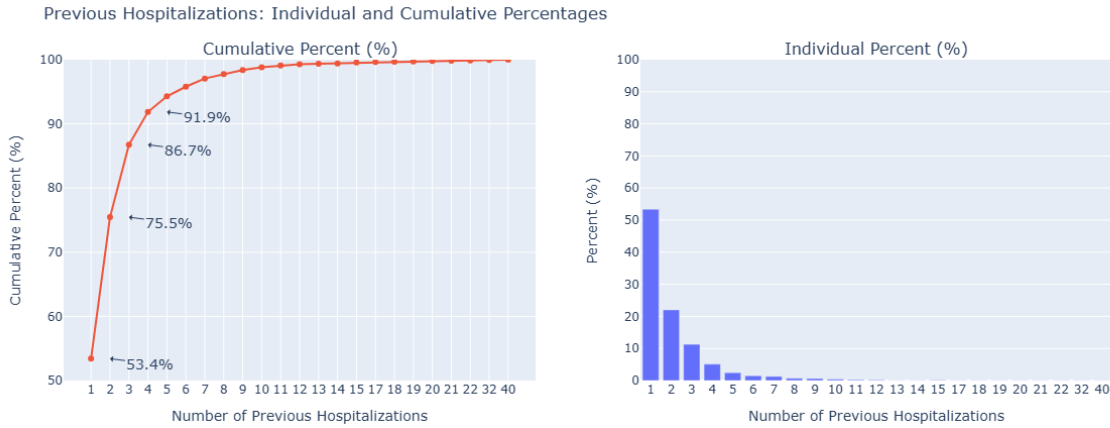


Figure 5 – Distribution of the number of prior hospital visits per patient in the selected cohort, considering only patients with any prior hospitalizations before the last readmission event

This short and limited longitudinal history might affect the ability of the deep learning models studied in this work to outperform classical machine learning baselines that do not explicitly model temporal dependencies. The investigation of this question is one of the main objectives of this project (see section 1.2).

Another challenge is the heterogeneity of the diseases and diagnoses. Such heterogeneity is illustrated in Figures 6 and 7. Despite the restriction of taking subjects with at least one registered diagnosis of the specified chronic conditions (see section 5.2.2), patients often suffer from multiple diseases at the same time, as shown by Figure 7, which adds complexity to the modeling task.

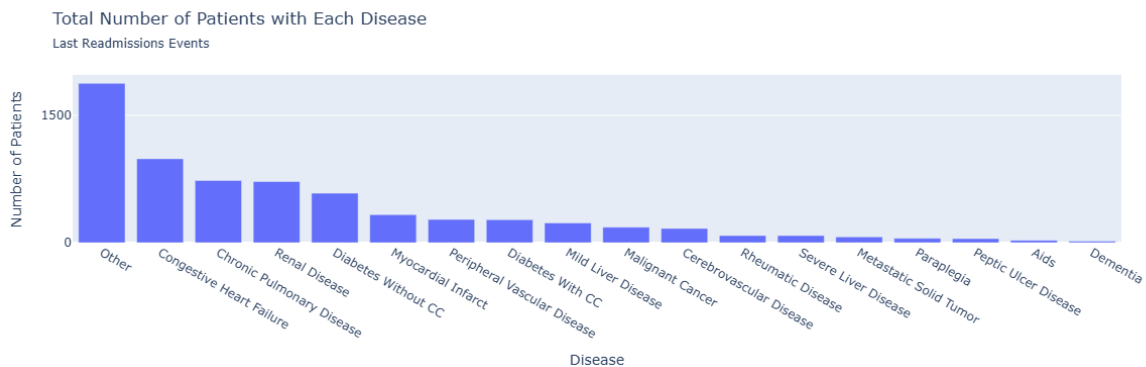


Figure 6 – Distribution of diseases in the selected cohort based on CCI categories. Each bar represents the percentage of patients diagnosed with each condition, highlighting the prevalence of various chronic diseases within the cohort



Figure 7 – Distribution of the number of diseases (comorbidities) of CCI per patient for the last hospital readmission

Finally, another important challenge noticed during the EDA phase is the weak predictive power of individual features, measured by the correlation between each feature and the target variable (30-days readmission). The Pearson correlation of the binary features with the binary target variable is shown in Figure 8, while the rank biserial correlation of the continuous ones is illustrated in Figure 9 (Garcia, 2025b). Such weak predictive power of individual features was also observed by Garcia (2025b). For the description of the features used in this work, see Appendix C.

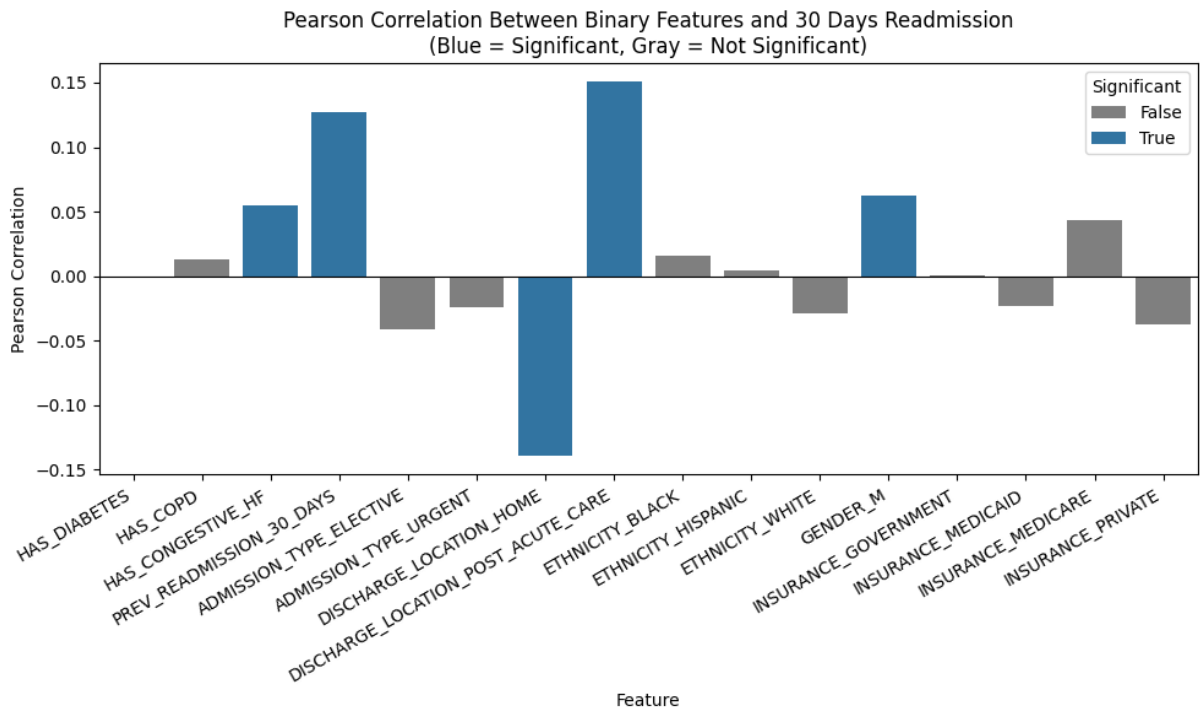


Figure 8 – Pearson correlation of binary features to the target variable (30-days readmission). Features are sorted by absolute correlation values. Results obtained by [Garcia \(2025b\)](#)

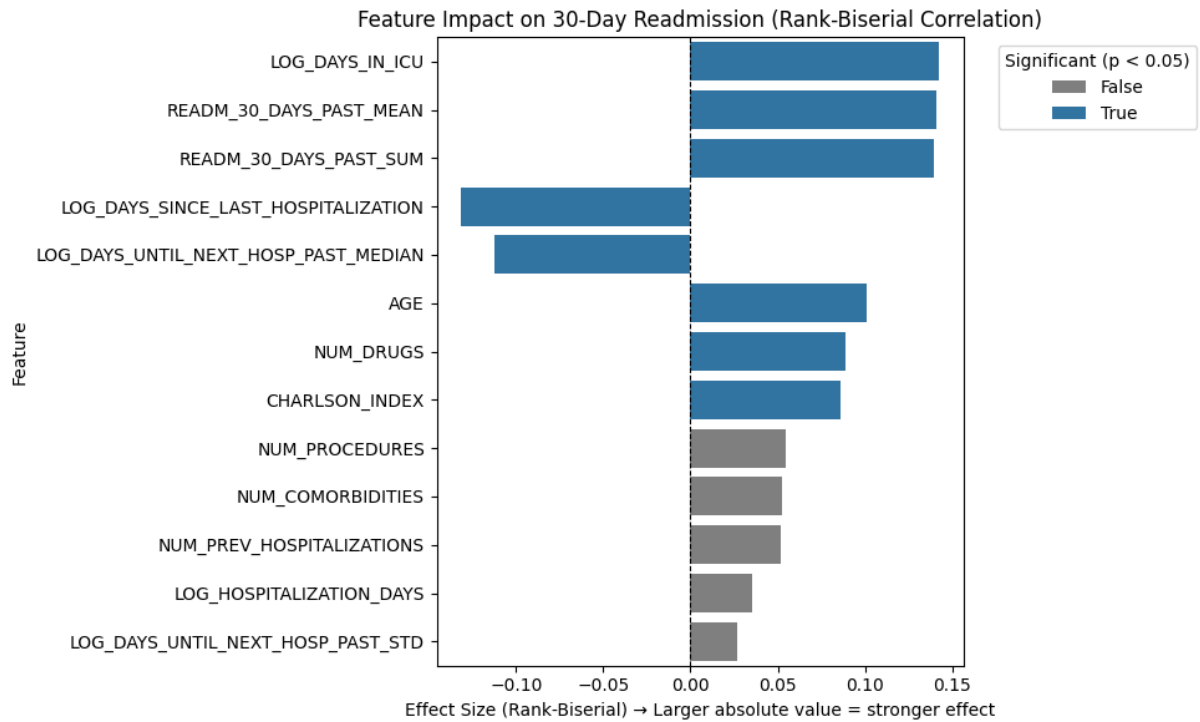


Figure 9 – Rank biserial correlation of continuous features to the target variable (30-days readmission). Features are sorted by absolute correlation values. Results obtained by [Garcia \(2025b\)](#)

Most features exhibit very low correlation values (absolute values below 0.15), indicating that no single feature strongly predicts readmission on its own. This observation suggests that complex interactions among multiple features may be necessary to effectively

model readmission risk.

5.2.5 Classical ML Baselines

For comparison with the deep learning models described in section 5.2.6, Logistic Regression with regularization and a Random Forest classifier were implemented using the `scikit-learn` framework (Pedregosa et al., 2011).

Since these traditional models assume the input data are i.i.d., the summary feature approach described in section 2.4.1 was applied for training and evaluation. This same approach was also used as a baseline in previous work (Garcia, 2025b). In this setup, each patient is represented by a single sample corresponding to the last hospital visit. The feature vector includes information from the current hospitalization along with summary statistics from previous visits (e.g., number of past admissions, time since last discharge, and historical readmission rate).

5.2.6 Deep Learning Architectures

Given that the focus of this work is exploring deep learning models, especially the ones with attention mechanisms, for 30-days hospital readmission prediction (see section 1.2), several architectures were designed and investigated. Such architectures were implemented using the PyTorch framework (Paszke et al., 2019).

All models considered in this study share a common design structure and objective in order to allow a fair and direct comparison all models. Each model outputs a logit value, which can be transformed into a predicted probability using a sigmoid function. Despite architectural differences, all models have the following elements in common:

- **Inputs:** Three input tensors are used:
 - A tensor of current hospital visit features with dimension $[B, D_{\text{curr}}]$, in which B is the batch size and D_{curr} is the number of features for the current visit vector $\mathbf{x}_{\text{curr}}$. Such input is the concatenation of the observation vector $\mathbf{x}_k$ with the static variables $\mathbf{s}$ stated in the problem formulation (see section 1.4);
 - A tensor representing the ordered sequence of feature vectors with D_{past} from the previous T hospital visits (dimension $[B, T, D_{\text{past}}]$). Such input corresponds to the longitudinal history $\mathcal{H}_N$ defined in section 1.4;
 - A binary mask $[B, T]$ indicating valid past visits with the value 1, used to handle padded positions (feature vectors with zeros).
- **Feature combination:** The models combine the current visit features with a representation vector $\mathbf{z}$ extracted from past visits;

- **Classification head:** The concatenated vector $[\mathbf{z} \parallel \mathbf{x}_{\text{curr}}]$ is fed into an MLP with one hidden layer using ReLU activation, followed by a dropout layer. The final linear layer outputs the logit used for classification. These settings were defined in order to avoid vanishing gradients issues and bring regularization to the model.

The usage of a binary mask tensor is crucial for handling variable-length sequences of past visits. Since patients may have different numbers of prior hospitalizations, shorter sequences are padded with zeros to match the maximum length T . The mask ensures that the model only attends to valid past visits during processing, effectively ignoring the padded positions.

The classification head, shared by all model architectures, is illustrated in Figure 10.

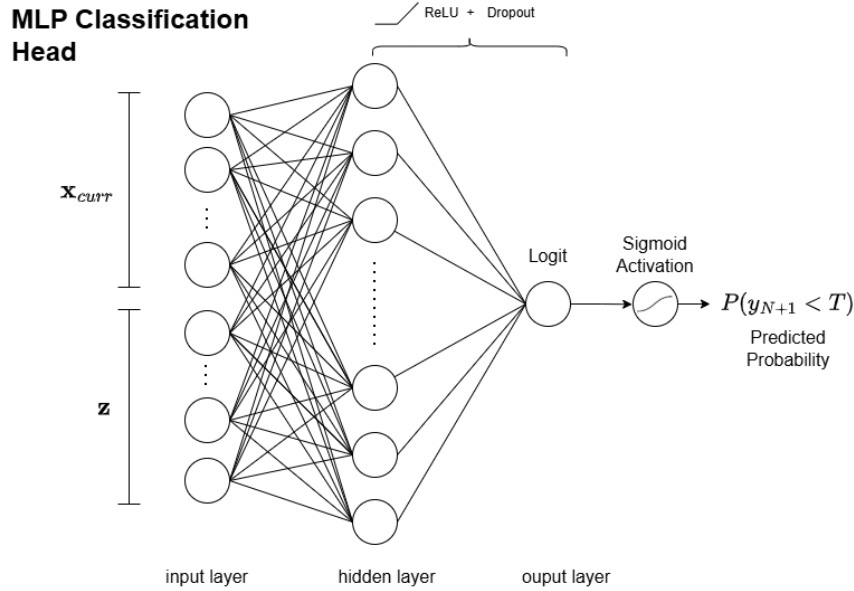


Figure 10 – Shared MLP classification head architecture

The main difference between the models lies in how the representation vector $\mathbf{z}$ is extracted from the longitudinal history $\mathcal{H}_N$. Two main approaches are explored: one using a sequential recurrent block based on a GRU, and another using an attention-based block. The first is investigated with the architecture described in section 5.2.6.1, while the second one with the architectures explained in sections 5.2.6.2, 5.2.6.3 and 5.2.6.4.

The reason for exploring both recurrent and attention-based architectures is to compare their effectiveness in modeling the temporal dependencies present in the sequence of hospital visits. While recurrent models like GRUs are well-suited for capturing sequential patterns, attention mechanisms have shown promise in effectively modeling long-range dependencies and providing explainability through attention weights.

5.2.6.1 GRUNet

The deep learning sequential model evaluated in this study is based on a GRU block, which is briefly explained in the theoretical background section 2.2.2.

As illustrated in Figure 11, the GRU processes the sequence of past visits $\mathbf{x}_{\text{past}} \in \mathbb{R}^{B \times T \times D_{\text{past}}}$ and outputs a summary vector $\mathbf{z} \in \mathbb{R}^{B \times M}$ corresponding to the last hidden state of the top recurrent layer. This summary is concatenated with the current visit features $\mathbf{x}_{\text{curr}}$, forming $[\mathbf{z} \parallel \mathbf{x}_{\text{curr}}]$, which is then passed through the shared MLP classification head to produce the output logit.

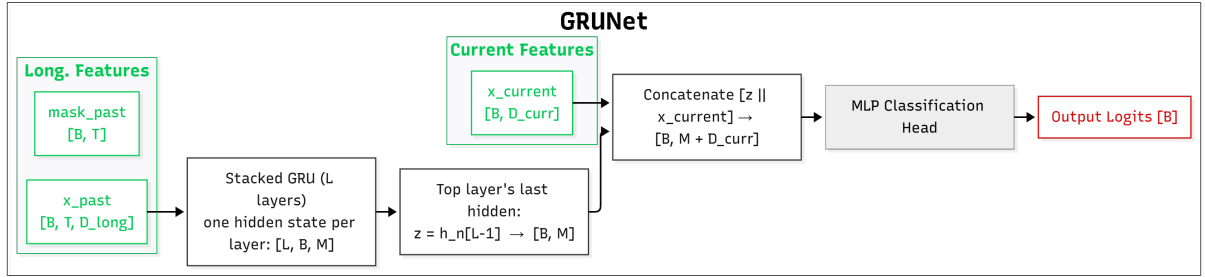


Figure 11 – Overview of the GRU-based model architecture. The GRU encodes the longitudinal sequence into a summary vector $\mathbf{z}$, which is concatenated with the current visit features before classification

The choice of the GRU architecture was guided by both practical and empirical considerations. As described in section 2.2.2, unlike an LSTM network, the GRU has a simpler gating mechanism and fewer parameters, which helps reduce the risk of overfitting (Perumal et al., 2024) – particularly relevant in this dataset, in which most patients have only a few recorded hospital visits. At the same time, GRUs are generally more expressive and stable than simple RNNs, often achieving comparable or better performance in sequence modeling tasks while requiring less computational cost (Perumal et al., 2024).

This balance between representational power and generalization makes the GRU a suitable choice for modeling limited longitudinal hospital data.

5.2.6.2 Simple Attention Pooling: SimpleAttnPool

Figure 12 illustrates the *SimpleAttnPool* architecture, which applies a single-query attention mechanism to aggregate information from past visits into a compact representation vector $\mathbf{z}$.

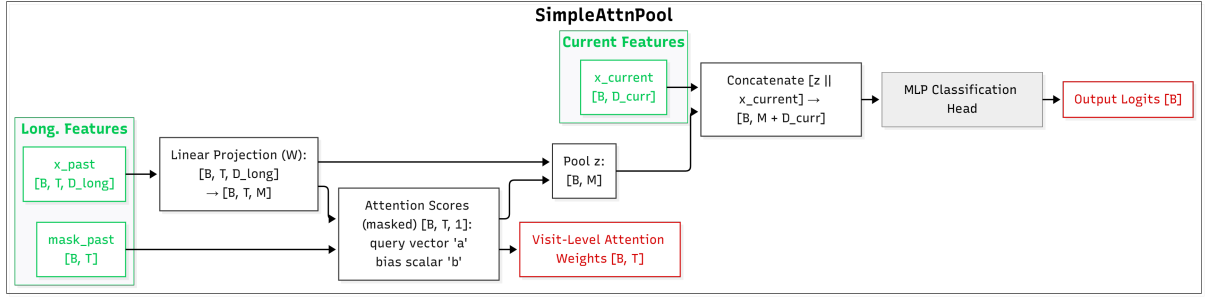


Figure 12 – Overview of the SimpleAttnPool model architecture. The model uses a single-query attention mechanism to aggregate information from past visits into a compact representation vector $\mathbf{z}$

Each past visit i , which is described by a feature vector $\mathbf{x}_i \in \mathbb{R}^{D_{\text{past}}}$, is projected into an embedding space using a learnable matrix $W \in \mathbb{R}^{M \times D_{\text{past}}}$, as stated in Equation 5.1:

$$\mathbf{h}_i = W\mathbf{x}_i. \quad (5.1)$$

A global learnable query vector $\mathbf{a} \in \mathbb{R}^M$ and a scalar bias b are used to compute attention scores for each visit, following the dot-product formulation described in section 2.2: $s_i = \text{score}(\mathbf{h}_i) = \mathbf{h}_i^\top \mathbf{a} + b$.

For simplicity and to limit overfitting given the modest dataset size, the values and the keys of the attention mechanism described in 2.2 are assumed equal to the embeddings ($\mathbf{h}_i = \mathbf{v}_i = \mathbf{k}_i$).

The attention weights α_i of each past visit i are obtained via a softmax over s_i , indicating their relative importances. The representation vector $\mathbf{z}$ is then computed through weighted pooling of the value vectors, summarizing the patient's longitudinal history, as stated in Equation 5.2.

$$\mathbf{z} = \sum_{i=1}^T \alpha_i \mathbf{v}_i. \quad (5.2)$$

Finally, $\mathbf{z}$ is concatenated with the current visit features $\mathbf{x}_{\text{current}}$ and passed through the shared MLP classification head to produce the final logit.

The only hyperparameter of this attention block is the embedding dimension M , which defines the size of the projection space and controls the model's capacity to capture temporal patterns in the patient's history.

5.2.6.3 Simple Attention Pooling with Current Query: SimpleAttnPoolCurrQuery

Figure 13 illustrates the *SimpleAttnPoolCurrQuery* architecture, which extends the original *SimpleAttnPool* mechanism by replacing the global learnable query vector with a *context-dependent* query derived from the current visit features.

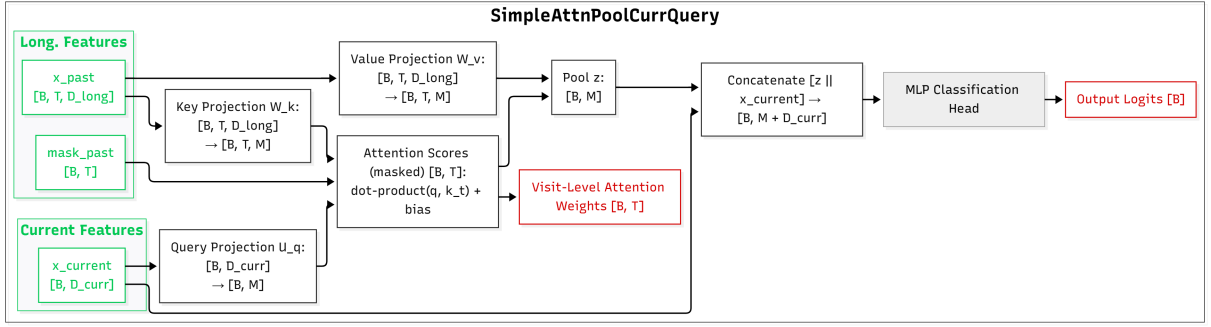


Figure 13 – Overview of the SimpleAttnPoolCurrQuery model architecture. The model computes the query based on the current visit features

Instead of relying on a fixed vector $\mathbf{a}$ to score past visits, the model computes a query as described in Equation 5.3:

$$\mathbf{q} = U_q \mathbf{x}_{\text{current}}, \quad (5.3)$$

On Equation 5.3, $U_q \in \mathbb{R}^{M \times D_{\text{curr}}}$ is a learnable projection matrix. This formulation enables the model to search for and attend to patterns in the patient’s historical visits that are specifically relevant to the current hospitalization, which can provide in theory a more context-aware retrieval of longitudinal information.

As in the standard attention formulation, each past visit vector $\mathbf{x}_i$ is projected into an embedding space to obtain key vectors $\mathbf{k}_i = W_k \mathbf{x}_i$. In contrast to *SimpleAttnPool*, in which keys and values are tied ($\mathbf{k}_i = \mathbf{v}_i$), the *SimpleAttnPoolCurrQuery* architecture optionally supports a separate value projection, as stated in Equation 5.4:

$$\mathbf{v}_i = W_v \mathbf{x}_i, \quad (5.4)$$

This separate value projection option is controlled by the hyperparameter `use_separate_values`. When this option is disabled, the model uses the keys as values, matching the behavior of *SimpleAttnPool*. When enabled, the model learns distinct representations for computing attention scores (keys) and for constructing the pooled summary vector (values).

The latter steps of the attention mechanism and the classification head follow the same structure described for *SimpleAttnPool*.

5.2.6.4 MultiHead-Attention-Pooling: MultiHeadAttnPool

Figure 14 illustrates the *MultiHead-Attention-Pooling* model investigated in this work. The motivation behind this architecture is to explore whether a more expressive attention mechanism, compared to the simpler *SimpleAttnPool* and *SimpleAttnPoolCurrQuery*, can better capture the temporal and contextual dependencies in the sequence of

visits. This experimental variant, named *MultiHeadAttnPool*, extends *SimpleAttnPool* and *SimpleAttnPoolCurrQuery* by adopting the additional ideas of the Transformer architecture (Vaswani et al., 2023), which has shown strong performance in various sequence modeling tasks. The tradeoff of this approach is the increased model complexity and number of parameters, which may lead to overfitting, especially given the limited dataset size.

The additional elements of the original Transformer architecture (Vaswani et al., 2023) that are incorporated in *MultiHead-Attention-Pooling* are: (i) positional encoding of visit embeddings, (ii) multi-head attention, and (iii) internal linear transformations for the query, key, and value vectors.

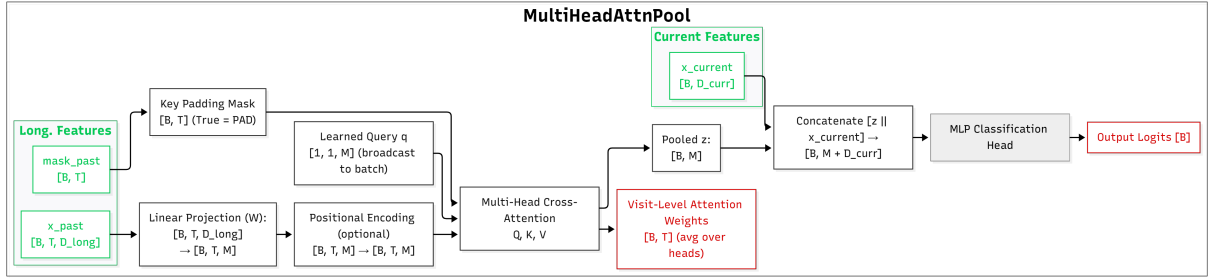


Figure 14 – Overview of the MultiHead-Attention-Pooling model architecture. The model employs multi-head attention to capture complex dependencies across visits, enhancing the representation of the patient’s history

As in the previous model, an embedded representation $\mathbf{h}_i$ is computed for each past visit i through a linear projection of the input features $\mathbf{x}_i$ using a learnable matrix $W \in \mathbb{R}^{M \times D_{\text{past}}}$.

In contrast to *SimpleAttnPool*, the sequence of projected embeddings $\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_T]$ may optionally receive sinusoidal positional encodings. This component enables the model to encode the temporal order of visits, providing an explicit notion of sequence position that a purely content-based attention mechanism would otherwise lack (see section 2.2).

Similar to the traditional attention mechanism described in section 2.2, the query, key, and value vectors are linearly transformed using learnable matrices W_Q , W_K , and W_V . This extends the *SimpleAttnPool* model, which directly used the projected embeddings as keys and values without additional transformations.

Analogous to the *SimpleAttnPool* model, a single fixed parameter vector is used to produce the query representation. However, due to the linear transformation by W_Q , each attention head learns its own query vector, allowing the model to attend to different aspects of the input sequence simultaneously. This multi-head formulation increases representational flexibility and helps the model capture a richer set of dependencies, though it also introduces additional parameters and a higher risk of overfitting.

Each attention head computes its own attention scores s_i through dot-product

similarity between the query and the keys. The resulting head-specific outputs are then combined: the output vectors from all heads are concatenated to form the final pooled representation $\mathbf{z} \in \mathbb{R}^M$. The final attention weights are obtained by averaging across all heads.

The remaining steps of the model, including concatenation with the current visit features and the MLP classification head, follow the same structure described for *SimpleAttnPool*.

5.2.7 Model Training

The training and validation splits from the data pipeline described in section 5.2.3 are used to train the architectures described in section 5.2.6. The data obtained after normalization, one-hot encoding, and missing value input entry is used in this step. A total of 80% of the available data is given to the train and validation splits, while the remaining 20% is reserved for the final evaluation, which is described in section 5.2.8. The reason for choosing 80% of data as the training set proportion is to have sufficient training data without compromising the sample size of the unseen test set with the dataset used in this project.

Each model input described in section 5.2.6 is created using a sliding window over the patient’s hospitalization history. The window length is set to five visits: the current visit (for which the readmission period is predicted) and the four previous hospitalizations (parameter $T = 4$ stated in section 5.2.6). This strategy increases the number of available samples while preserving the sequential structure of patient trajectories. The window length was chosen based on the dataset distribution depicted in Figure 5, as most patients (98,9%) have fewer than five recorded hospitalizations. If a patient has less than 4 past hospitalizations, a padding strategy fills the remaining positions with zeros (no hospitalization).

Training samples are processed in fixed-size batches, with the batch size treated as a hyperparameter of the training process. Models are trained by minimizing cross-entropy loss using gradient descent, with gradients computed through backpropagation. Optimization is performed using the Adam optimizer with weight decay, which adapts learning rates for each parameter, stabilizes updates, and helps preventing trapping into local minima. The weight decay value remains constant during training.

The Learning Rate (LR) begins at a predefined value and follows one of two scheduling strategies, both of which also help prevent trapping into local minima:

- *Cosine Annealing*, which gradually reduces the learning rate following a cosine curve.

- *Reduce LR On Plateau*, which lowers the learning rate when a monitored metric stops improving.

Training is done up to a fixed maximum number of epochs (each epoch corresponds to a full pass through the dataset). To prevent overfitting, early stopping is applied based on the AUC score computed on the validation set. If no improvement of this metric is observed for five consecutive epochs, training is finished and the best-performing epoch (based on validation AUC) is retained.

Since the task is a binary classification problem, it is necessary to establish a decision threshold to convert predicted probabilities into class labels. Given the strong class imbalance in the dataset and the need to balance false positives and false negatives in hospital readmission prediction, the F1-score is used to select the probability threshold for the final prediction $\hat{y}_{N+1}$, as it jointly considers precision and recall.

As monitored metrics during training, in addition to the validation AUC, the training loss is also tracked at the end of each epoch. These metrics are logged using `Neptune.ai` (Neptune Labs, 2025) for experiment tracking and visualization. Besides, `TensorBoard` (Google, 2024) is used to monitor the histogram of model learned parameters and gradients over time. This helps identify potential issues such as vanishing or exploding gradients during training.

All training hyperparameters are chosen based on the procedure described in section 5.2.9. The classical machine learning baselines described in section 5.2.5 are trained similarly, but only the last visit of each patient is used as input, together with the summary statistics of previous visits. Hyperparameter tuning for these models is also performed as described in section 5.2.9.

For both deep learning and classical machine learning models, the positive class corresponds to patients who were readmitted within 30 days after discharge from the current hospitalization, while the negative class corresponds to patients who were not readmitted within this period.

5.2.8 Model Evaluation

After training and hyperparameter tuning, the best configuration of each architecture was evaluated on the held-out test split produced by the data preprocessing pipeline (see section 5.2.3). The test set remained unseen during training and tuning to ensure an unbiased evaluation of model performance.

Evaluation followed the standard metrics for binary classification described in section 2.1: accuracy, precision, recall, F1-score, and AUC. The decision threshold for

converting predicted probabilities into class labels was determined based on F1-score optimization during training.

For the classical machine learning baselines (see section 5.2.5), the same test split and evaluation metrics were applied to maintain a fair comparison with the deep learning models. As these models use a single sample per patient, evaluation was performed at each patient’s last recorded hospital visit. For consistency, the deep learning models were also evaluated only at the last visit of each patient in the test set.

5.2.9 Hyperparameter Tuning

The goal of hyperparameter tuning is to find the best combination of model and training settings that performs well on the validation data split, which includes samples not seen during training. Firstly, the hyperparameters domains were defined based on exploratory analyses and experimentation. Afterwards, an *informed search* strategy is used to explore the hyperparameter space for the deep learning models, which have a much larger set of hyperparameters than the classical baseline ones. Unlike grid search, which tries all possible predefined combinations, or random search, which randomly picks combinations, informed search measures which hyperparameters most influence model performance and focuses on exploring them more.

This optimization strategy for the deep learning architectures was managed using the **Optuna** framework (Akiba et al., 2019). This library was chosen because it allows early stopping of unpromising trials based on intermediate results, which helps avoid wasting computational time on poorly performing configurations. The hyperparameters tuned with **Optuna** are listed in Tables 5 and 6. In total, for each architecture, 70 trials were run. The best configuration was selected based on the highest AUC score achieved on the validation set.

Table 5 – Hyperparameters for model training

Hyperparameter	Description
Batch size = {32, 64, 128}	Number of samples processed before updating model parameters.
Learning rate = $[1e-4, 1e-2]$	Initial step size for the optimizer during training.
Weight decay = $[0, 1e-1]$	L2 regularization coefficient to prevent overfitting.
Learning rate scheduler = {CosineAnnealing, ReduceLROnPlateau}	Strategy for adjusting the learning rate during training.

Table 6 – Hyperparameters of model architectures

Architecture	Hyperparameter	Description
All Models	Head dropout rate = $[0, 0.5]$	Dropout applied in the MLP classification head to reduce overfitting.
	Head hidden size (H) = $\{16, 32, 64, 128\}$	Size of the hidden layer in the MLP classification head.
GRUNet	Sequential hidden size (H) = $\{16, 32, \dots, 256\}$	Dimensionality of the GRU hidden state.
	Number of layers = $\{1, 2\}$	Number of stacked GRU layers in the recurrent encoder.
SimpleAttnPool	Embedding dimension M = $\{8, 16, 32, \dots, 128\}$	Size of the linear projection for visit embeddings in the attention block.
MultiHeadAttnPool	Embedding dimension M = $\{32, 64, \dots, 256\}$	Total dimension of concatenated heads; projection space size.
	Number of attention heads = $\{1, 2, 4, 8\}$	Parallel attention heads used to capture diverse patterns.
	Positional encoding = $\{True, False\}$	Whether positional encodings are added to visit embeddings.

For the classical machine learning baselines, which have fewer hyperparameters, tuning was carried out with random search using the `scikit-learn` library (Pedregosa et al., 2011) due to ease of use and lack of potential negative performance side effects (Garcia, 2025b). Cross-validation with five folds was used to evaluate each hyperparameter configuration for the same reasons. The best configuration was selected based on the highest average AUC score across folds. The search focused on the main parameters affecting model performance for Logistic Regression and Random Forest. The tuned parameters are reported in Table 7.

Table 7 – Hyperparameters of classical ML baseline models

Model	Hyperparameter	Description
Logistic Regression	C = $[1e-4, 1e4]$	Inverse regularization strength; smaller values specify stronger regularization.
	Penalty = $\{l1, l2\}$	Type of regularization to apply (L1 or L2).
	Fit-Intercept = $\{True, False\}$	Whether to fit the intercept term.

(continued on next page)

Model	Hyperparameter	Description
Random Forest	Number of estimators = {50, 100, 200, 300}	Number of trees in the forest.
	Max depth = {None, 3, 5, 10, 12}	Maximum depth of each tree.
	Min samples split = {2, 5, 10, 15, 20}	Minimum number of samples required to split an internal node.
	Min samples leaf = {1, 2, 4}	Minimum number of samples required to be at a leaf node.
	Max features = {sqrt, log2}	Number of features to consider when looking for the best split.

5.2.10 Feature Selection

In an attempt to improve model performance of the deep learning architectures described in section 5.2.6, a feature selection step was implemented prior to the final model training. The goal of the procedure is to identify the smallest set of informative features that maintains or improves predictive performance while avoiding overfitting caused by weak or irrelevant variables.

Each model is firstly trained with all available features, using a configuration fixed after hyperparameter tuning. Absolute IG attributions are then computed for every feature with the library **Captum** (Kokhlikyan et al., 2020). For longitudinal variables, attributions are averaged across time steps. Current and longitudinal features are ranked separately based on their mean absolute attribution.

Several reduced input spaces are then evaluated by retraining the architecture with feature subsets of increasing size. These subsets are constructed by selecting the top-ranked current hospitalization features together with the top-ranked longitudinal features, according to the IG-based absolute attributions scores. The following combinations are considered to balance combinational explosion and relevant results based empirical exploration:

1. 3 current + 2 longitudinal features;
2. 5 current + 3 longitudinal features;
3. 8 current + 4 longitudinal features;
4. 10 current + 5 longitudinal features, and

5. 15 current + 7 longitudinal features.

Each reduced model is trained and validated independently, and the feature subset with the highest validation AUC is selected. If the selected subset is smaller than the full feature set, it becomes the final input configuration for the architecture, which is then tuned again and retrained using the new reduced set. Otherwise, the same original non-reduced model is kept.

5.2.11 Additional Experiment: Two-Stage Training Approach

Given that the deep learning models did not outperform the classical baselines (see Chapter 6), an additional experiment was carried out to test whether a two-stage training strategy could improve performance.

In the first stage, the classification head of each architecture was pretrained using only the current-visit features, including data from patients without any prior visits. This step allowed the model to make better use of the MIMIC-III dataset, since patients with no longitudinal history represent a much larger portion of the cohort. During this stage, the component that encodes past visits (the attention or GRU block) was kept frozen so its weights remained unchanged. The aim was to let the classification head first learn stronger mappings from current-visit features to the target before introducing the complexity of longitudinal information.

In the second stage, the full model was fine-tuned using both current and longitudinal features, with all parameters unfrozen. Both stages followed the same training procedure described in section 5.2.7.

5.2.12 API for Inference

The backend was developed in Python using the **FastAPI** framework, which provides a fast and reliable interface between the hospital readmission prediction frontend and the trained analytics model. **FastAPI** was selected due to its simplicity, high performance, and integrated data validation feature (Ramírez, 2018). For demonstration purposes, the API currently relies on the **SimpleAttnPoolCurrQuery** architecture described in section 5.2.6.3.

The system exposes two main endpoints: **POST "/predict"** for generating readmission predictions and **POST "/explain_single_patient"** for providing explainability insights about the prediction for specific patients. If the input data does not match predefined schemas for both endpoints, for example due to missing fields or incorrect formats, validation error messages are returned to the user before any model execution occurs.

The endpoints developed are briefly described below. Further details about the API interface and data formats are provided in Appendix F.

5.2.12.1 POST `"/predict"`

This endpoint accepts hospitalization data structured as in Table 4. In addition to the required input tables, it can optionally receive the corresponding true target variables for 30-day readmission. These targets are used only for demonstration and evaluation purposes, hence, these targets are not part of the actual use of the system in a real hospital information system. Model performance metrics are returned only when such true targets are provided in the request.

The response includes the predicted probabilities of 30-day readmission for each patient in the batch, the predicted labels, and (if available) the provided true labels, together with performance metrics and execution metadata.

5.2.12.2 POST `"/explain_single_patient"`

This endpoint accepts the same hospitalization input data and can also optionally receive true target variables. It returns Integrated Gradients attributions computed with Captum (Kokhlikyan et al., 2020), including the attributions of longitudinal features derived from past hospitalizations, the attributions of the current-admission features, and a split highlighting the relative contribution of longitudinal versus current feature sets. The derived features used by the model are also returned.

5.2.13 Web Application (Frontend)

A web application was developed in Python with the library Streamlit (Streamlit Inc., 2025). Its main goal is to allow both clinicians (or any other experts) and data scientists to interact with the backend API described in section 5.2.12.

The web application allows clinicians to upload hospitalization data in CSV format and view the model's predicted probability of 30-day readmission for all patients in the CSV files, together with explanations that highlight which factors most influence each prediction. Data scientists could also do the same and use the application to monitor model behavior, specially based on the Integrated Gradients attributions and the performance metrics returned by the API.

These two actors interact with the application through the use cases shown in Figure 15.

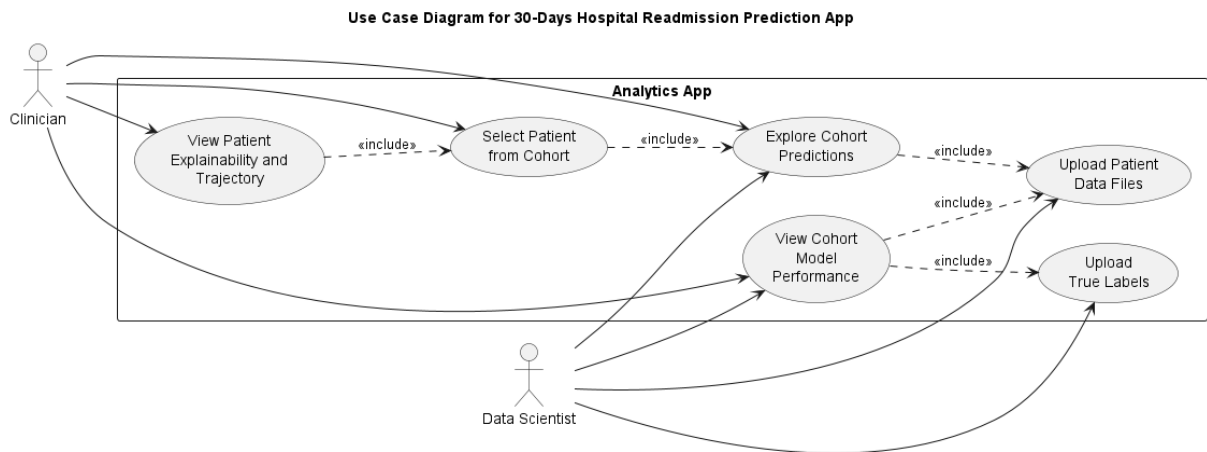


Figure 15 – Use cases of the web application developed for hospital readmission prediction. Users can upload hospitalization data, obtain predictions from the backend API, and visualize explainability insights for individual patients

As illustrated in Figure 15, both actors must first upload the hospitalization data files (optionally, true labels for the events as well) and request a prediction. In the interface, this is done in the sidebar of web page (see Figures 16 and 17).

Once the data are loaded, the user can visualize cohort-level predictions (use case *"Explore Cohort Predictions"*). A table is displayed with each patient's predicted probability of 30-day readmission, the corresponding predicted label based on the model's probability threshold, and ranking information such as cohort percentile. A histogram of predicted probabilities and the total number of predicted readmissions are also displayed (see Figures 18 and 19).

After the predictions are available, the user may select an individual patient (Figure 20, use case *"Select Patient from Cohort"*) to inspect its specific results, explainability outputs and its clinical trajectory (use case *"View Patient Explainability and Trajectory"*). This view presents the longitudinal and current-admission feature values, together with their Integrated Gradients attributions. Attributions for past features are aggregated across all previous hospitalizations, and the relative contribution of longitudinal versus current feature sets is also shown (see Figure 21). Attention weights assigned to each past visit are visualized as well, providing insights into which hospitalizations the model considered most relevant for the prediction.

Finally, the system provides cohort-level model performance metrics, including AUC, precision, recall, F1 score, the confusion matrix, and a calibration curve, which allow users—particularly data scientists—to assess how the model performs on the uploaded dataset (use case *"View Model Performance Metrics"*). These metrics are displayed in a dedicated tab of the web application (see Figure 22).

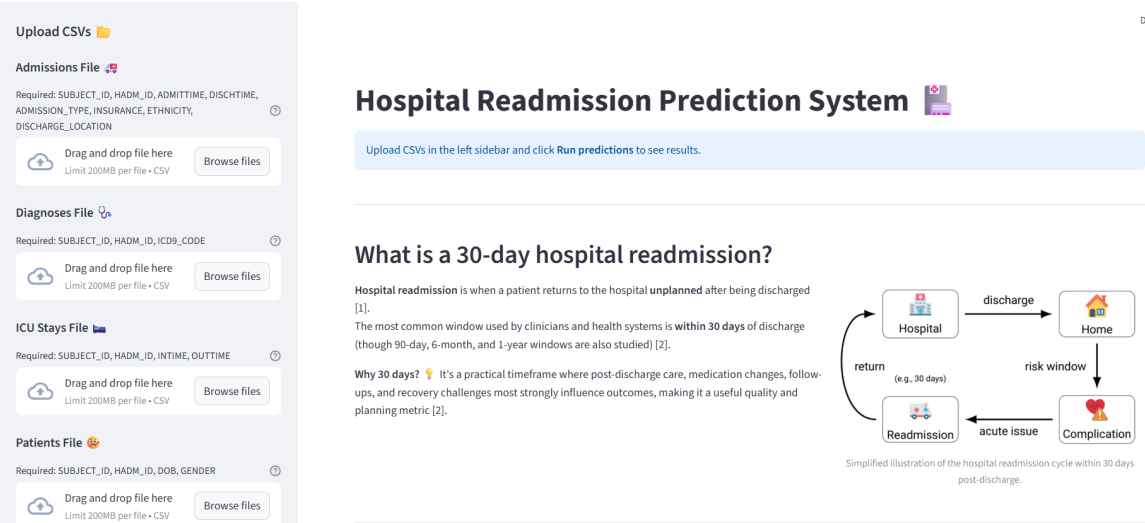


Figure 16 – Home screen of the web application for hospital readmission prediction. Users can upload hospitalization data and request predictions from the backend API

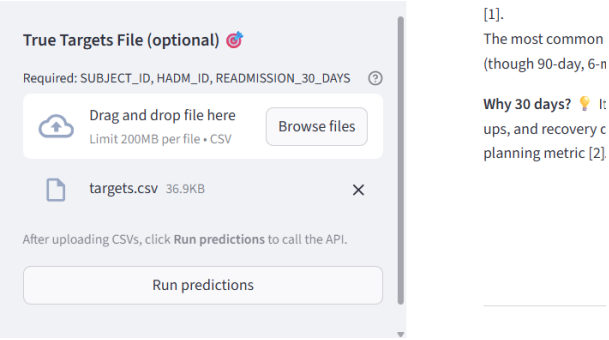


Figure 17 – Data upload interface of the web application. Users can upload hospitalization data files in CSV format and optionally provide true labels for evaluation

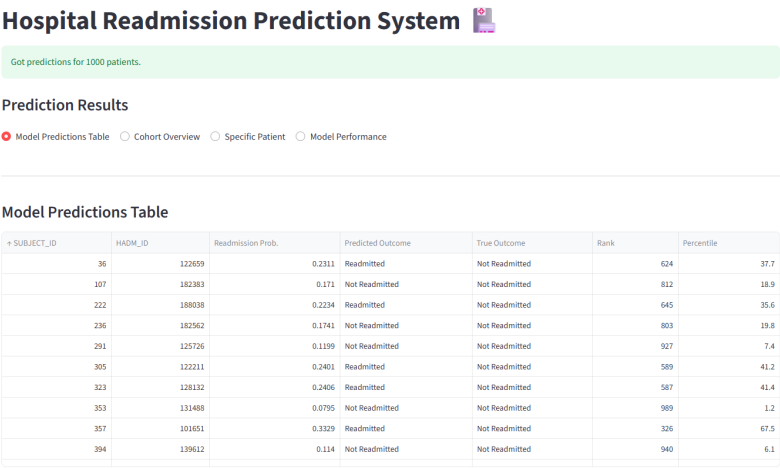


Figure 18 – Cohort predictions table in the web application. Users can view each patient's predicted probability of 30-day readmission, predicted label, and ranking information

Hospital Readmission Prediction System

Prediction Results

☐ Model Predictions Table ☒ Cohort Overview ☐ Specific Patient ☐ Model Performance

Cohort Overview

Total Patients: 1,000 Expected Readmissions: 651 Total Readmissions: 259

A probability threshold of 0.22 is used.

Distribution of Readmission Probabilities by True Outcome

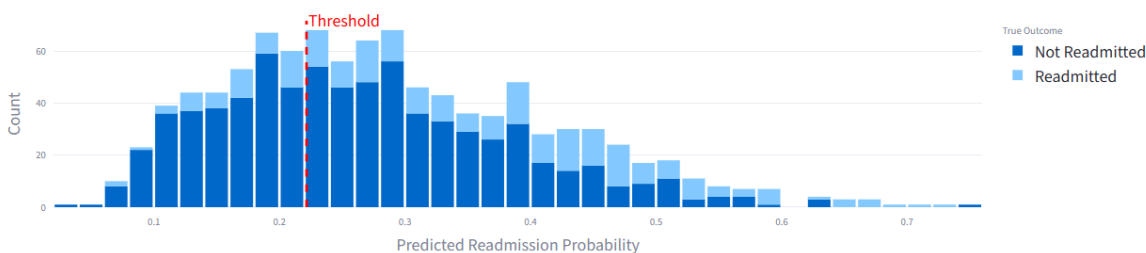


Figure 19 – Cohort overview tab in the web application. Users can visualize a histogram of predicted probabilities and the total number of predicted readmissions

Hospital Readmission Prediction System

Prediction Results

☐ Model Predictions Table ☐ Cohort Overview ☒ Specific Patient ☐ Model Performance

Results for Specific Patient

Select Patient SUBJECT_ID: 2187 Check Results

Tip: changing the selector won't run the model until you click Check Results.

Readmission Prob.	True Outcome Readmitted	Risk Rank	Risk Percentile
38.2%	No	237	76.40

Admission Details of Patient 2187

Current Admission Past Admissions Feature Attributions

Current Hospital Admission Details

Figure 20 – Specific patient overview in the web application. Users can inspect the longitudinal and current-admission feature values, along with their Integrated Gradients attributions and attention weights assigned to past visits

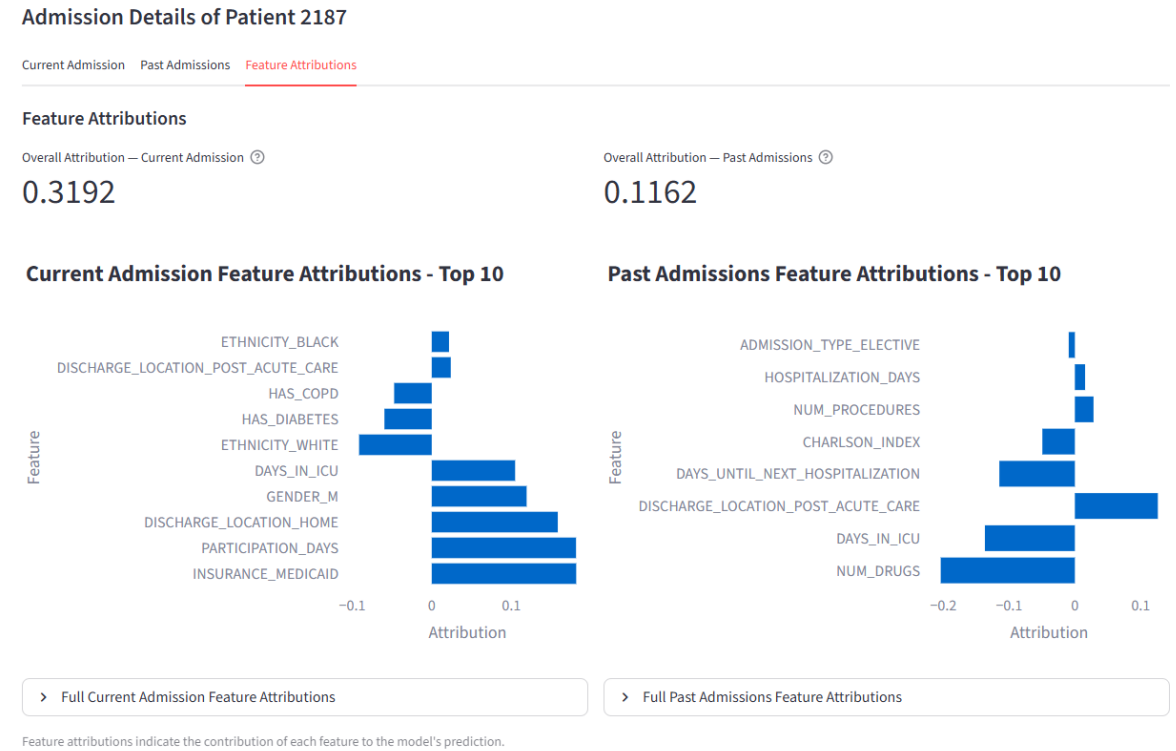


Figure 21 – Patient explainability in the web application. Users can inspect Integrated Gradients attributions for longitudinal and current-admission features

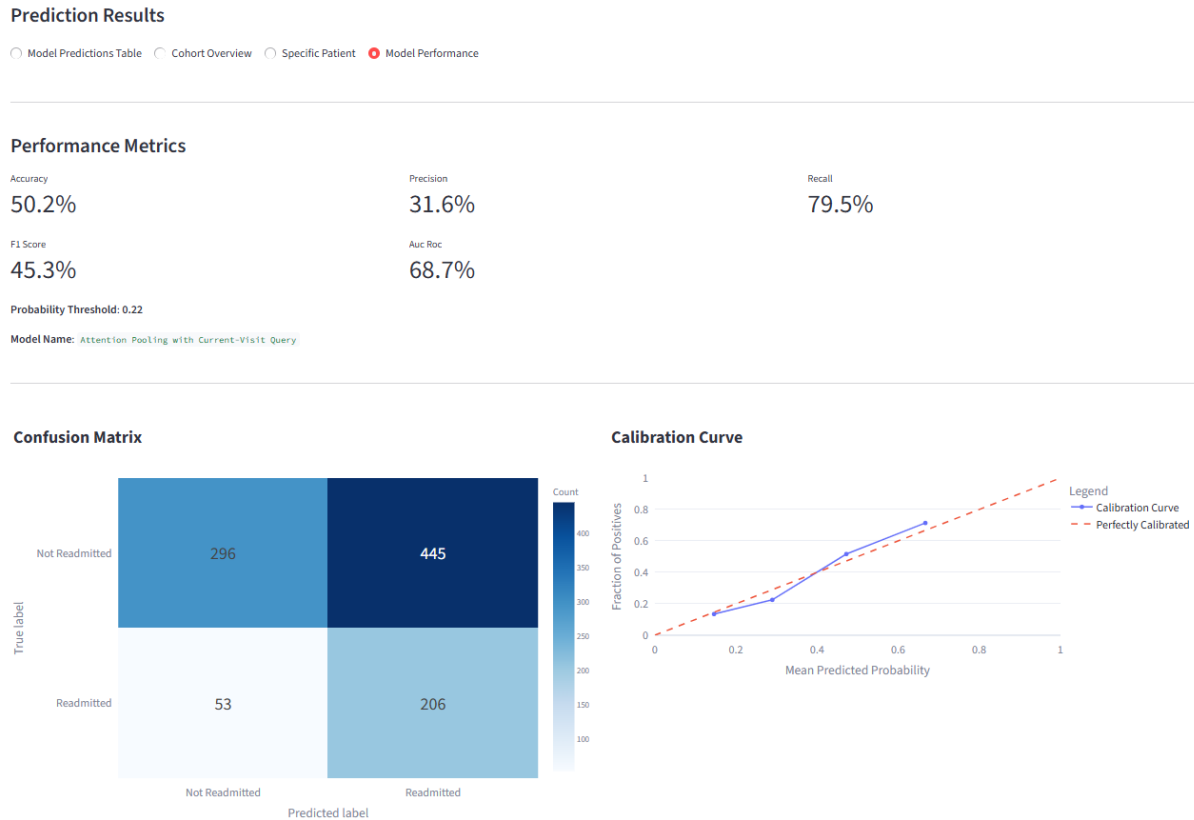


Figure 22 – Model performance metrics tab in the web application. Users can assess the model's performance using AUC, precision, recall, F1 score, confusion matrix, and calibration curve

5.3 Testing

In this section, an overview of the test strategy used to verify the correct functioning of the different components developed in this project is provided. The goal is also to provide needed evidence to support whether the requirements defined in Chapter 4 have indeed been met. In section 5.3.1, the unit tests implemented for individual modules are described. End-To-End (E2E) tests, which verify the interaction between the frontend and the backend API, are presented in section 5.3.2.

5.3.1 Unit Tests

Unit tests were carried out following a test-driven development approach. This approach was implemented using `pytest` (Krekel et al., 2004), a popular testing framework in Python. The main goal of these tests in this work is to verify the correct functionality of the following elements:

- Data preprocessing functions;
- Feature engineering methods;
- PyTorch dataset objects used to load data during model training and evaluation;
- API endpoints for inference.

As seen in Figure 23, all unit tests pass successfully, indicating that the individual elements behave as expected.

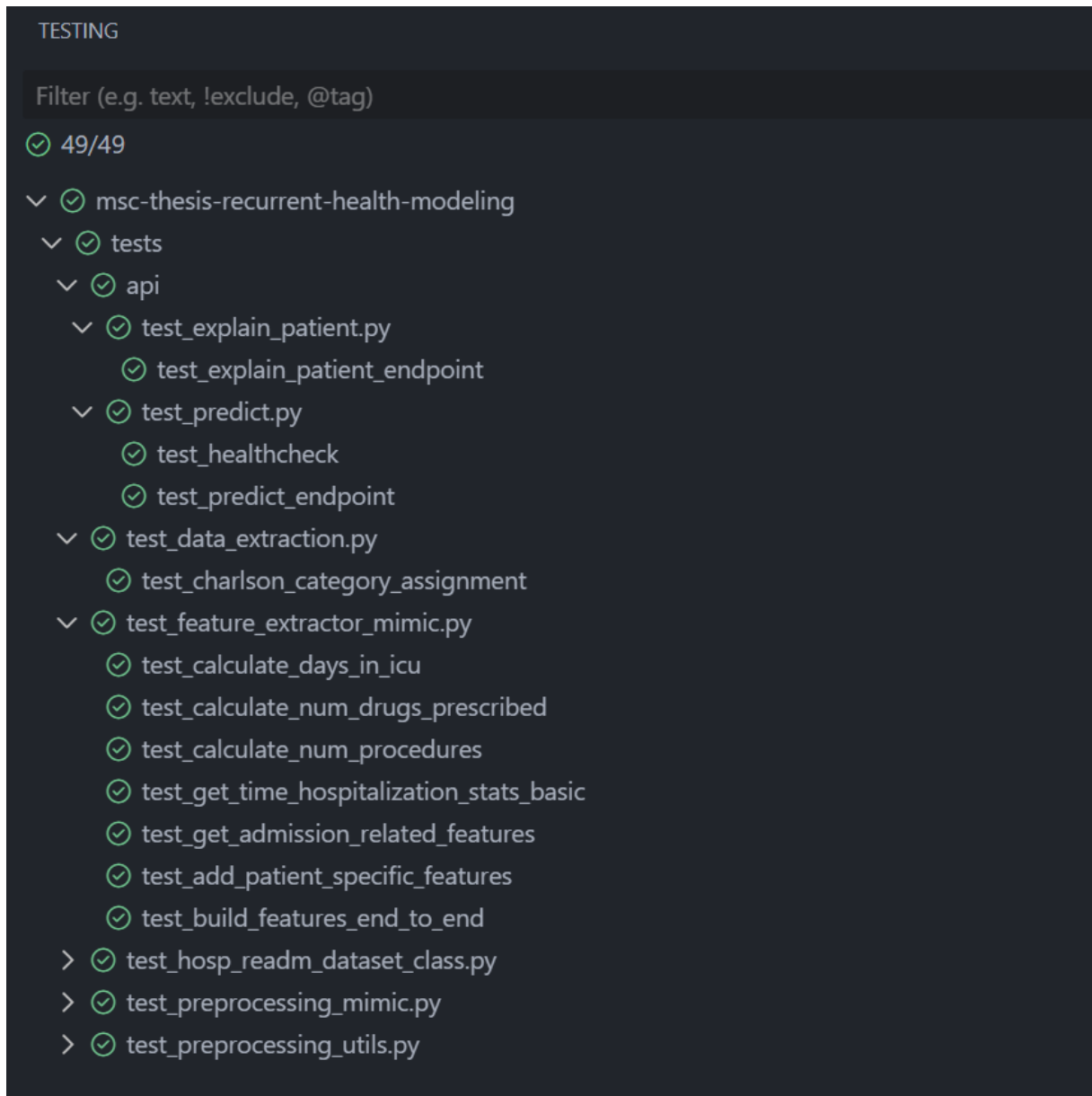


Figure 23 – Execution of unit tests using the `pytest` framework. The tests cover API endpoints, data preprocessing functions, feature engineering methods and `PyTorch` datasets objects

Such tests consist of functions that check if the output of specific methods under test given predefined inputs match expected results. Such predefined inputs are fixed according to the expected behavior of each method.

5.3.2 End-To-End Tests

A set of manual E2E tests were carried out to verify the correct interaction between the web application (frontend) and the backend API for inference. The main goal of these tests is to ensure that data uploaded through the web interface is correctly sent to the API, processed by the model, and that predictions and explainability outputs are accurately returned and displayed in the frontend. With that, it is possible to assess if

the evidence collected from the tests supports that functional requirements defined in Chapter 4 are met.

The E2E tests carried out are listed in Tables 8 and 9. These tests were designed to follow the main user workflows in the use case diagram of Figure 15. For each one of them, the preconditions, steps to execute, expected results, and the functional requirements (FR) whose verification they support.

Table 8 – End-to-End Test Cases (Part 1)

Test Name	Preconditions	Steps	Expected Result	FR
Connectivity check	Backend API and web application running.	Launch the web application, expand the sidebar section Backend , and observe the API healthcheck status.	The interface reports that the API is reachable and no communication errors occur.	—
Valid file upload	Valid CSV files described in section 5.2.1 are available.	In the sidebar, use the file inputs to submit the required CSV files and verify that they are listed as uploaded.	All required files are accepted without errors and become available for subsequent processing.	FR-01
Invalid file handling	CSV files with missing fields or incorrect schema are available.	In the sidebar upload panel, upload invalid CSV files and observe the feedback shown by the interface.	The system rejects invalid files and displays appropriate validation or error messages to the user.	FR-02, FR-03
Cohort prediction request	Backend API reachable and valid CSV files uploaded.	In the sidebar, click the Run predictions button and wait for the operation to complete.	The prediction workflow is triggered, the backend processes the cohort, and a success message confirms that predictions were obtained.	FR-04

Table 9 – End-to-End Test Cases (Part 2)

Test Name	Preconditions	Steps	Expected Result	FR
Prediction table display	Predictions successfully returned by the backend.	In the horizontal navigation control, select Model Predictions Table and inspect the displayed results.	A table is shown with patient identifiers, readmission probabilities, risk ranks and basic interactions such as sorting and filtering.	FR-05
Cohort summary plots	Cohort predictions available; true labels optionally uploaded.	In the navigation control, select Cohort Overview and review the metrics and plots.	Summary metrics and the probability distribution plot are displayed, along with counts of expected and (if labels exist) true readmissions.	FR-06
Performance metrics display	True outcome labels uploaded and predictions computed.	Select the Model Performance view and inspect the metrics and visualizations.	Performance metrics, confusion matrix and the calibration curve are shown in the interface.	FR-07, FR-08
Single-patient explanation	Cohort predictions available for at least one patient.	In the navigation control, select Specific Patient , choose a patient by SUBJECT_ID from the dropdown, and click the Check Results button; then check the tabs Current Admission , Past Admissions , and Feature Attributions .	The single-patient view displays readmission probability, true outcome (if available), risk rank and percentile, admission details, temporal plots, attention weights, and feature attributions.	FR-9, FR-10

6 Results Analysis

In this chapter, the final results obtained with experiments of model tuning, feature selection, training and evaluation (described previously in sections 5.2.9, 5.2.7 and 5.2.8) are presented and analyzed from section 6.1 to 6.7. In sections 6.8 and 6.9, the response time of the API endpoints and the ease of use of the web application are analyzed. Both correspond to non-functional requirements in Table 2.

6.1 Classical ML Baselines Results

The results of the baseline classical machine learning models (Random Forest and Logistic Regression) are presented in Tables 10 and 11. Such models were trained and evaluated together in a single **Neptune** experiment according to the strategy described in section 5.2.7. As mentioned in that section, the positive class corresponds to patients who were readmitted within 30 days after discharge, while the negative class corresponds to patients who were not readmitted within that period.

Table 10 – Baseline classifiers: training and cross-validation performance metrics

Neptune ID	Model	Prob. Threshold	Train AUC	CV AUC ($\mu \pm \sigma$)
REC-415	Random Forest	0.496	0.716	0.622 +/- 0.040
REC-415	Logistic Regression	0.230	0.651	0.620 +/- 0.044

Note: CV = Cross-Validation; μ = Mean; σ = Standard Deviation. 5-fold cross-validation was used.
Number of patients in train set: 1,412 (positive: 27.4%; negative: 72.6%).

Table 11 – Baseline classifiers: test performance metrics

Neptune ID	Model	Test AUC Last Events	Test F1-Score Last Events	Test Accuracy Last Events
REC-415	Random Forest	0.634	0.421	0.572
REC-415	Logistic Regression	0.618	0.420	0.467

Note: Number of patients in last events test set: 353 (positive: 25.8%; negative: 74.2%).

In Table 10, it can be observed that both models achieved similar cross-validation AUC scores, with Random Forest slightly outperforming Logistic Regression. The standard deviation values represent roughly 6.5% of the mean performance for both models, which indicates a high variability across different folds. Such variability may be attributed to the limited size of the dataset, which can lead to overfitting in certain folds. Since the range

of AUC values $\mu \pm \sigma$ for both models overlap significantly, it is not possible to definitively state that one model is superior to the other based on cross-validation results alone.

The AUC metrics on the test set, performed considering only the last events of each patient, are in the interval 0.61 – 0.63, which indicates a low to moderate discriminative ability of both models. They also fall below the often reported performance range in the literature described in section 2.4.2: (inter quartile range: 0.64–0.76). The F1-Score values are also relatively low, suggesting that the models struggle to balance precision and recall effectively.

In Table 11, Logistic Regression uses a much lower probability threshold than Random Forest. This helps the model deal with the class imbalance in the test set (25.8% positive cases). A lower threshold increases recall and improves the F1-Score because it reduces false negatives. However, this also lowers accuracy, which is expected: accuracy is heavily influenced by the majority class, and a model that predicts, for example, every patient as non-readmitted would still get a high accuracy but have no real predictive value.

The underperformance of these baseline models compared to literature benchmarks was also observed by Garcia (2025b). As stated in this previous work, the reason behind it is likely a consequence of the following factors:

1. The dataset used for the baseline models includes only 1,412 patients. This small sample may not give the models enough information to learn complex readmission patterns. The limited size comes from focusing only on patients with more than one hospital readmission, which significantly reduces the number of eligible cases in the original MIMIC-III dataset;
2. The high class imbalance in the dataset, with a significantly lower proportion of positive cases (readmissions) compared to negative cases (non-readmissions). This imbalance can lead to biased model predictions favoring the majority class;
3. The models were trained using simple summary features, such as ICU length of stay, number of diagnoses, and number of medications. These aggregated features likely lack detail and may not fully capture the patient’s health status. An important hint of this limitation is the low correlation values between the individual features and the binary target readmission variable, as shown in section 5.2.4.

Many studies report stronger performance by using larger datasets, not restricting their samples to patients with multiple readmissions, and by applying more advanced feature engineering techniques (Garcia, 2025b).

In Figure 24, the SHAP (Lundberg; Lee, 2017) feature importances for the Random Forest baseline model are presented.

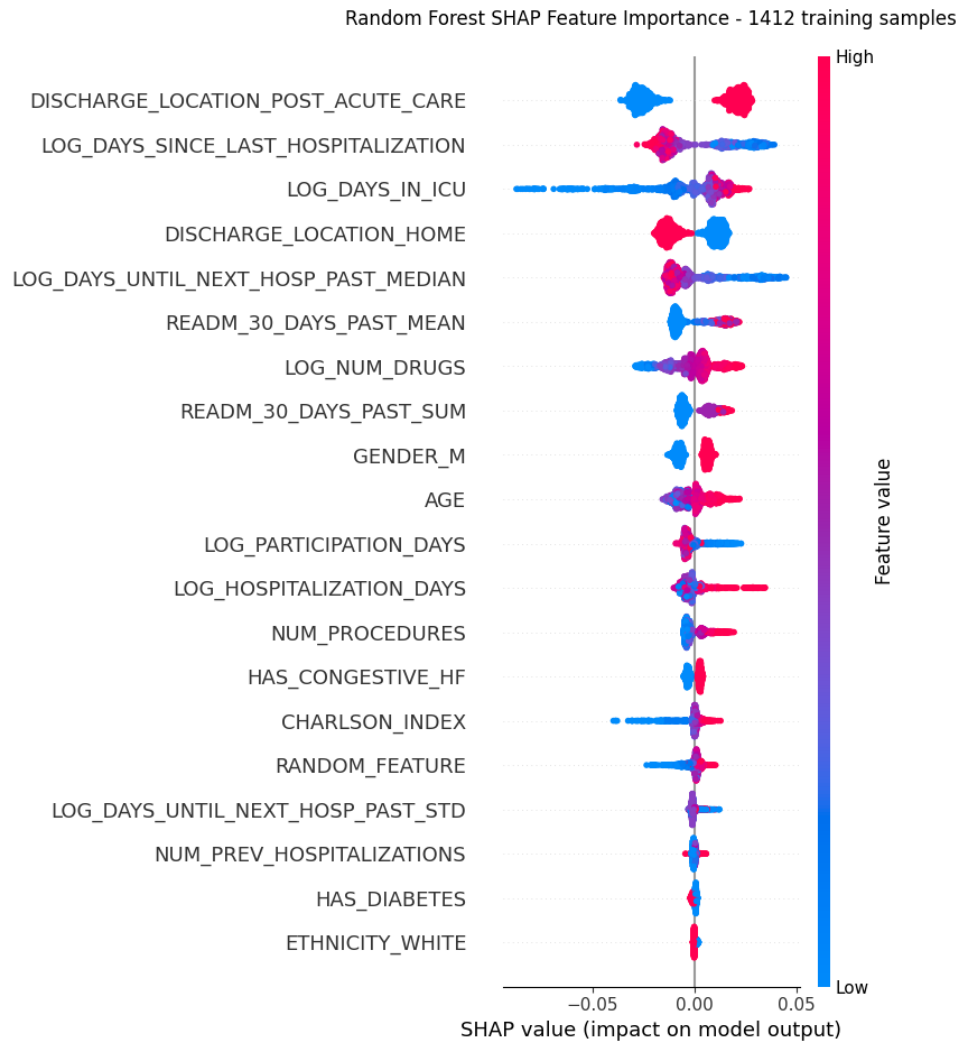


Figure 24 – SHAP feature importances – Random Forest baseline model. Features are ranked by their overall importance in predicting 30-day readmission

The SHAP plot shows intuitive behavior: for example, lower values of `LOG_DAYS_IN_ICU` (blue) tend to yield negative SHAP values, decreasing the predicted readmission risk, while lower values of `LOG_DAYS_UNTIL_NEXT_HOSP_PAST_MEDIAN` (blue) push SHAP values higher, increasing the predicted probability.

Moreover, the past summary features `LOG_DAYS_UNTIL_NEXT_HOSP_PAST_MEDIAN` and `READM_30_DAYS_PAST_SUM` stand out with higher importance than many other predictors, indicating that they capture a substantial share of the relevant signal from prior hospitalization patterns.

The SHAP feature importances obtained for the Logistic Regression baseline model also showed similar patterns, with small differences in the ranking of features.

6.2 Deep Learning Feature Selection

The results of the feature selection experiments for each deep learning architecture are presented in Table 12. In this table, "Full" denotes the configuration with all features included. Such experiments were carried out varying the number of input features according to the procedure described in section 5.2.10. As an example, the different AUC scores obtained for the architecture `SimpleAttnPool` are illustrated in Figure 25, while the feature importances computed with absolute IGs for the same architecture are shown in Figures 26 and 27.

Table 12 – Deep learning: feature selection results

Neptune ID	Architecture	Running Time	Best/Full AUC	Best/Full N° Curr. Feat.	Best/Full N° Long. Feat.
REC-490	GRUNet	73s	0.649/0.645	15/19	7/8
REC-489	MultiHeadAttnPool	60s	0.659/0.659	19/19	8/8
REC-488	SimpleAttnPoolCurrQuery	38s	0.642/0.642	19/19	8/8
REC-486	SimpleAttnPool	52s	0.644/0.643	10/19	5/8

Note: Long. Feat. = Longitudinal Features; Curr. Feat. = Current Admission Features. Full = Configuration with all features included. Best = Configuration with highest validation AUC achieved during feature selection experiments.

Overall, it is possible to note that most architectures achieved their best AUC scores when using all available features. The only exceptions are the `SimpleAttnPool` and `GRUNet` architectures, which obtained a slightly higher AUC with less than the full set of features. The observed improvements are negligible, though. Therefore, it can be concluded that feature selection did not lead to significant performance gains for the deep learning models in this specific task and dataset.

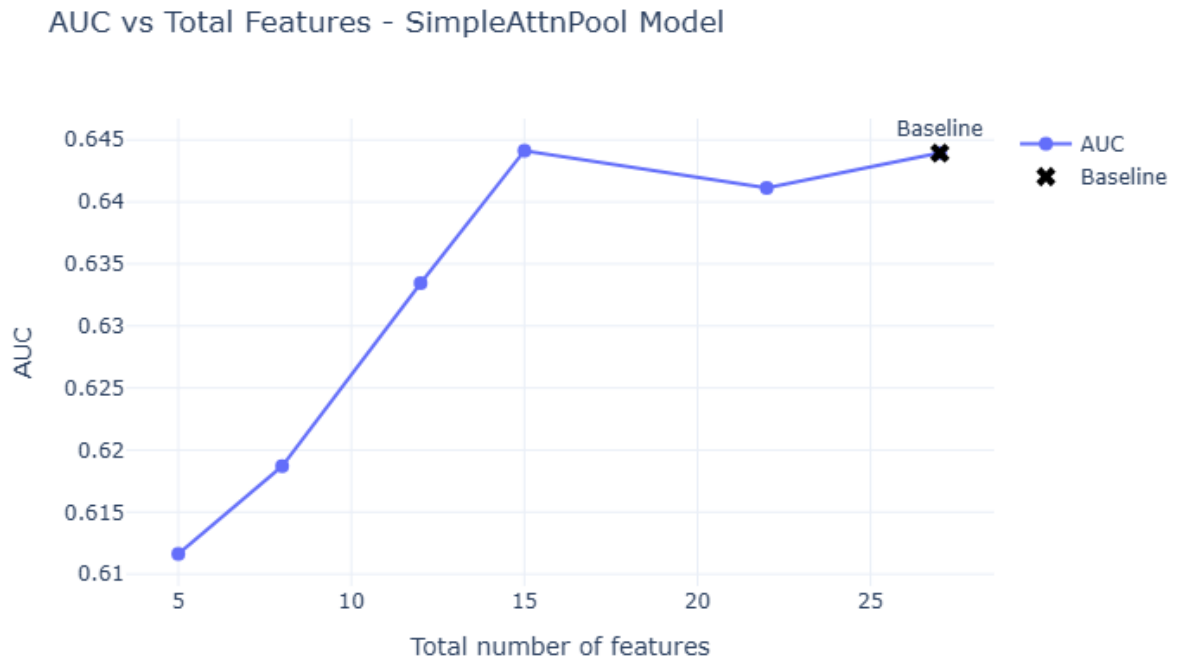


Figure 25 – Feature set selection – SimpleAttnPool example: AUC vs. number of features

Regarding the feature importances computed with IG for the **SimpleAttnPool** architecture, it can be observed in Figures 26 and 27 that the current admission features tend to have much higher importance than the longitudinal features. Most longitudinal features have IG attributions lower than 0.04, while most current admission features have higher values than 0.04. This suggests that, for this specific task and dataset, the most recent hospitalization contains more relevant information for predicting 30-day readmission than the patient’s historical data. Another explanation for this behavior is the fact that most patients in the dataset have few past hospitalizations (see section 5.2.4), which probably limited the model from extracting useful patterns from the longitudinal features.

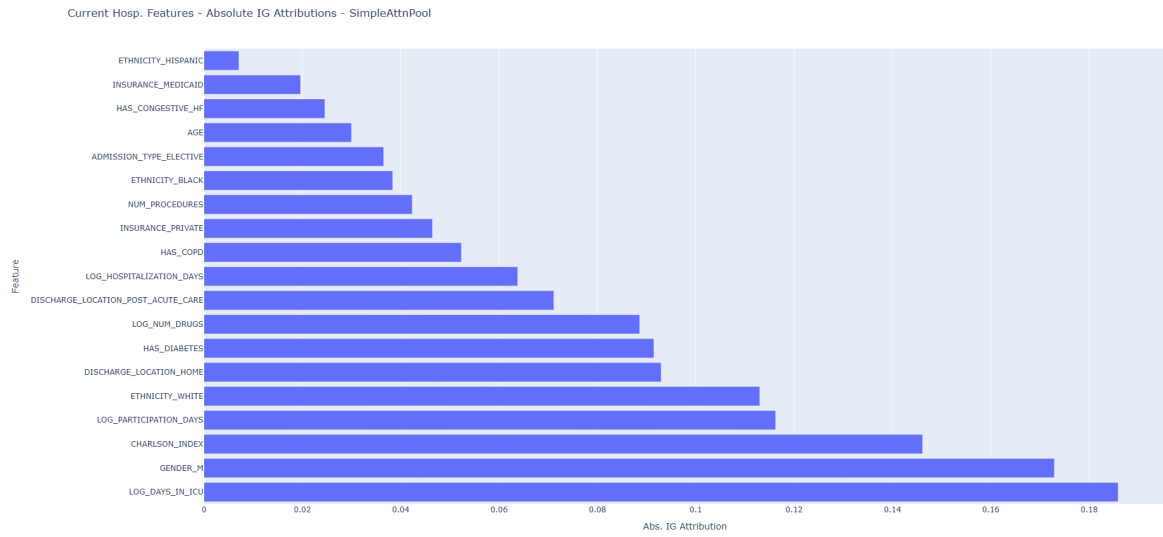


Figure 26 – IG attributions for current features – SimpleAttnPool example. Features in the bottom have higher importance

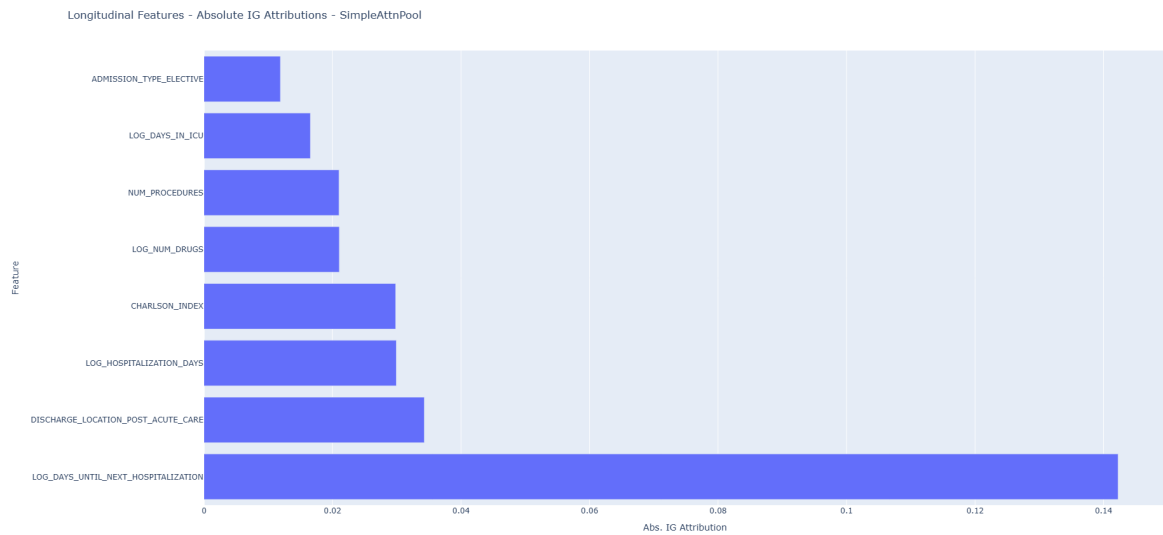


Figure 27 – IG attributions for longitudinal features – SimpleAttnPool example. Features in the bottom have higher importance

During the experiments execution, it was also noticed that the feature importance ranking had slightly different orders when the same architecture was trained with the same configuration but with different random seeds. This indicates that the feature importance results may be sensitive to the initial conditions of the training process, which is a common characteristic of deep learning models due to their non-convex optimization landscapes (Goodfellow; Bengio; Courville, 2016). As future work, ensemble methods or averaging techniques to obtain more stable feature importance estimates could be explored.

When comparing the feature importance results from SimpleAttnPool shown in Figures 26 and 27 with those obtained from the Random Forest baseline using SHAP values (section 6.1), some interesting differences and similarities emerge.

Among the differences, for example, features like `GENDER_M`, `CHARLSON_INDEX`, and `LOG_PARTICIPATION_DAYS` rank high in SimpleAttnPool but appear far less important in the Random Forest results. This suggests that different model architectures focus on different signals in the data. Such divergences require further investigation to understand the underlying reasons, which may include differences in how each model captures feature interactions or handles non-linearities.

At the same time, there are clear points of agreement. Both models highlight the strong influence of `LOG_DAYS_IN_ICU` and timing-related hospitalization features (e.g., `LOG_DAYS_UNTIL_NEXT_HOSP_PAST_MEDIAN` and `LOG_DAYS_UNTIL_NEXT_HOSPITALIZATION`). They also assign moderate but meaningful importance to `DISCHARGE_LOCATION_HOME` and `LOG_NUM_DRUGS`.

6.3 Deep Learning Model Tuning

After selecting the best input feature sets of each deep learning architecture, the hyperparameter tuning procedure described in section 5.2.9 was carried out. The results of this step are presented in Table 13.

Table 13 – Deep learning: model tuning results

Neptune ID	Architecture	Running Time	Best Validation AUC	Final N° Parameters
REC-462	GRUNet	5min 29s	0.6558	3617
REC-491	SimpleAttnPool	4min 41s	0.6533	1890
REC-460	MultiHeadAttnPool	2min 33s	0.6496	28097
REC-475	SimpleAttnPoolCurrQuery	1min 32s	0.6496	2930

Although hyperparameter tuning is usually a computationally expensive task in many deep learning projects, the relatively small size of the dataset used allowed for a quick experimentation cycle. The time taken was considerably reduced by the informed search strategy with pruning described in section 5.2.9, which avoided unnecessary training of poorly performing configurations.

Overall, it is possible to notice that the best validation AUC scores achieved are all around 0.65. Even MultiHeadAttnPool, which has a very high number of parameters (28,097) compared to the others, did not have a higher performance. The validation AUC scores observed in Table 13 are still close to the ones observed for the baseline Random Forest and Logistic Regression models in section 6.1. This is a first hint that, even after hyperparameter tuning, the deep learning architectures were not able to outperform classical machine learning models on this specific task and dataset.

6.4 Deep Learning Training and Evaluation Results

After the hyperparameter tuning step, the best configurations of each deep learning architecture were trained and evaluated according to the procedure described in sections 5.2.7 and 5.2.8. The results of the training step are presented in Table 14, while Table 15 depicts the evaluation results on the test set.

Table 14 – Deep learning: training performance metrics

Neptune ID	Architecture	Running Time	N° Epochs	Train AUC	Train F1-Score	Prob. Threshold
REC-467	GRUNet	31s	11	0.698	0.484	0.221
REC-492	SimpleAttnPool	53s	13	0.677	0.479	0.262
REC-464	MultiHeadAttnPool	36s	9	0.662	0.468	0.219
REC-476	SimpleAttnPoolCurrQuery	30s	11	0.713	0.503	0.259

Note: Number of samples in train set: 4003 (positive: 26.5%; negative: 73.5%) of 1482 patients. Prob. Threshold = Probability Threshold for classifying positive cases, selected based on maximizing the F1-Score on the train set.

Table 15 – Deep learning: test performance metrics

Neptune ID	Architecture	Test AUC All Events	Test AUC Last Events	Test F1-Score Last Events	Test Accuracy Last Events
REC-467	GRUNet	0.603	0.619	0.432	0.561
REC-492	SimpleAttnPool	0.589	0.608	0.445	0.584
REC-464	MultiHeadAttnPool	0.600	0.605	0.424	0.408
REC-476	SimpleAttnPoolCurrQuery	0.591	0.601	0.426	0.473

Note: Number of samples in last events test set: 353 (positive: 25.8%; negative: 74.2%). Number of samples in all events test set: 1,085 (positive: 25.7%; negative: 74.3%).

The evaluation results combining both the classical models and deep learning models on the last events test set are also illustrated in Figure 28 for better comparison.

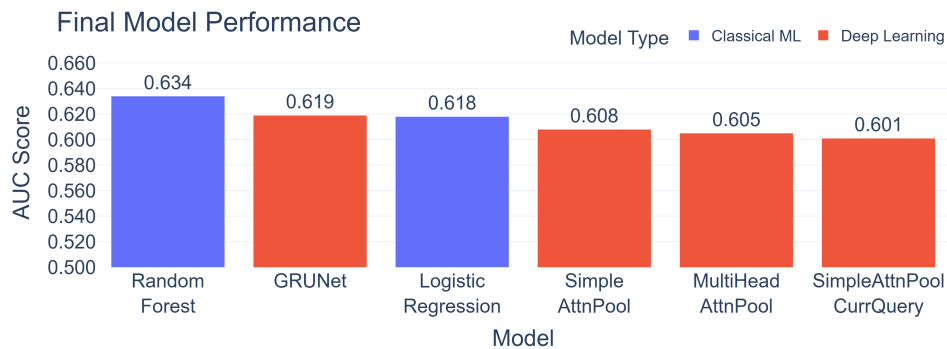


Figure 28 – AUC scores comparison in last events set: Deep learning models vs. classical ML baselines

The training set used in the experiments presented in Tables 14 is larger than the one used in the hyperparameter tuning step (4,003 samples vs. 1,412 samples). This is because such models were trained with the sliding window approach described in section 5.2.7, which generates multiple samples per patient. As mentioned in that section, this approach is valid because these models are designed to handle sequential data and can learn from multiple time steps per patient.

The training AUC scores presented in Table 14 range from 0.66 to 0.71. Such values were prevented from being too high (≥ 0.8) by the early stopping mechanism described in section 5.2.7, which stops the training when validation performance does not improve after a certain number of epochs. As a result, overfitting is mitigated.

In Table 14, the probability thresholds selected for classifying positive cases (readmissions) are all below 0.3. This is similar to what was observed for the Logistic Regression baseline model in section 6.1, and it is likely a consequence of the highly imbalanced nature of the dataset (only 25.8% positive cases). Consequently, lower thresholds helped increase the F1-Scores of Table 15 by improving recall, but at the cost of accuracy (only around 50% in most cases).

However, the last-events test AUC scores shown in Table 15, computed on the same unseen patients used to evaluate the classical ML models in section 6.1, fall within a similar range as the baselines (0.60–0.62). Although some models achieve slightly higher or lower values, the differences are small (within 0.02) and therefore not large enough to conclude that any model clearly outperforms the others. In practice, AUC estimates can vary across runs due to factors such as random initialization, stochastic optimization, and data-split variability. As a result, repeated experiments could easily shift the observed scores within this narrow range, making it difficult to definitively rank the models based on such small differences.

Therefore, there is no evidence to support the initial hypothesis that deep learning models would generalize better than the classical ML models, despite their more complex architectures and their access to the full history of the last four hospital admissions for each patient (rather than only summary features of the most recent admission, as in the baselines). Challenges of the dataset described in section 5.2.4 likely played a significant role in limiting the potential advantages of deep learning in this specific task.

Despite incorporating longitudinal features, the deep learning models have very limited temporal information to learn from. As shown in the EDA (Figure 5), most patients have only one or two previous admissions, meaning that only a small portion of the cohort provides meaningful sequences. As anticipated in section 5.2.4, the results in Table 11 confirm that the available histories are too short for GRU or attention mechanisms to extract temporal patterns. In practice, these architectures behave similarly to simpler feed-forward models, while the baseline classifiers already capture most of the signal

present in the summary features of the most recent hospitalization.

A second limitation, also identified in section 5.2.4, is the weak predictive power of the input features. Most variables are aggregated counts or simple summaries, offering only coarse information about patient health and its progression over time. The correlation analyses in the appendix show that their associations with 30-day readmission are generally small. Without richer temporal features – such as laboratory trends, vital signs, or clinical notes – the deep learning models cannot leverage their capacity to learn complex patterns, which helps explain why their performance remains similar to the classical baselines.

6.5 Two-Stage Approach Results

The results of the two-stage approach described in section 5.2.11 are presented in Table 16. As described in that section, the goal was to try to improve the results presented in section 6.4 by pretraining the deep learning architectures with additional data from patients with only one hospitalization (7,095 patients), then fine-tuning them with data from patients with multiple admissions.

Table 16 – Deep learning: two-stage approach test performance metrics

Neptune ID	Architecture	Test AUC All Events	Test AUC Last Events	Test F1-Score Last Events	Test Accuracy Last Events
REC-504	GRUNet	0.594	0.573	0.445	0.329
REC-506	MultiHeadAttnPool	0.624	0.612	0.461	0.456
REC-502	SimpleAttnPoolCurrQuery	0.616	0.605	0.451	0.380

Note: Models first pretrained with additional data of the first hospitalization of 7,095 patients with no previous admissions, then fine-tuned with data of patients with multiple admissions. Only 7.9% of patients in the pretraining dataset had positive cases (readmissions).

However, it is possible to notice that the two-stage approach did not lead to performance improvements compared to the results presented in Table 15. Only the architecture MultiHeadAttnPool achieved a slightly higher last events test AUC (0.61 vs. 0.60). The performance of the other architectures actually decreased. It is also interesting to note that the test accuracy values are very low, especially for GRUNet and SimpleAttnPoolCurrQuery, whose accuracies are below 40%. This might be a consequence of the difference in class imbalance of the pretraining dataset (only 7.9% positive cases) compared to the fine-tuning and test datasets (around 25.8% positive cases). Such difference happens because patients with no recurrent hospitalizations are less likely to be readmitted within 30 days. Therefore, the models might have learned biased representations during pretraining that did not transfer well to the fine-tuning and test phases.

6.6 Assessment of Model Performance Functional Requirements

The results presented in section 6.4 indicate that neither of the function requirements stated in Table 1 regarding model performance (FR-11 nor FR-12) was satisfied.

According to FR-11, an AUC of at least 0.70 on unseen data is desirable. However, all deep learning and classical models reached AUC values in the range of 0.60–0.62.

As specified in FR-12, at least one of the developed deep learning models should outperform a Logistic Regression baseline by at least +0.05 AUC on unseen data, which also did not occur. The observed differences between models were small – typically below 0.02 – and within the variability expected from factors such as random initialization, stochastic optimization, and data-split randomness. Therefore, there is no evidence that the deep learning models perform either substantially better or substantially worse than the classical baselines.

6.7 Examples of Explainability Results

In this section, the objective is to show how the explainability results shown by the web application described in section 5.2.13 could help clinicians understand the model’s predictions. All examples provided in this section are based on the final trained model of the `SimpleAttnPoolCurrQuery` architecture, which is currently used by the API for inference (see section 5.2.12 for more details).

The architecture `SimpleAttnPoolCurrQuery` is used by the API as an arbitrary example, but any of the other developed architectures could be used instead, as there is no evidence that one model outperforms the others (section 6.4).

6.7.1 IG Attributions for Individual Predictions

For illustrating how IG attributions can help explain model predictions, two patients are used as examples: one (subject 24846) with a high predicted risk of readmission (67.3%), and another (subject 29810) with a low predicted risk (6.4%).

The IG attributions plots obtained with the project’s web application for the high risk patient (subject 24846) are shown in Figure 29. Figure 30 presents the associated tables with feature values of this same patient and the baseline¹ feature values used for computing the attributions.

¹ In this context, baseline refers to a reference feature value. In the experiments it was set to the mean value of the feature in the training set

Admission Details of Patient 24846

Current Admission Past Admissions **Feature Attributions**

Feature Attributions

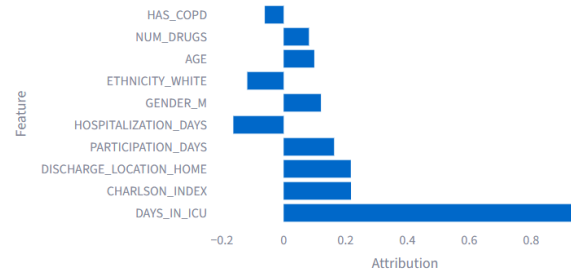
Overall Attribution — Current Admission ⓘ

1.3912

Overall Attribution — Past Admissions ⓘ

0.3733

Current Admission Feature Attributions - Top 10



Past Admissions Feature Attributions - Top 10

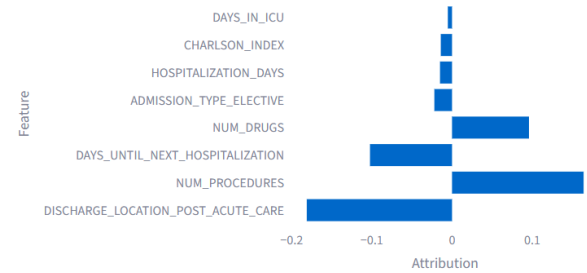


Figure 29 – IG attributions for high-risk patient (subject 24846): features in the bottom have higher absolute contributions to the predicted readmission risk of 67.3%. Plots were taken from the web application interface

Full Current Admission Feature Attributions			
Feature	Attribution	Value	Baseline Value
DAYS_IN_ICU	0.948	37.815312	2.920
CHARLSON_INDEX	0.218	9	5.002
DISCHARGE_LOCATION_HOME	0.217	False	0.544
PARTICIPATION_DAYS	0.163	912	135.513
HOSPITALIZATION_DAYS	-0.163	37.775000	7.924
GENDER_M	0.121	True	0.535
ETHNICITY_WHITE	-0.118	True	0.665
AGE	0.099	85	60.380
NUM_DRUGS	0.082	87	25.291
HAS_COPD	-0.061	True	0.369

Full Past Admissions Feature Attributions			
Feature	Attribution	Value	Baseline Value
DISCHARGE_LOCATION_POST_ACUTE_CARE	-0.181	1.000	0.351
NUM_PROCEEDURES	0.164	1.000	3.543
DAYS_UNTIL_NEXT_HOSPITALIZATION	-0.102	428.181	101.845
NUM_DRUGS	0.096	46.500	22.767
ADMISSION_TYPE_ELECTIVE	-0.022	0.000	0.077
HOSPITALIZATION_DAYS	-0.015	9.125	7.534
CHARLSON_INDEX	-0.014	7.500	4.625
DAYS_IN_ICU	-0.005	2.455	2.829

Figure 30 – IG attributions, feature values and reference feature values (baseline) for high-risk patient (subject 24846). Tables were taken from the web application interface

As explained in section 2.3.2, Captum reveals how each feature affects the final prediction by comparing the patient feature values to a training set reference and observing how changes in each feature move the model's prediction (Kokhlikyan et al., 2020). Positive contributions indicate that the patient's value pushes the probability upward, while negative contributions indicate a downward influence. The absolute magnitude of each attribution reflects the strength of that influence. There is no limitation of the range of attributions, so they can be larger than 1 or smaller than -1.

For the high-risk patient (subject 24846), features such as DAYS_IN_ICU and CHARLSON_INDEX for the the current admission have large positive attributions. This patient spent around 38 days in the ICU, compared with a training set reference of about 3 days, which strongly increases the predicted risk. A higher comorbidity score (9 compared with a training set reference of 2) also contributes positively to its readmission. Both

positive contributions make clinical sense, as longer ICU stays and higher comorbidity levels usually indicate more severe health conditions, which are commonly associated with an increased likelihood of readmission. Regarding the longitudinal features, it is noteworthy that the high value of `DAYS_UNTIL_NEXT_HOSPITALIZATION` (428 days), which measures time between past hospitalizations, compared to its training set reference (around 100 days), contributes negatively to the prediction. This suggests that a longer interval between previous admissions is associated with a lower risk of readmission, which aligns with clinical intuition.

On the other hand, some variables have non-intuitive behavior, such as hospitalization days in the current admission, which shows a negative attribution even though this patient has a longer hospital stay than the training set reference value. This behavior may reflect more complex interactions learned by the model, where the effect of length of stay depends on other clinical variables. It is also possible that some noise or instability in the model’s attributions influences this result. This case indicates that the explanation of IG attributions should be done carefully, considering the broader clinical context, and may be imprecise for certain features.

In Figures 31 and 32, the IG attributions plots and tables for the low-risk patient (subject 29810) are presented.

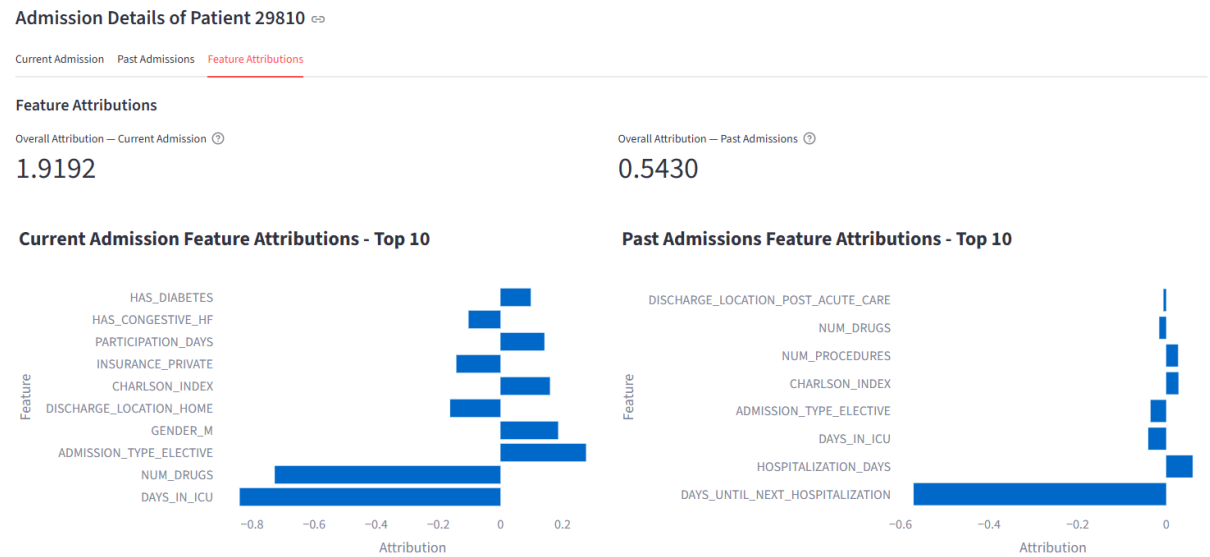


Figure 31 – IG attributions for low-risk patient (subject 29810): features in the bottom have higher absolute contributions to the predicted readmission risk of 6.4%. Plots were taken from the web application interface

Full Current Admission Feature Attributions			
Feature	Attribution	Value	Baseline Value
DAYS_IN_ICU	-0.840	0.000000	2.920
NUM_DRUGS	-0.727	0	25.291
ADMISSION_TYPE_ELECTIVE	0.275	True	0.090
GENDER_M	0.186	True	0.535
DISCHARGE_LOCATION_HOME	-0.162	True	0.544
CHARLSON_INDEX	0.159	8	5.002
INSURANCE_PRIVATE	-0.142	True	0.196
PARTICIPATION_DAYS	0.142	823	135.513
HAS_CONGESTIVE_HF	-0.103	False	0.502
HAS_DIABETES	0.098	False	0.450

Full Past Admissions Feature Attributions			
Feature	Attribution	Value	Baseline Value
DAYS_UNTIL_NEXT_HOSPITALIZATION	-0.571	817.566	101.845
HOSPITALIZATION_DAYS	0.060	5.434	7.534
DAYS_IN_ICU	-0.041	1.878	2.829
ADMISSION_TYPE_ELECTIVE	-0.035	1.000	0.077
CHARLSON_INDEX	0.028	8.000	4.625
NUM_PROCEDURES	0.027	5.000	3.543
NUM_DRUGS	-0.016	31.000	22.767
DISCHARGE_LOCATION_POST_ACUTE_CARE	-0.006	0.000	0.351

Figure 32 – IG attributions, feature values and reference values (baseline) for low-risk patient (subject 29810). Tables were taken from the web application interface

For the low-risk patient (subject 29810), some features contribute strongly with negative attributions. For example, in the current admission, this patient had zero ICU days and no drugs prescribed, which is expected to indeed decrease the readmission risk. The time between past hospitalizations of this patient (more than 800 days) is also very high compared to the training set reference, what causes a negative contribution. Few features, such as the `CHARLSON_INDEX` with a high relatively value, contribute positively but only with small magnitudes, so they do not overcome the overall negative contributions.

6.7.2 Example of Attention Weights Over Time

In this section, an example is provided to illustrate how the attention weights computed by the `SimpleAttnPoolCurrQuery` architecture can help explain the model's focus on different time steps in a patient's hospitalization history.

Figure 34 shows an example of the attention weights assigned to the last four admissions of a patient (subject 2187) whose clinical condition worsened over time, with increasing comorbidity burden as shown in Figure 33. This patient reached the limit of the four-admission observation window, allowing for a more detailed analysis of how the model distributes attention across earlier visits.

Evolution of SUBJECT_ID 2187



Figure 33 – Evolution of selected clinical features across the last four admissions

Attention over the last admissions - Subject ID: 2187

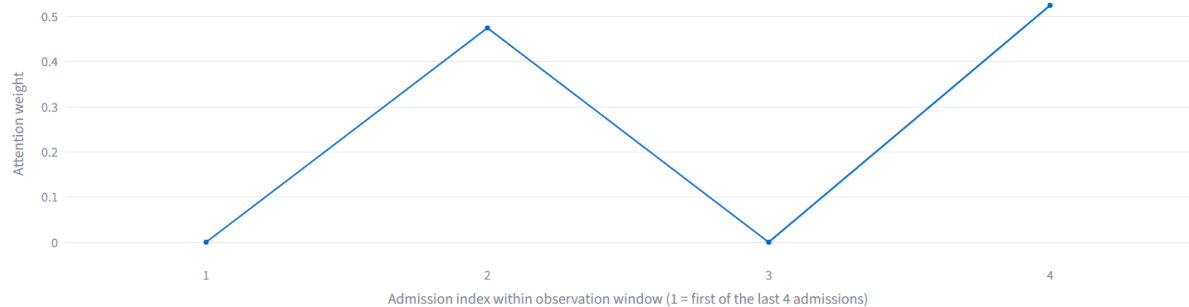


Figure 34 – Attention weights assigned to the last four admissions of the patient

In Figure 33, the second admission displays the longest hospitalization period and a two-point increase in the CCI, suggesting a deterioration in the patient’s health status. The fourth admission is the most recent and is followed by a very short interval until the next hospitalization; it also involves a long hospital stay and a one-point increase in the CCI. These characteristics may help explain why the model assigns greater relevance (higher attention) to these two admissions.

In contrast, admissions 1 and 3 receive attention values close to zero. This likely happens because they show shorter hospitalization durations, lower medication counts, and stable CCI values without increases, what results in weaker signals of clinical instability and therefore lower contribution to the model’s prediction.

6.8 API Response Times

As described in Table 2, the system must meet specific computational performance targets regarding response times for both single-patient explanations and cohort predictions:

1. NFR-01: Explanation Time (Single Patient) - Response time ≤ 5 s
2. NFR-02: Prediction Time (Cohort) - ≤ 10 s for 1,000 patients

To verify these non-functional requirements (NFR-01 and NFR-02), API calls to the corresponding endpoints (i.e. POST `/explain_single_patient` and POST `/predict` described in section 5.2.12) were performed using sample data. The hardware used for such tests is the one described in section 4.2.

First, each endpoint was tested once to confirm that it returned a successful HTTP 200 response, ensuring that the services were operational. Then, the response time was measured over 10 consecutive calls. For each endpoint, the average response time and standard deviation were computed to assess performance stability.

```

Explain Single Patient - Response Time

url = "http://127.0.0.1:8000/explain_single_patient"
headers = {"Content-Type": "application/json"}

response = requests.post(url, headers=headers, data=json.dumps(request_payload_single_patient))

print("Response status code:", response.status_code)
response.json()["metadata"]
✓ 0.5s Python

Response status code: 200

{'model_name': 'Attention Pooling with Current-Visit Query',
 'number_of_predictions': 1,
 'timestamp': '2025-12-02T16:22:16.478930',
 'prob_threshold': 0.2216}

print("Response time '/explain_single_patient' endpoint for single patient:")
%timeit -n 1 -r 10 response = requests.post(url, headers=headers, data=json.dumps(request_payload_single_patient))
✓ 3.1s Python

Response time '/explain_single_patient' endpoint for single patient:
The slowest run took 4.11 times longer than the fastest. This could mean that an intermediate result is being cached.
317 ms ± 170 ms per loop (mean ± std. dev. of 10 runs, 1 loop each)

```

Figure 35 – API response time for single-patient explanation endpoint (POST `/explain_single_patient`). The average response time over 10 calls is shown, along with the standard deviation

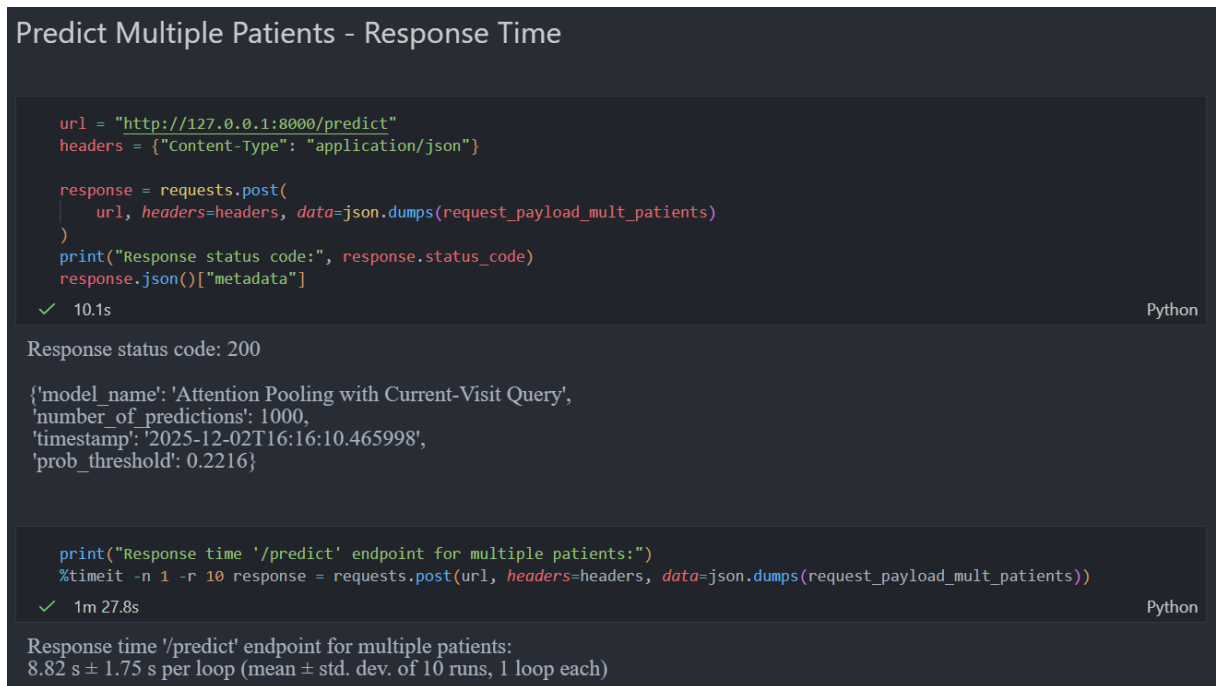


Figure 36 – API response time for cohort prediction endpoint (POST `"/predict"`). The average response time over 10 calls is shown, along with the standard deviation

As shown in Figures 35 and 36, the observed execution times (317 ± 170 ms and 8.82 ± 1.75 s) for both the single-patient explanation and the cohort prediction (1,000 patients) meet the specified performance requirements with the specific hardware defined in section 4.2.

6.9 Ease-of-Use of the Web Application

Non-functional requirement NFR-03 from Table 2 specifies that the user interface should be intuitive, provide clear instructions, maintain consistent visual elements, and require minimal steps to complete core tasks. The implemented web application, described in section 5.2.13, satisfies this requirement within the scope and priorities of the project.

Ease of use is supported by three main aspects of the interface's design. First, the sidebar clearly indicates which files must be uploaded and provides short descriptions of their expected content. Second, hover-based help messages and captions are available throughout the application, offering explanations for key components such as performance metrics, ranking elements, and visualizations. Third, the interaction flow requires only a small number of actions: after uploading the necessary files, a single click initiates the prediction process, and another click allows navigating between the results views.

Informal usability verification was conducted during the project by the developer, and the resulting interface was assessed as satisfactory given the project's time constraints and its primary emphasis on investigating the proposed attention-based modeling architec-

tures. For a more rigorous evaluation of usability, future work could involve user testing sessions with clinicians or potential end-users. These sessions would gather feedback on the intuitiveness of the interface, clarity of instructions, and overall user experience. Metrics such as task completion time, error rates, and subjective satisfaction ratings could be collected to quantitatively assess usability and identify areas for improvement.

7 Final Remarks

In this chapter, the main conclusions of the work – along with the objectives that were achieved – are presented in Section 7.1. A summary of the contributions and lessons learned is given in Section 7.2. Finally, directions for future work are discussed in Section 7.3.

7.1 Conclusion

In this work, the prediction of 30-day hospital readmission for patients with recurrent hospitalizations is addressed. The study corresponds to an extension of *Predictive Modeling of Recurrent Health Events* (Garcia, 2025b), in which the use of DT-HMMs to create compressed representations of patient trajectories was investigated. In this earlier work, it was observed that such HMM-based approach did not improve performance compared to classical machine learning models that rely on simple summary statistics of the past, such as Logistic Regression and Random Forest.

Given the lack of performance gains in the previous study, a different way of representing patient histories is investigated in this work: attention-based deep learning models. The main investigated hypothesis is whether attention mechanisms can capture better temporal dependencies and improve 30-day readmission prediction by learning which parts of the input sequence are most relevant. For testing this hypothesis, the projected was carried out following an engineering project life cycle along with applied research: it comprises a literature review to identify the state of the art and the problem to be solved, requirements specification, architecture design, implementation with software engineering and data science techniques, unit tests, E2E tests and performance analysis.

The different attention-based architectures designed and analyzed in this work were intentionally simpler than many found in the literature, in order to reduce the risk of overfitting in the dataset available in this work, which offers limited data. All attention-based models (*GRUNet*, *SimpleAttnPool*, *SimpleAttnPoolCurrQuery*, and *Multi-HeadAttnPool*) were trained and evaluated following a structured process: hyperparameter tuning, feature selection experiments, training with early stopping and evaluation on the same unseen test set. Such models were compared directly with the baseline classifiers using the same data splits and the same metrics. Because only 1,412 patients in the original training set had multiple admissions, a two-stage training procedure was also examined: in this approach, the models were first pretrained on all patients, including those with no past history, and then fine-tuned on the subset with recurrent hospitalizations.

Despite these efforts, the attention-based models did not outperform the baselines. Across experiments, both classical and deep learning approaches reached AUC values in the low to mid 0.60 range. This limited performance is consistent with the expected challenges, including the small number of patients with multiple admissions, strong class imbalance, short temporal histories, and the weak predictive power of the available features. Under such conditions, the extra modeling capacity of attention or recurrent mechanisms could not be fully used, resulting in performance similar to simpler methods.

Despite the absence of performance gains, the first two objectives of this project, namely designing simplified attention-based architectures and comparing them with classical baseline models were successfully achieved. The work provides not only a scientific contribution, by demonstrating the limitations of attention-based models in this specific setting, thereby offering valuable insights for future research, but also a successful and thorough application of a systems engineering life cycle.

The third objective of the project – developing a prototype web application – was successfully completed. The system allows users to upload hospitalization data, obtain 30-day readmission predictions, and view performance results when labels are available. It also offers explainability features, such as IG-based attributions and attention weights. Most functional requirements were met, and their verification was confirmed through the end-to-end tests described in Section 5.3.2. Only two functional requirements related to model performance (FR-11 and FR-12) were not satisfied, as they required a minimum AUC of 0.70 and superior performance to Logistic Regression for the attention-based model. These conditions were not achieved due to the performance issues discussed earlier. All non-functional requirements were satisfied: response time and ease of use. It is worth noting that ease of use can benefit from future work stemming from feedback from actual users from the medical community.

Overall, the results highlight the difficulty of predicting 30-day readmissions under the constraints of the available data. In this context, meaningful progress is likely to depend more on richer datasets than on the development of increasingly complex modeling architectures.

7.2 Summary of Contributions and Lessons Learned

The main contributions of the work and the lessons learned are summarized in this section.

Contributions

- Design, implementation, and analysis of new and simpler attention-based architectures for modeling hospitalization trajectories;
- Structured evaluation pipeline with hyperparameter tuning, feature selection, and systematic comparison against classical ML baseline models;
- Integration of explainability methods, including attention-weight visualizations and Integrated Gradients;
- Development of a FastAPI backend and a Streamlit web interface for data upload, prediction, performance inspection, and explanation;
- Exploration of a two-stage training strategy for recurrent hospital readmissions, combining pretraining on single-admission patients followed by fine-tuning on patients with multiple admissions.

Lessons Learned

- Deeper understanding of attention mechanisms through literature review;
- Introduction to Integrated Gradients of **Captum** as an explainability approach;
- Experience with hyperparameter optimization using **Optuna**;
- Improved coding and deep learning training skills with **PyTorch**;
- Hands-on experience developing APIs with **FastAPI** and web interfaces with **Streamlit**;
- Hands-on experience developing unit tests using **pytest**;
- Practice organizing and comparing experiments using **Neptune**;
- Practice of a full engineering project with applied research background;
- Strengthened ability to design experiments, analyze results, and communicate findings.

7.3 Future Work

Several directions can be pursued to extend this work and address its current limitations.

- **Collaboration with Health Experts:** involve clinicians and healthcare professionals to validate model assumptions, explainability, and practical utility;

- **Test alternative models:** Explore other sequence-modeling approaches beyond the simplified attention architectures, such as variants of Transformers or temporal convolutional networks;
- **Use larger datasets:** Incorporate more patients, longer histories, and richer clinical variables. Possible data sources include the Medical Information Mart for Intensive Care IV database ([Johnson et al., 2023](#)) or the Nationwide Readmissions Database ([Healthcare Cost and Utilization Project, 2025](#));
- **Improve features:** Develop more informative temporal and clinical features to increase the predictive signal;
- **Add clinical text:** Use large language models to extract information from notes and combine it with structured data in order to possibly improve predictions;
- **Refine explainability:** Evaluate additional explanation methods and their usefulness to clinicians;
- **Extend the system:** Expand the web interface and backend to support larger datasets and more features;
- **Ease-of-use opinion survey:** Conduct user surveys to evaluate the ease of use regarding intuitiveness and usability of the interface. Possible improvements based on the feedback could be applied.

References

- AKIBA, T. et al. Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Association for Computing Machinery, p. 2623–2631, 7 2019. Available at: <https://arxiv.org/pdf/1907.10902>.
- CASCARANO, A. et al. Machine and deep learning for longitudinal biomedical data: a review of methods and applications. *Artificial Intelligence Review*, Springer Nature, v. 56, p. 1711–1771, 11 2023. ISSN 15737462. Available at: <https://link.springer.com/article/10.1007/s10462-023-10561-w>.
- CHARLSON, M. E. et al. A new method of classifying prognostic comorbidity in longitudinal studies: Development and validation. *Journal of Chronic Diseases*, Elsevier, v. 40, p. 373–383, 1 1987. ISSN 0021-9681. Available at: <https://www.sciencedirect.com/science/article/pii/0021968187901718>.
- CHEN, T. et al. Machine learning methods for hospital readmission prediction: systematic analysis of literature. *Journal of Reliable Intelligent Environments*, Springer, v. 8, n. 1, p. 49–66, 3 2022. ISSN 2199-4676. Available at: <https://link.springer.com/article/10.1007/s40860-021-00165-y>.
- CHOI, E. et al. *RETAIN: An Interpretable Predictive Model for Healthcare using Reverse Time Attention Mechanism*. 2017. Available at: <https://arxiv.org/abs/1608.05745>.
- DHALIWAL, J. S.; DANG, A. K. *Reducing Hospital Readmissions*. Treasure Island (FL): StatPearls Publishing, 2024. Accessed on: 9 Dec. 2025. Available at: <https://www.ncbi.nlm.nih.gov/books/NBK606114/>.
- EUSTACE, S.; Poetry contributors. *Poetry: Python packaging and dependency management made easy*. 2025. Accessed on: 9 Dec. 2025. Available at: <https://python-poetry.org>.
- GARCIA, L. von A. *msc-thesis-recurrent-health-modeling*. 2025. GitHub repository. Accessed on: 9 Dec. 2025. Available at: <https://github.com/LucasVon0645/msc-thesis-recurrent-health-modeling>.
- GARCIA, L. von A. *Predictive Modeling of Recurrent Health Events: Applications to Hospital Readmission and Drug Relapse*. Darmstadt, Germany: [s.n.], 2025. Master's Thesis, Technische Universität Darmstadt, Department of Electrical Engineering and Information Technology. Supervised by Prof. Dr.-Ing. Abdelhak M. Zoubir and Dr.-Ing. Christian Debes.
- GOLDBERGER, A. L. et al. Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals. *Circulation*, v. 101, n. 23, p. e215–e220, 2000. RRID:SCR_007345.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. Accessed on: 9 Dec. 2025. ISBN 978-0262035613. Available at: <http://www.deeplearningbook.org>.

- GOOGLE. *TensorBoard: Visualizations for TensorFlow*. 2024. Accessed on: 9 Dec. 2025. Available at: <https://www.tensorflow.org/tensorboard>.
- HARRIS, C. R. et al. Array programming with NumPy. *Nature*, Springer Science and Business Media LLC, v. 585, n. 7825, p. 357–362, set. 2020. Available at: <https://doi.org/10.1038/s41586-020-2649-2>.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*: Second edition. 2. ed. New York, NY: Springer New York, 2009. 745 p. (Springer Series in Statistics). ISBN 978-0-387-84857-0.
- Healthcare Cost and Utilization Project. *NRD Overview*. 2025. Agency for Healthcare Research and Quality, Rockville, MD. Accessed on: 9 Dec. 2025. Available at: <https://hcup-us.ahrq.gov/nrdoverview.jsp>.
- HUANG, Y. et al. Application of machine learning in predicting hospital readmissions: a scoping review of the literature. *BMC Medical Research Methodology*, BioMed Central, v. 21, n. 1, p. 1–14, 12 2021. ISSN 1471-2288. Available at: <https://bmcmmedresmethodol.biomedcentral.com/articles/10.1186/s12874-021-01284-z>.
- JIANG, H. J.; HENSCHKE, M. *Characteristics of 30-Day All-Cause Hospital Readmissions*. Agency for Healthcare Research and Quality, 2023. HCUP Statistical Brief No. 304. Accessed on: 9 Dec. 2025. Available at: <https://hcup-us.ahrq.gov/reports/statbriefs/sb304-readmissions-2016-2020.jsp>.
- JOHNSON, A.; POLLARD, T.; MARK, R. *MIMIC-III Clinical Database (version 1.4)*. PhysioNet, 2016. RRID:SCR_007345. Available at: <https://doi.org/10.13026/C2XW26>.
- JOHNSON, A. E. W. et al. MIMIC-iv, a freely accessible electronic health record dataset. *Scientific Data*, v. 10, n. 1, Jan 2023. Available at: <https://doi.org/10.1038/s41597-022-01899-x>.
- JOHNSON, A. E. W. et al. MIMIC-iii, a freely accessible critical care database. *Scientific Data*, Nature Publishing Group, v. 3, p. 160035, may 2016. Available at: <https://www.nature.com/articles/sdata201635>.
- KLEINBAUM, D. G.; KLEIN, M. *Survival Analysis: A Self-Learning Text*. 3rd. ed. New York: Springer, 2012. ISBN 978-1-4419-6645-2. Available at: <https://doi.org/10.1007/978-1-4419-6646-9>.
- KOKHLIKYAN, N. et al. Captum: A unified and generic model interpretability library for pytorch. 9 2020. Available at: <https://doi.org/10.48550/arXiv.2009.07896>.
- KREKEL, H. et al. *pytest 8.4.1*. 2004. Accessed on: 9 Dec. 2025. Available at: <https://github.com/pytest-dev/pytest>.
- KUMAR, A. et al. Explainable machine learning models for clinical decision support systems. In: *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. Kamand, India: IEEE, 2024. p. 1–6.
- LUNDBERG, S.; LEE, S.-I. *A Unified Approach to Interpreting Model Predictions*. 2017. Available at: <https://arxiv.org/abs/1705.07874>.

- LUNDBERG, S.; SHAP contributors. *SHAP: Explainable machine learning*. 2025. Python library for SHAP (SHapley Additive exPlanations). Accessed on: 9 Dec. 2025. Available at: <https://github.com/shap/shap>.
- LUNDBERG, S. M. et al. Explainable machine-learning predictions for the prevention of hypoxaemia during surgery. *Nature Biomedical Engineering*, Nature Publishing Group, v. 2, n. 10, p. 749, 2018. Available at: <https://www.nature.com/articles/s41551-018-0304-0>.
- Microsoft. *Development Containers Specification*. 2025. Accessed on: 9 Dec. 2025. Available at: <https://containers.dev/>.
- Neptune Labs. *neptune.ai: Experiment tracker for foundation models*. 2025. Accessed: 9 December 2025. Available at: <https://neptune.ai>.
- Pandas development team. *pandas-dev/pandas: Pandas*. Zenodo, 2020. Accessed on: 9 Dec. 2025. Available at: <https://doi.org/10.5281/zenodo.3509134>.
- PASZKE, A. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available at: <https://arxiv.org/abs/1912.01703>.
- PEDREGOSA, F. et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. Available at: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- PERUMAL, T. et al. A comprehensive overview and comparative analysis on deep learning models. *Journal on Artificial Intelligence*, Tech Science Press, v. 6, n. 1, p. 301–360, 2024. ISSN 2579-003X. Available at: <http://dx.doi.org/10.32604/jai.2024.054314>.
- Plotly Technologies Inc. *Collaborative data science*. Montreal, QC: Plotly Technologies Inc., 2015. Accessed on: 9 Dec. 2025. Available at: <https://plot.ly>.
- RAMÍREZ, S. *FastAPI*. 2018. A modern, fast (high-performance) web framework for building APIs with Python. Accessed on: 9 Dec. 2025. Available at: <https://fastapi.tiangolo.com/>.
- SHAH, T. et al. COPD readmissions: Addressing COPD in the era of value-based health care. *Chest*, Elsevier BV, v. 150, n. 4, p. 916–926, out. 2016. Available at: <https://pubmed.ncbi.nlm.nih.gov/27167208/>.
- SIT, M. et al. A comprehensive review of deep learning applications in hydrology and water resources. *Water Science and Technology*, v. 82, 08 2020. Available at: <https://iwaponline.com/wst/article/82/12/2635/75838/A-comprehensive-review-of-deep-learning>.
- SOMMERVILLE, I. *Software Engineering*. 10. ed. Harlow, England: Pearson Education Limited, 2016. ISBN 978-1-292-09361-3.
- SONG, H. et al. *Attend and Diagnose: Clinical Time Series Analysis using Attention Models*. 2017. Available at: <https://arxiv.org/abs/1711.03905>.
- Streamlit Inc. *Streamlit: The fastest way to build data apps*. 2025. Version 1.39.0. Accessed on: 9 Dec. 2025. Available at: <https://streamlit.io>.

TWISK, J. W. R. Applied longitudinal data analysis for medical science: A practical guide. *Applied Longitudinal Data Analysis for Medical Science*, Cambridge University Press, 4 2023. Available at: <https://www.cambridge.org/core/books/applied-longitudinal-data-analysis-for-medical-science/7E6131A2F90C79AC8C99653143E89368>.

VASWANI, A. et al. *Attention Is All You Need*. 2023. Available at: <https://arxiv.org/abs/1706.03762>.

Võ, H. et al. Safe enough: Subjective determinations in hospital discharge for patients with medical complexity. *Pediatrics*, v. 155, n. 6, jun 2025. Available at: <https://pubmed.ncbi.nlm.nih.gov/40355131/>.

WALRAVEN, C. van et al. Proportion of hospital readmissions deemed avoidable: a systematic review. *CMAJ*, CMAJ, v. 183, n. 7, p. E391–E402, 2011. ISSN 0820-3946. Available at: <https://www.cmaj.ca/content/183/7/E391>.

WASKOM, M. L. seaborn: statistical data visualization. *Journal of Open Source Software*, The Open Journal, v. 6, n. 60, p. 3021, 2021. Available at: <https://doi.org/10.21105/joss.03021>.

WEISS, A. J.; JIANG, H. J. *Overview of Clinical Conditions With Frequent and Costly Hospital Readmissions by Payer, 2018*. Agency for Healthcare Research and Quality, 2021. HCUP Statistical Brief #278. Accessed on: 9 Dec. 2025. Available at: <https://pubmed.ncbi.nlm.nih.gov/34460186/>.

Appendices

APPENDIX A – Project Planning Details

Project Planning Details								
Task	Blocked by	Blocking	Category	Start Date	End Date	# Work Days	Priority	Additional Description
Project Definition		Literature Review, Requirements Spec., Study SW Tools	Preparation	1. September 2025	9. September 2025	6	High	
Project Report			Documentation	15. September 2025	9. December 2025	61	High	
Planning		Project Definition	Preparation	15. September 2025	23. September 2025	6	High	
Requirements Spec.		Project Definition	Preparation	15. September 2025	23. September 2025	6	High	
Study SW Tools 1		Project Definition	Preparation	15. September 2025	17. November 2025	45	High	Initial study of software (SW) tools for starting model implementation
Study SW Tools 2		Project Definition	Preparation	1. October 2025	7. November 2025	27	High	Additional review of software (SW) tools, if unforeseen issues or further investigation arise during the project.
System Arch		Project Definition	Preparation	15. September 2025	24. September 2025	7	High	High-level and software (SW) architecture
Literature Review 1		Project Definition	Preparation	25. September 2025	29. September 2025	2	High	Initial literature review for starting system architecture definition and model implementation
Literature Review 2		Project Definition	Preparation	1. October 2025	7. November 2025	27	High	Further literature review if new sources or gaps are identified during the project.
Model Implementation		System Arch Model Training, API Model Inference	Analytics Backend	1. October 2025	14. October 2025	9	High	Code for implementing model architecture, training and inference.
Unit Tests			Test	8. October 2025	30. October 2025	16	Medium	Performed during analytics backend development
Integration Tests			Test	7. November 2025	17. November 2025	6	Medium	Performed during frontend development
Model Training		Model Implementation	Analytics Backend	16. October 2025	28. October 2025	8	High	Run training of main model and baselines. Code for training and evaluation tracking with neptune.ai
API Model Inference		Model Implementation	Analytics Backend	20. October 2025	28. October 2025	6	High	Receive input data from interface and to calculate model prediction, as well as SHAP values
Model Tuning		Model Training	Analytics Backend	28. October 2025	3. November 2025	4	Medium	Code for Cross-Validation Metrics. Experiment with different hyperparameters.
Base Interface		Model Performance	Analytics Backend	5. November 2025	13. November 2025	6	High	Interface showing prediction results of multiple patients.
Model Selection		Model Tuning	Results Analysis	6. November 2025	13. November 2025	5	High	
Features' Interpretability		Base Interface	Frontend	13. November 2025	18. November 2025	3	Medium	Add to basic interface ability to see overall SHAP values of patients or those for specific one.
Performance Comparison		Model Selection	Results Analysis	14. November 2025	24. November 2025	6	High	Compare baseline with final best selected model
Patient Data Visualization		Base Interface	Frontend	17. November 2025	18. November 2025	1	Medium	See the trajectory of a specific patient
Poster & Press Release		Performance Comparison	Documentation	24. November 2025	2. December 2025	6	High	
Presentation Video		Performance Comparison	Documentation	1. December 2025	10. December 2025	7	High	

Figure 37 – Description of project tasks present in the Gantt chart in Figure 2

APPENDIX B – Python Packages Used in the Project

Table 17 – Main libraries used in this project

Library	Version	Description	Purpose
PyTorch (Paszke et al., 2019)	2.8.0	Deep learning framework	Development and training of the attention-based model
TensorBoard (Google, 2024)	2.20.0	Visualization toolkit for deep learning	Monitor training progress and metrics
Optuna (Akiba et al., 2019)	4.5.0	Hyperparameter optimization framework	Automated hyperparameter tuning
scikit-learn (Pedregosa et al., 2011)	1.6.1	General ML framework	Data splitting, AUC, baseline ML models
neptune.ai (Neptune Labs, 2025)	1.14.0	Experiment tracking	Track experiments, hyperparameters, and results
shap (Lundberg; SHAP contributors, 2025)	0.48.0	Explainable AI library	Compute SHAP values
Captum (Kokhlikyan et al., 2020)	0.7.0	Model interpretability library	Compute feature attributions for PyTorch models
plotly (Plotly Technologies Inc., 2015)	6.1.1	Interactive plotting library	Interactive visualizations
seaborn (Waskom, 2021)	0.13.2	Statistical data visualization	Static plots and summaries
pandas (Pandas development team, 2020)	2.2.3	Data analysis library	Data manipulation and preprocessing
numpy (Harris et al., 2020)	2.2.6	Numerical computation library	Array operations and numerical preprocessing
pytest (Krekel et al., 2004)	8.4.1	Testing framework for Python	Automated testing of model components and feature engineering functions
Streamlit (Streamlit Inc., 2025)	1.39.0	Web application framework	Prototype frontend interface
FastAPI (Ramírez, 2018)	0.120.0	Web framework for building APIs	Backend API implementation

APPENDIX C – Features Description

The following Tables 18 and 19 list and describe the features produced by the data preprocessing pipeline described in section 5.2.3.

Table 18 – Numerical features with definitions and types

Feature	Category / Type		Description
HOSPITALIZATION_DAYS	Category:	hospital-level	Total length of stay for the current hospitalization.
	Type:	Numerical-continuous	
DAYS_IN_ICU	Category:	hospital-level	Total ICU time during the hospital stay.
	Type:	Numerical-continuous	
CHARLSON_INDEX	Category:	hospital-level	Charlson Comorbidity Index (CCI) score of the patient during the current hospital stay.
	Type:	Numerical-discrete	
NUM_PREV_HOSPITALIZATIONS	Category:	Historical summary	Number of hospitalizations prior to the current hospital stay.
	Type:	Numerical-count	
NUM_COMORBIDITIES	Category:	hospital-level	Number of distinct comorbidity categories present in the diagnosis of the patient in the current hospital stay.
	Type:	Numerical-count	
NUM_PROCEDURES	Category:	hospital-level	Number of coded medical procedures during the current hospitalization.
	Type:	Numerical-count	
NUM_DRUGS	Category:	hospital-level	Number of distinct medications prescribed during the index hospitalization.
	Type:	Numerical-count	
DAYS_SINCE_LAST_HOSPITALIZATION	Category:	Time-related	Days from the previous discharge to the current admission.
	Type:	Numerical-continuous	
READM_30_DAYS_PAST_MEAN	Category:	Historical summary	Proportion of prior admissions followed by a readmission within 30 days.
	Type:	Numerical-continuous	
READM_30_DAYS_PAST_SUM	Category:	Historical summary	Count of prior readmissions that occurred within 30 days.
	Type:	Numerical-count	
DAYS_UNTIL_NEXT_HOSP_PAST_MEDIAN	Category:	Historical summary	Median interval between consecutive past hospitalizations.
	Type:	Numerical-continuous	
DAYS_UNTIL_NEXT_HOSP_PAST_STD	Category:	Historical summary	Standard deviation of intervals between consecutive past hospitalizations.
	Type:	Numerical-continuous	
AGE	Category:	Demographic	Age at admission (derived from date of birth).
	Type:	Numerical-continuous	

Note: Some numerical continuous features were scaled using a $\log(1 + x)$ transformation. A LOG_ prefix indicates such transformation.

Table 19 – Categorical and binary features with definitions and types

Feature	Category / Type		Description
GENDER	Category:	Demographics	Recorded patient gender: Male or Female.
	Type:	Categorical (nominal)	
INSURANCE	Category:	Administrative	Insurance at admission: Medicare, Private, Medicaid, Government, or Self Pay.
	Type:	Categorical (nominal)	
ADMISSION_TYPE	Category:	Administrative	Admission type at index admission: Emergency, Urgent, or Elective.
	Type:	Categorical (nominal)	
ETHNICITY	Category:	Demographics	Recorded ethnicity: White, Black, Hispanic, or Others. Original categories were grouped into these smaller sets.
	Type:	Categorical (nominal)	
DISCHARGE_LOCATION	Category:	Administrative	Disposition at discharge: Home, Post-Acute Care, or Others. Original categories were grouped into these smaller sets.
	Type:	Categorical (nominal)	
HAS_DIABETES	Category:	Comorbidity indicator	Presence of diabetes based on ICD codes.
	Type:	Binary (0/1)	
HAS_COPD	Category:	Comorbidity indicator	Presence of chronic obstructive pulmonary disease based on ICD codes.
	Type:	Binary (0/1)	
HAS_CONGESTIVE_HF	Category:	Comorbidity indicator	Presence of congestive heart failure based on ICD codes.
	Type:	Binary (0/1)	

Note: Categorical values were one-hot encoded. The category names are added as suffixes to the feature names, separated by an underscore. For example, the feature `DISCHARGE_LOCATION_HOME` indicates whether the discharge location was *HOME* (1) or not (0).

APPENDIX D – Web Application Sequence Diagrams

A sequence diagram is a type of Unified Modeling Language diagram that illustrates how objects interact in a system to perform a specific function or process over time. It shows the sequence of messages exchanged between different components or actors in the system.

The following two sequence diagrams give a high-level view of how the system handles batch cohort prediction and patient-level explainability. They show the main components involved (Streamlit interface, FastAPI backend, prediction pipeline, model, and explainability engine) and how they exchange information during these two workflows. Figure 38 is associated to the use cases *"Explore Cohort Predictions"* and *"View Cohort Model Performance"*, while Figure 39 corresponds to the *"View Patient Explainability and Trajectory"* use case. These use cases are described in section 5.2.13.

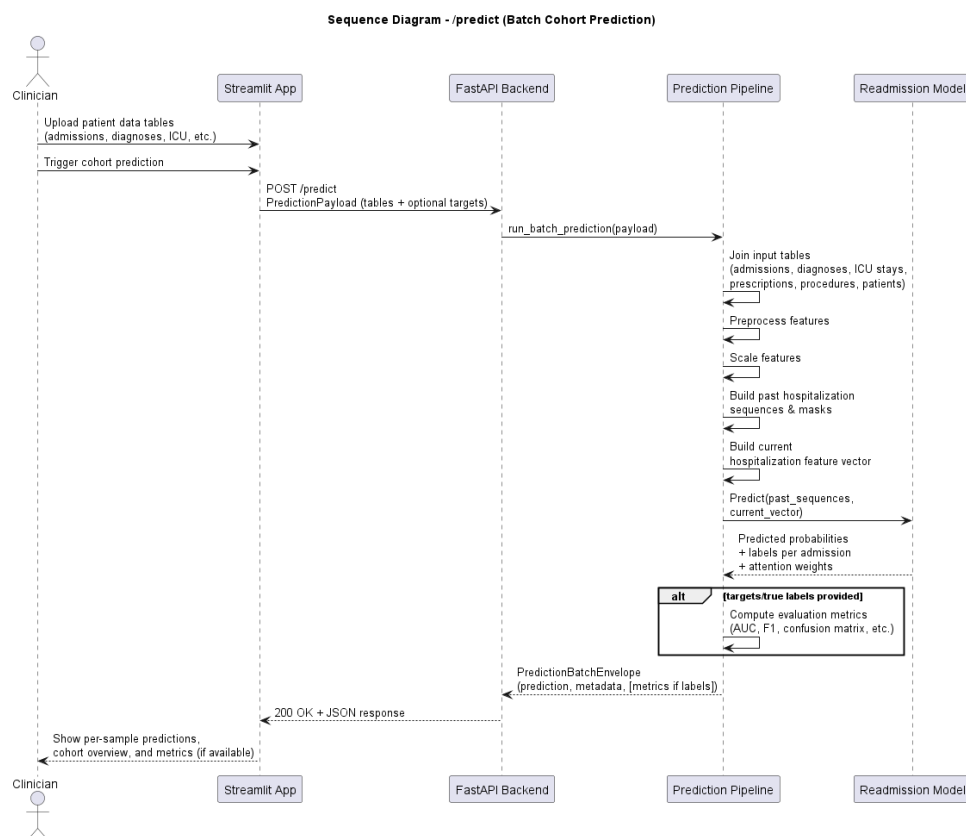


Figure 38 – Sequence diagram for the *"Explore Cohort Predictions"* and *"View Cohort Model Performance"* use cases

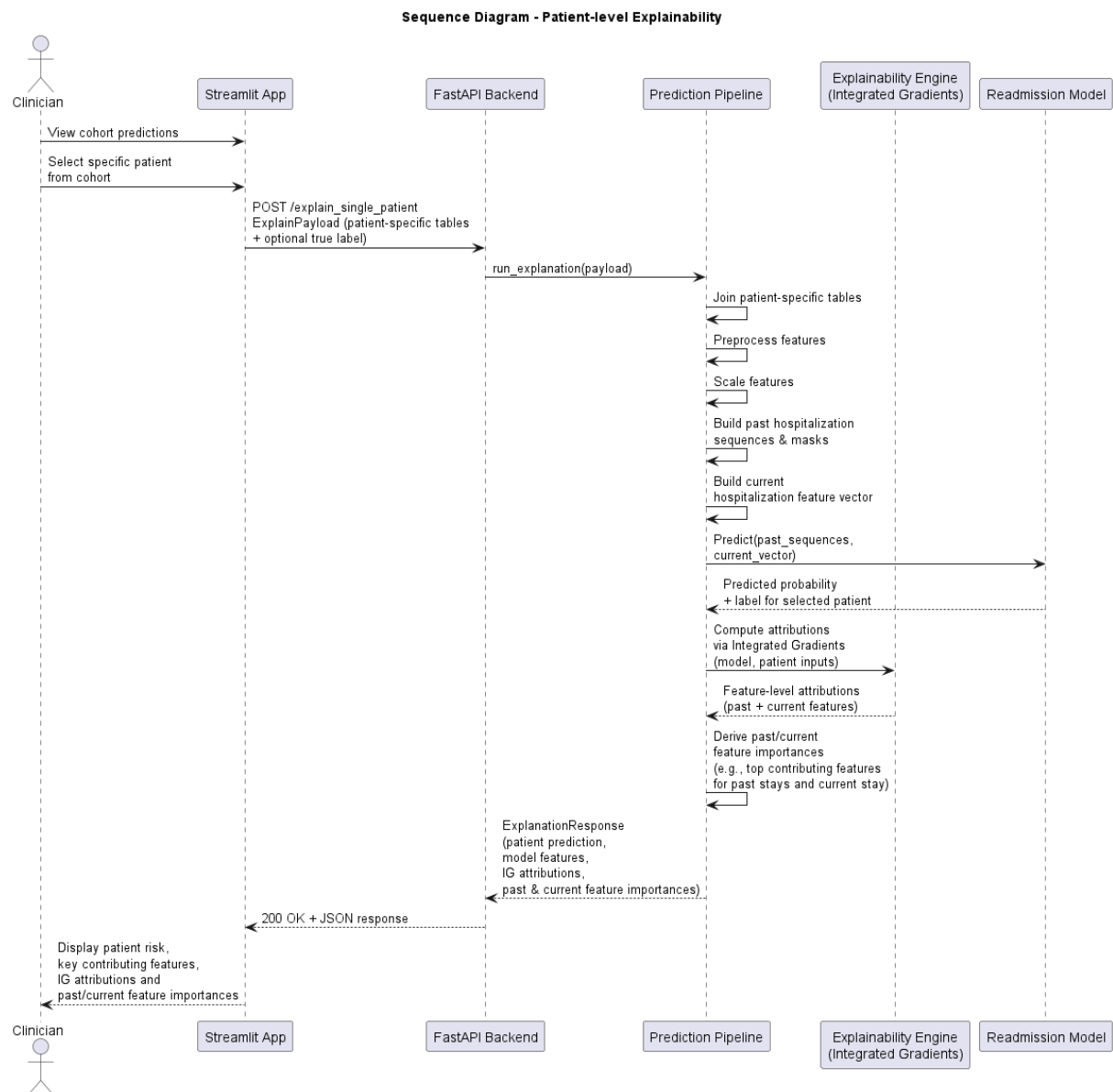


Figure 39 – Sequence diagram for the "Explain Single Patient" use case

APPENDIX E – Further Details of API Model Performance

This appendix provides additional evaluation results for the final model of the architecture `SimpleAttnPoolCurrQuery`, complementing the results presented in Table 15 of section 6.4. The results were obtained with the last events test set reported in that section. In Figure 40 the confusion matrix is depicted. The calibration curve is shown in Figure 41, while the distribution of the predicted probabilities by true label is presented in Figure 42.

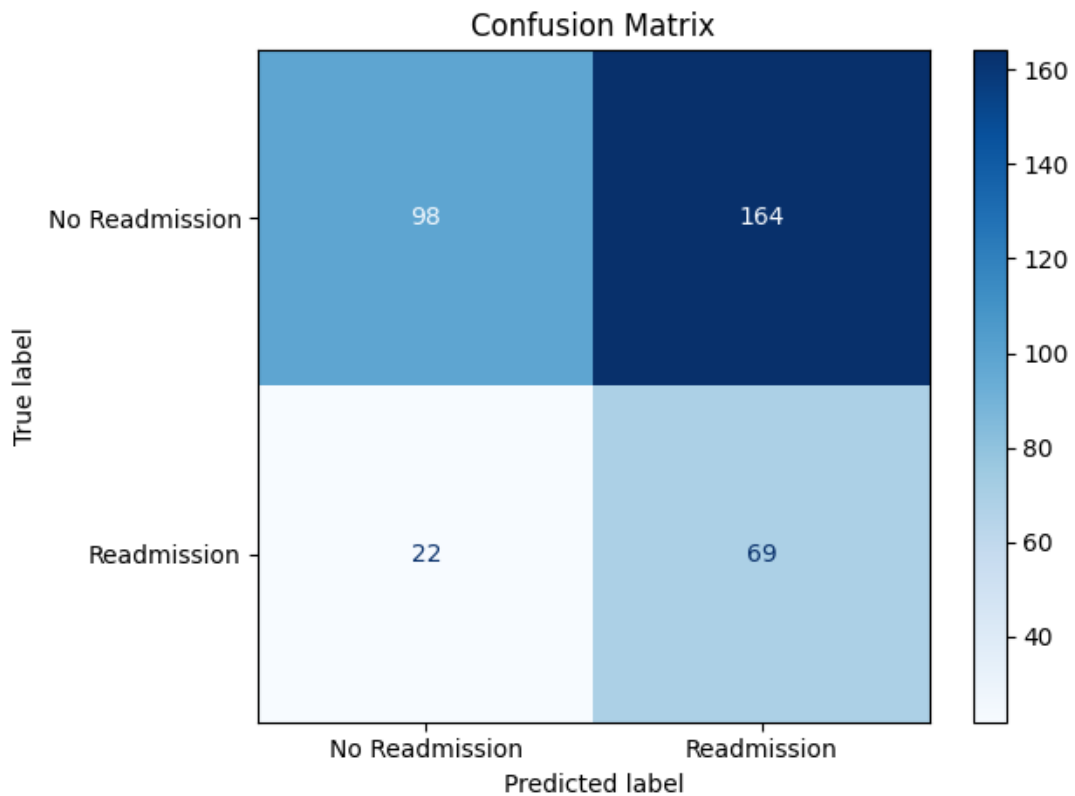


Figure 40 – Confusion matrix for the model used by the API on test data

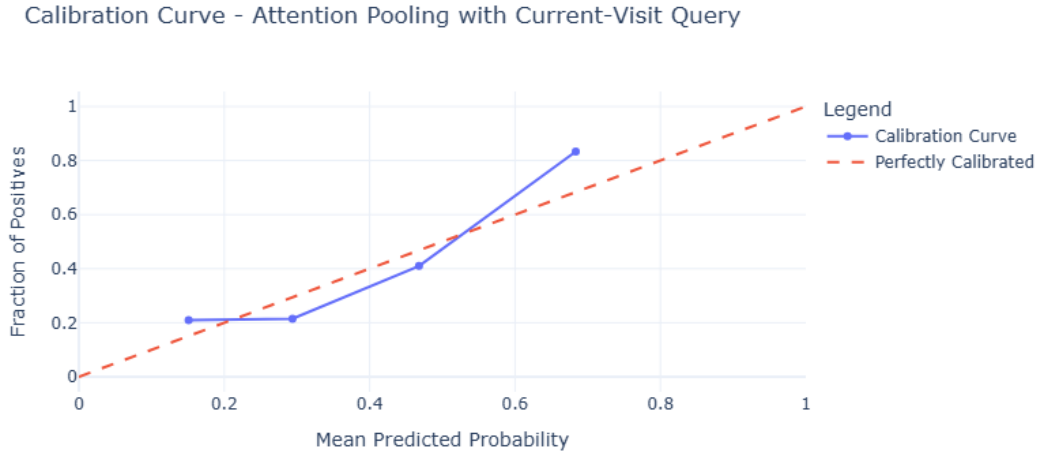


Figure 41 – Calibration curve for the model used by the API on test data

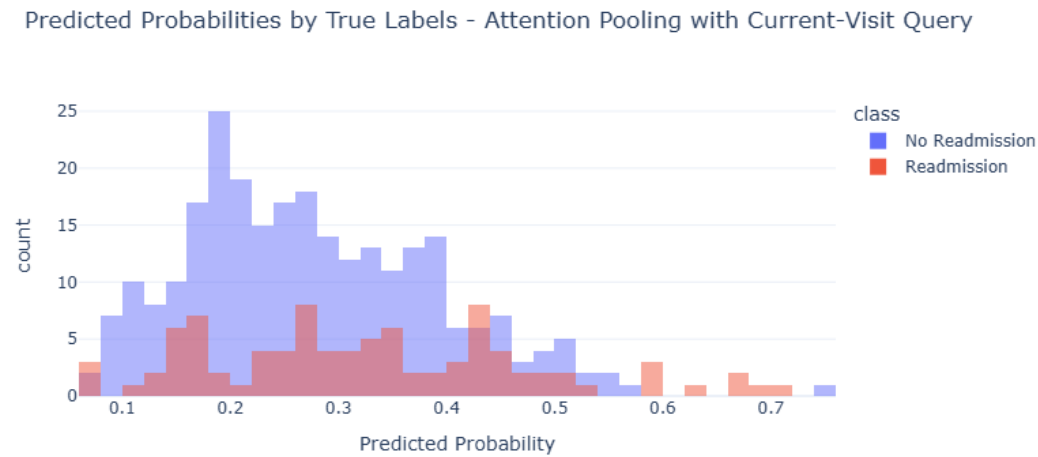


Figure 42 – Predicted probability distribution for the model used by the API on test data

For predictions in the confusion matrix of Figure 40, a probability threshold of 0.22 is used, giving a precision of 0.30 and a recall of 0.76 for the positive class. This matches the low AUC and F1 reported in Table 15, indicating that the model has limited discriminative power on unseen data.

The calibration curve in Figure 41 shows that the model is reasonably well-calibrated, with predicted probabilities closely matching observed readmission rates across most bins. However, there is some deviation at higher predicted probabilities, for which the model tends to underestimate the risk. Such underestimation of the risk is likely due to the major dominance of the negative class (non-readmissions) in the dataset, which biases the model towards lower probability outputs.

In the predicted probability distribution of Figure 42, non-readmissions are more concentrated at lower predicted probabilities, as expected. However, true readmissions show

a flatter distribution across probability values, contributing to the model’s low precision (0.30). This explains why the threshold of 0.22 is needed to achieve a high recall (0.76), as many true readmissions receive low predicted probabilities.

APPENDIX F – Hospital Readmission Prediction API Specification

In this appendix, the specification of the *Hospital Readmission Prediction API* is presented. This API provides hospital readmission risk predictions and explanations using machine learning techniques.

Overview

The API provides predictions and explanations of 30-day hospital readmission risk.

Endpoints

GET /health Simple API health-check endpoint. Returns an empty JSON object {} on success.

POST /predict Batch prediction endpoint for multiple patient admissions.

POST /explain_single_patient Endpoint returning detailed feature-attribution explanations for a single patient.

All endpoints may return the following errors:

- 422: validation error (missing required fields).
- 500: internal server error (prediction failure).

POST /predict Endpoint

Summary

Function: Batch prediction for patient data.

Description: Receives multiple tables associated with patient information and returns model predictions.

Request body (JSON): A PredictionPayload object.

The `PredictionPayload` object contains six required tables and one optional label table (`targets`). Each table is a list of objects (rows) with mandatory fields.

Root Object: PredictionPayload

Table 20 – API specification – `PredictionPayload` fields

Field	Type	Description
<code>admissions</code>	<code>array<AdmissionsRow></code>	Hospital admissions.
<code>diagnoses</code>	<code>array<DiagnosesRow></code>	Diagnoses linked to admissions.
<code>icu_stays</code>	<code>array<ICUStayRow></code>	ICU stay intervals.
<code>prescriptions</code>	<code>array<PrescriptionRow></code>	Medication prescriptions.
<code>procedures</code>	<code>array<ProcedureRow></code>	Performed procedures.
<code>patients</code>	<code>array<PatientRow></code>	Patient demographic data.
<code>targets</code>	<code>array<TargetRow></code> or null	Optional labels for evaluation.

Row Structures

AdmissionsRow

Table 21 – API specification – `AdmissionsRow` fields

Field	Type	Description
<code>HADM_ID</code>	integer	Admission identifier.
<code>SUBJECT_ID</code>	integer	Patient identifier.
<code>ADMITTIME</code>	string	Admission timestamp.
<code>DISCHTIME</code>	string	Discharge timestamp.
<code>ADMISSION_TYPE</code>	string	Type of admission.
<code>INSURANCE</code>	string	Insurance category.
<code>ETHNICITY</code>	string	Patient ethnicity.
<code>DISCHARGE_LOCATION</code>	string	Post-discharge destination.

DiagnosesRow

Table 22 – API specification – `DiagnosesRow` fields

Field	Type
<code>HADM_ID</code>	integer
<code>SUBJECT_ID</code>	integer
<code>ICD9_CODE</code>	string

ICUStayRow

Table 23 – API specification – `ICUStayRow` fields

Field	Type
<code>SUBJECT_ID</code>	integer
<code>HADM_ID</code>	integer
<code>INTIME</code>	string
<code>OUTTIME</code>	string

PrescriptionRowTable 24 – API specification – **PrescriptionRow** fields

Field	Type
HADM_ID	integer
SUBJECT_ID	integer
DRUG	string

PatientRowTable 25 – API specification – **PatientRow** fields

Field	Type
SUBJECT_ID	integer
GENDER	string
DOB	string

ProcedureRowTable 26 – API specification – **ProcedureRow** fields

Field	Type
HADM_ID	integer
SUBJECT_ID	integer
ICD9_CODE	string

TargetRow (optional)Table 27 – API specification – **TargetRow** fields

Field	Type
HADM_ID	integer
SUBJECT_ID	integer
READMISSION_30_DAYS	integer

Response: 200 OK

A successful response to `/predict` follows the **PredictionBatchEnvelope** schema.

Root Fields of PredictionBatchEnvelopeTable 28 – API specification – **PredictionBatchEnvelope** fields

Field	Type	Description
<code>prediction</code>	PredictionBody	Prediction vectors and IDs.
<code>metadata</code>	PredictionMetadata	Model execution metadata.
<code>metrics</code>	PredictionMetrics	Evaluation metrics (if <code>targets</code> were provided).

PredictionBody

Table 29 – API specification – PredictionBody fields

Field	Type	Description
pred_probs	array<float>	Predicted probabilities.
pred_labels	array<integer>	Predicted class labels.
true_labels	array<integer> or null	True labels provided by caller.
attention_weights	array of arrays<float> or null	Model attention values.
hadm_ids	array or null	Ordered admission identifiers.
subject_ids	array or null	Ordered patient identifiers.

PredictionMetadata

Table 30 – API specification – PredictionMetadata fields

Field	Type	Description
model_name	string	Model identifier.
number_of_predictions	integer	Total number of predictions.
timestamp	string	Prediction timestamp.
prob_threshold	float	Probability threshold used.

PredictionMetrics

Table 31 – API specification – PredictionMetrics (optional)

Metric	Type
accuracy	float or null
precision	float or null
recall	float or null
f1_score	float or null
auc_roc	float or null
confusion_matrix	array<array<integer> or null

Metrics are returned only if `targets` is provided in the `PredictionPayload`.

POST /explain_single_patient Endpoint

Summary

Function: Explain the model’s predictions for a single patient.

Description: Uses the same request body as `/predict`, but returns a feature-attribution object and derived features.

Request body: Identical to `/predict` (`PredictionPayload`).

Response: 200 OK

The response follows the `ExplainSinglePatientEnvelope` schema.

Root Fields of `ExplainSinglePatientEnvelope`Table 32 – API specification – `ExplainSinglePatientEnvelope` fields

Field	Type	Description
<code>explanation</code>	<code>ExplanationBody</code>	Feature attribution values.
<code>input_features</code>	<code>InputFeatures</code>	Derived features used by the model.
<code>metadata</code>	<code>PredictionMetadata</code>	Same metadata as in <code>/predict</code> .

`InputFeatures`Table 33 – API specification – `InputFeatures` fields

Field	Type	Description
<code>past</code>	<code>array<object></code> or null	Features from past admissions.
<code>current</code>	<code>object</code>	Current admission features.

`ExplanationBody`Table 34 – API specification – `ExplanationBody` fields

Field	Type	Description
<code>feature_attribution_split</code>	<code>FeatureAttributionSplit</code>	Impact of past vs. current features.
<code>past_features_attributions</code>	<code>array<FeatureAttributionRow></code>	Longitudinal feature attributions.
<code>current_features_attributions</code>	<code>array<FeatureAttributionRow></code>	Current feature attributions.

`FeatureAttributionSplit`Table 35 – API specification – `FeatureAttributionSplit` fields

Field	Type
<code>past_attribution</code>	float
<code>current_attribution</code>	float

`FeatureAttributionRow`Table 36 – API specification – `FeatureAttributionRow` fields

Field	Type
<code>feature</code>	string
<code>attribution</code>	number